Version of Record: https://www.sciencedirect.com/science/article/pii/S1383762123001698
Manuscript_ac64e50cf56f6ebb80b6407e21c889d2

Journal of Systems Architecture

A Survey of Techniques for Optimizing Transformer Inference

--Manuscript Draft--

Manuscript Number:	JSA-D-23-00789R1
Article Type:	Review Article
Keywords:	Transformers; GPT; Vision Transformers; Hardware Acceleration; Neural Architecture Search
Corresponding Author:	Sparsh Mittal Indian Institute of Technology Roorkee Roorkee, Uttarakhand INDIA
First Author:	Krishna Teja Chitty-Venkata
Order of Authors:	Krishna Teja Chitty-Venkata
	Sparsh Mittal
	Murali Emani
	Venkatram Vishwanath
	Arun K. Somani
Abstract:	Recent years have seen a phenomenal rise in performance and applications of transformer neural networks. The family of transformer networks, including Bidirectional Encoder Representations from Transformer (BERT), Generative Pretrained Transformer (GPT) and Vision Transformer (ViT), have shown their effectiveness across Natural Language Processing (NLP) and Computer Vision (CV) domains. Transformer-based networks such as ChatGPT have impacted the lives of common men. However, the quest for high predictive performance has led to an exponential increase in transformers' memory and compute footprint. Researchers have proposed techniques to optimize transformer inference at all levels of abstraction. This paper presents a comprehensive survey of techniques for optimizing the inference phase of transformer networks. We survey techniques such as knowledge distillation, pruning, quantization, neural architecture search and lightweight network design at the algorithmic level. We further review hardware-level optimization techniques and the design of novel hardware accelerators for transformers. We summarize the quantitative results on the number of parameters/FLOPs and accuracy of several models/techniques to showcase the tradeoff exercised by them. We also outline future directions in this rapidly evolving field of research. We believe this survey will educate both novice and seasoned researchers and also spark a plethora of research efforts in this field.
Suggested Reviewers:	Jihao Liu jihaoliu@link.cuhk.edu.hk
	Haichao Zhang dhu_zhc@mail.dhu.edu.cn
Response to Reviewers:	Our response to reviewers is attached as a separate PDF.

Powered by Editorial Manager® and ProduXion Manager® from Aries Systems Corporation

© 2023 published by Elsevier. This manuscript is made available under the Elsevier user license
https://www.elsevier.com/open-access/userlicense/1.0/

RESPONSE TO REVIEWER COMMENTS

Editor: The paper needs improvements in grammar. You can convert PDF into DOCX format by opening the PDF file with WORD, then check grammar with Grammarly.https://www.grammarly.com/office-addin/downloading

We have now rigorously checked our paper using grammarly.

REVIEWER #1:

The paper is very well written and it comprehensively covers several types of optimizations for Transformer inference. It discusses several recent works. There are only minor comments to address:

We thank the reviewer for the comments.

1) Please compare this work with other related surveys on Transformer inference optimizations, such as:

I) Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient Transformers: A Survey," ACM Comput. Surv., vol. 55, no. 6, pp. 109:1-109:28, 2023.

II) S. Kim, C. Hooper, T. Wattanawong, M. Kang, R. Yan, H. Genc, G. Dinh, Q. Huang, K. Keutzer, M. W. Mahoney, Y. S. Shao, and A. Gholami, "Full Stack Optimization of Transformer Inference: A Survey," CoRR, vol. abs/2302.14017, 2023.

We have now compared our survey with other related surveys. Specifically, we have written the following in the introduction section.

Comparison with related surveys: Our paper is significantly different from earlier works in terms of the variety of methods discussed and in-depth analysis of optimization techniques for transformer inference. Tay et al. [55] limit their coverage to architectural innovations for minimizing computation overheads, e.g., minimizing the quadratic complexity, parameter sharing for low model size, and methods related to sparsity and pooling. Fournier et al. [56] provide a high-level summary of optimizations such as pruning, quantization, NAS, knowledge distillation, hashing, low-rank factorization and sparse attention, and then focus on transformer-specific techniques. They do not discuss hardware architectures for transformers.

The survey by Kim et al. [57] focuses primarily on pruning, quantization, and NAS. However, it has only a limited focus on hardware architecture methods. It also lacks classification or taxonomy for the techniques described in their paper. Their paper discusses both CNN and transformer methods, applying CNN-related methods to Transformers. It does not cover lightweight architecture design, knowledge distillation and binarization methods. Unlike their paper, our review article exclusively concentrates on transformers and also summarizes vision transformer-related pruning and quantization methods. Further, different from all these surveys, our survey also provides figures that summarize the quantitative characteristics of well-known transformer models (refer Figures 1, 10, 11, 21, 29 and 31). By providing quantitative comparison of attributes of various categories of transformer networks, these figures will provide significant insights to the readers.

2) Please extend the section on hardware accelerators for Transformers by discussing complete Transformer engines such as: I) X. Yang and T. Su, "EFA-Trans: An Efficient and Flexible Acceleration Architecture for Transformers," Electronics, vol. 11, no. 21, 2022. II) A. Marchisio, D. Dura, M. Capra, M. Martina, G. Masera, and M. Shafique, "SwiftTron: An Efficient Hardware Accelerator for Quantized Transformers," CoRR abs/2304.03986, 2023. III) Z. Liu, G. Li, and J. Cheng, "Hardware Acceleration of Fully Quantized BERT for Efficient Natural Language Processing," in Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 513-516, IEEE, 2021.

We have now referenced all these research works in our survey. Also, we have summarized two of these methods in the section on quantization. Specifically, we have written the following:

SwiftTron [154] is an accelerator designed for quantized networks that supports the common transformer operations. On a high-level, SwiftTron hardware consists of following units which are executed in integer precision: (i) MHSA unit for attention and softmax computation, (ii) FFN unit for MLP and GELU computation, and (iii) LayerNorm unit for element-wise multiplication and square root operation. Figure 40 depicts the core structure of the accelerator which consists of five MatMul blocks, each one for QKV, QK^T and AV matrix computation. The network parameters are quantized using the traditional quantized function while a requantization circuit (Req) is introduced in the accelerator to quantize the output activations back to 8-bits from 32-bits using Dyadic Numbers concept [152]. The hardware is synthesized in a 65 nm CMOS technology and the RoBERTa-base model [155] requires only 1.83 ns, while occupying an area of 273 mm^2 and consuming 33.64 mW power.

Liu et al. [142] first propose FQ-BERT to quantize all the tensors (weight and activation matrices, including biases) and operations (softmax and normalization) and then develop an efficient accelerator to deploy the quantized BERT models. The FQ-BERT method is based on the linear symmetric quantization [44] where the quantized 4-bit weight and 8-bit activation model suffer less than 3.6% accuracy drop while attaining a compression ratio of 7.94×. The accelerator consists of processing units (PUs), a softmax core, a layer normalization core and on-chip buffers. The PU consists of several processing elements which support the multiplication of different weight and activation precisions, such as (8-bit W × 4-bit A) for XW^Q and (8-bit W × 8-bit A) for QK^T. The softmax core approximates the expensive softmax function using a lookup table instead of computing the exponential values during inference runtime. The table requires only 256 entries as the input and output activations are quantized to 8-bit integers. The layer normalization core comprises a 3-stage pipelined SIMD (single instruction multiple data) to utilize the parallelism in element-wise norm operation. The low-precision model accelerated on Xilinx ZCU102, and ZCU111 FPGA achieves a performance-per-watt improvement of 28.91× and 12.72× over Intel i7 CPU and NVIDIA K80 GPU, respectively.

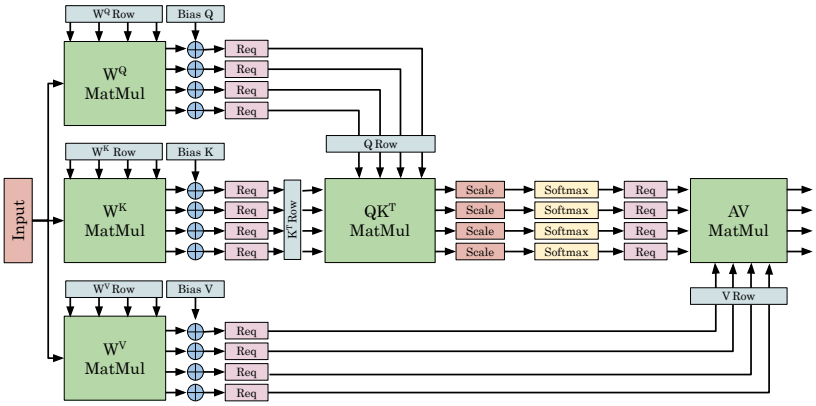


Fig. 40. Overall accelerator design in SwiftTron technique [154] (Req= requantization)

REVIEWER #2:

Please cite these papers to put this survey paper into context: A Practical Survey on Faster and Lighter Transformers https://dl.acm.org/doi/full/10.1145/3586074

Full Stack Optimization of Transformer Inference: a Survey https://arxiv.org/abs/2302.14017

We have now compared our survey with other related surveys. Specifically, we have written the following in the introduction section.

Comparison with related surveys: Our paper is significantly different from earlier works in terms of the variety of methods discussed and in-depth analysis of optimization techniques for transformer inference. Tay et al. [55] limit their coverage to architectural innovations for minimizing computation overheads, e.g., minimizing the quadratic complexity, parameter sharing for low model size, and methods related to sparsity and pooling. Fournier et al. [56] provide a high-level summary of optimizations such as pruning, quantization, NAS, knowledge distillation, hashing, low-rank factorization and sparse attention, and then focus on transformer-specific techniques. They do not discuss hardware architectures for transformers.

The survey by Kim et al. [57] focuses primarily on pruning, quantization, and NAS. However, it has only a limited focus on hardware architecture methods. It also lacks classification or taxonomy for the techniques described in their paper. Their paper discusses both CNN and transformer methods, applying CNN-related methods to Transformers. It does not cover lightweight architecture design, knowledge distillation and binarization methods. Unlike their paper, our review article exclusively concentrates on transformers and also summarizes vision transformer-related pruning and quantization methods. Further, different from all these surveys, our survey also provides figures that summarize the quantitative characteristics of well-known transformer models (refer Figures 1, 10, 11, 21, 29 and 31). By providing quantitative comparison of attributes of various categories of transformer networks, these figures will provide significant insights to the readers.

Manuscript File Click here to view linked References

A Survey of Techniques for Optimizing Transformer Inference

Krishna Teja Chitty-Venkata[1]⋆, Sparsh Mittal[2]⋆, Murali Emani[3],
Venkatram Vishwanath[3] and Arun K. Somani[1]
[1] Iowa State Univeristy, Ames, IA, USA
[2] Indian Institute of Technology Roorkee, Uttarakhand, India
[3] Argonne National Laboratory, Lemont, IL, USA
krishnat@iastate.edu, sparsh.mittal@ece.iitr.ac.in, memani@anl.gov, venkat@anl.gov, arun@iastate.edu

***Abstract*—Recent years have seen a phenomenal rise in the performance and applications of transformer neural networks. The family of transformer networks, including Bidirectional Encoder Representations from Transformer (BERT), Generative Pretrained Transformer (GPT) and Vision Transformer (ViT), have shown their effectiveness across Natural Language Processing (NLP) and Computer Vision (CV) domains. Transformer-based networks such as ChatGPT have impacted the lives of common men. However, the quest for high predictive performance has led to an exponential increase in transformers' memory and compute footprint. Researchers have proposed techniques to optimize transformer inference at all levels of abstraction. This paper presents a comprehensive survey of techniques for optimizing the inference phase of transformer networks. We survey techniques such as knowledge distillation, pruning, quantization, neural architecture search and lightweight network design at the algorithmic level. We further review hardware-level optimization techniques and the design of novel hardware accelerators for transformers. We summarize the quantitative results on the number of parameters/FLOPs and the accuracy of several models/techniques to showcase the tradeoff exercised by them. We also outline future directions in this rapidly evolving field of research. We believe that this survey will educate both novice and seasoned researchers and also spark a plethora of research efforts in this field.**

***Index Terms*—Transformers, Self-attention, BERT, GPT, Vision Transformers, Hardware Acceleration, Pruning, Quantization, Neural Architecture Search, Knowledge Distillation, ASIC, FPGA, GPU, CPU**

I. Introduction

Artificial intelligence (AI) has achieved tremendous success in a wide range of applications due to its automatic representation capability. The global AI market was valued at USD 136B in 2022 and is expected to reach USD 1,591B by 2030 [1]. The availability of large datasets, efficient network design and hardware architecture optimization have driven this progress. The advancements in architectural design and the development of innovative topologies such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), graph neural networks, and transformers [2] have pushed its applications into interdisciplinary domains.

By virtue of modeling long-range dependencies, transformers [2] have achieved state-of-the-art performance on various Natural Language Processing (NLP) and Computer Vision (CV) tasks. The field of NLP has advanced significantly due to the emergence of large-scale Pretrained Language Models, which include Bidirectional Encoder Representations from Transformer (BERT) and Generative Pre-trained Transformer (GPT). These models have improved NLP tasks' efficiency and enabled new applications, including ChatGPT [3], BARD [4] and content generation. In fact, researchers have recently used Large Language Models (LLMs) [5] to identify potential COVID-19 variants of concern. Similarly, vision-transformer (ViT) [6] and subsequent models have shown remarkable effectiveness on computer vision tasks such as the image classification [7] and object detection [8], and have outperformed CNNs.

The enhancement in predictive performance and scope of application has come at the cost of a steep increase in memory and computation overheads. Figure 1 illustrates the number of parameters for state-of-the-art (SOTA) language models. Clearly, SOTA models have up to 1.2 trillion parameters! The sizes will increase even further as more powerful hardware platforms are developed. ChatGPT inference consumes 500ml water for a simple conversation of nearly 50 questions and answers [9]. Also, recent work has shown that vision transformers can be scaled up to 22 billion model parameters [10].

These factors call for efficient model compression techniques and hardware acceleration methods to facilitate the deployment and usage of such large models in practical settings. Additionally, given the high computational cost associated with training and fine-tuning large models, there is a growing demand for a more robust and scalable computing infrastructure.

These challenges have motivated researchers to propose techniques for reducing transformers' size, latency and energy consumption for efficient inference for a wide range of applications. The methods include pruning [40, 41], Quantization [42–44], Knowledge Distillation [45] and Neural Architecture Search [46]. These methods allow better scalability and environment-friendliness. Orthogonal to advances in model compression, designing hardware architecture tailored for transformers is a promising solution to overcome the computational limitations of the transformer models. This involves identifying the computational bottlenecks in the transformer model, such as the self-attention operator and fully connected network, and developing hardware architectures that

⋆Equal Contribution

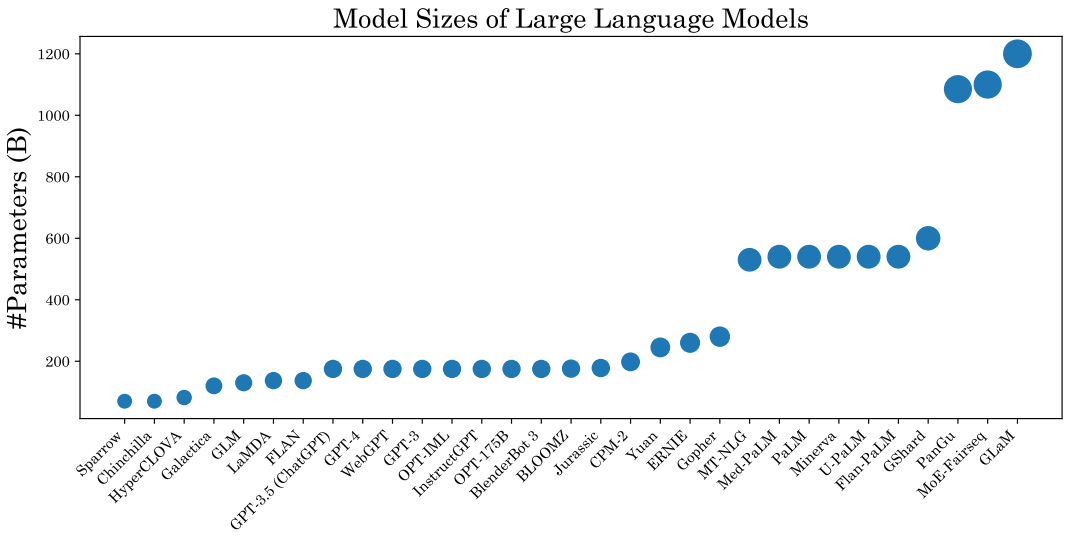


Fig. 1. Model size of SOTA large language models (Sparrow [11], Chinchilla [12], HyperCLOVA [13], Galactica [14], GLM [15], LaMDA [16], FLAN [17], GPT-3.5 (ChatGPT) [18], GPT-4 [18], WebGPT [19], GPT-3 [18], OPT-IML [20], InstructGPT [21], OPT-175B [22], BlenderBot 3 [23], BLOOMZ [24], Jurassic [25], CPM-2 [26], Yuan [27], ERNIE [28], Gopher [29], MT-NLG [30], Med-PaLM [31], PaLM [32], Minerva [33], U-PaLM [34], Flan-PaLM [35], GShard [36], PanGu [37], MoE-Fairseq [38], GLaM [39])

can accelerate these modules. This can be accomplished by efficiently mapping transformer models on FPGAs and ASICs and through optimization techniques such as parallelization, pipelining and avoiding redundant/ineffectual computations.

The scope and outline of this paper: In this paper, we survey several optimization methods for *efficient inference* of transformer architectures and their family of architectures, such as BERT, GPT, and ViT. We discuss the challenges, advances and future opportunities in this ever-growing space of transformer research, whose goal is to reduce inference time, minimize memory requirements, and enhance hardware performance. To provide a comprehensive synopsis of key advances, we limit our discussion to inference-related optimizations and, thus, exclude training-related techniques. We also forecast possible future directions in this fast-evolving field of research. The following list summarizes different dimensions of transformer optimization/compression/acceleration methods and provides high-level definitions and the paper organization:

1. In Section II, we provide a background on the fundamentals of the transformer model, including embedding, general attention and multi-headed attention (MHA). We also discuss the networks used in NLP and computer vision domains, such as BERT, GPT and vision transformer.

2. Section III presents several motivating factors and challenges for optimizing transformer models. The motivating factors include increasing model size and the need for improved performance. The challenges include the availability of computing resources and transformer-specific data/weight distribution.

3. Knowledge Distillation (KD) is a model compression technique where a relatively small student model is trained to mimic the behavior of a large pre-trained teacher network. For example, using KD, DistilBERT [47] compresses the BERT-base model by 40% while retaining 97% of its language capabilities. In Section IV, we first present an overview of distillation methods and distillation loss functions and then summarize KD techniques for transformers.

4. The transformer models are often large and heavily over-parameterized [48]. Pruning refers to the process of identifying and removing redundant or unimportant parameters in such a way that the predictive performance is minimally affected. For instance, oBERT [49] compresses the BERT model and attains 10× inference speedup on Intel CPU with less than 1% accuracy drop. In Section V, we first provide a taxonomy of pruning schemes and then review pruning techniques organized along several categories, such as weight, node, neuron, filter, head, and token pruning. We also review post-training pruning techniques and hardware-aware pruning techniques.

5. During the training process, the weights and activations are generally stored in 32-bit floating-point precision. However, the inference can be performed at a lower precision, such as an 8-bit integer. Quantization reduces their precision/bitwidth to 16-bit, 8-bit or even 1-bit. Thus, while pruning reduces the number of parameters, quantization reduces the storage precision of each parameter. For example, Q8BERT [50] quantizes the weights and activations of the BERT model from 32-bit precision to 8 bits, thereby achieving model size reduction by 4× without compromising accuracy. In Section VI, we present a comprehensive discussion on quantization procedures, the taxonomy of transformer quantization, and binarization methods, followed by summarizing prominent transformer, BERT and ViT-centric low-precision acceleration methods.

6. MHA operation has quadratic time complexity; hence, it is a crucial performance bottleneck in a transformer. Several methods have been proposed to simplify this operation. MobileViT [51] is an example of such lightweight ViT, which attains six percentage points better accuracy than the DeiT model [52], with 3.4M fewer parameters. Section VII summarizes such efficient and lightweight architectural design

methods for NLP and vision applications. We further analyze the accuracy vs. parameter counts of several tiny ViT models.

7. Neural Architecture Search (NAS) is a process of automating the design process of a neural architecture for the given application and dataset. Hardware-aware NAS (HW-NAS) searches for a network with the highest possible accuracy on a dataset and compute performance on a target hardware. For instance, Hardware-aware Transformers (HAT) [53] developed a methodology to search transformer models which have better validation metrics than original transformer [2] while having lower latency on CPU, GPU and mobile platforms. In Section VIII, we first present an overview of general NAS methods, followed by classifying the transformer NAS and HW-NAS methods based on search space, and search technique. We then review the use of search methods for model compression.

8. The high computational demands of transformers call for hardware optimization techniques and designs of novel hardware accelerators. For example, a hardware-unaware pruning technique may compress a model to 0.2× the original size. Yet, such a model is unlikely to provide a 5× reduction in latency, memory accesses or energy on conventional computing systems. In fact, due to its random sparsity patterns, such a pruned model may forgo vectorization and tiling and incur higher latency than the uncompressed model. Similarly, while approximate computing requires fewer operations than exact computation, the former incurs higher latency on a GPU [54]. Evidently, there is a need to synergistically design the processing system and transformer to obtain optimal performance in both worlds. For example, novel dataflows can expose reuse opportunities, and structured pruning techniques can lead to hardware-friendly memory accesses Section IX reviews hardware-level techniques for compute- and memory-optimization.

Contributions: The three main contributions of this paper are as follows:

1. Comprehensive overview: We provide a high-level overview of the SOTA enhancement techniques for transformer inference, covering various network and hardware optimization strategies. To make the survey self-contained and thus useful for both beginners and seasoned researchers, we include the essential background on transformer architecture and transformer-based models. Our goal is to help readers understand the wide landscape of optimization strategies and their challenges and limitations. Our paper is useful for both neural network enthusiasts and hardware practitioners.

2. Taxonomy and tradeoffs : We provide a taxonomy of methods for optimized transformer inference based on several key factors, including the type of optimization technique, granularity within the transformer model, type of transformer architecture and domain. The categorization helps readers with a clear and organized framework for understanding different approaches and allows them to easily identify and compare different optimization techniques and understand each approach's strengths and weaknesses. We discuss the practical considerations for each optimization technique, such as the tradeoffs between accuracy and efficiency. In addition to qualitative insights, we present quantitative results on the number of parameters/FLOPs and the accuracy of several optimization techniques. This provides insights into the tradeoff exercised by those techniques.

3. Future directions: Finally, we identify the gaps in current literature and promising future research directions, such as developing more efficient hardware architectures, investigating co-design benefits, combining different optimization techniques and the need for novel benchmarks. Overall, this paper will be a valuable resource for the research community and industry practitioners seeking to optimize transformer inference efficiency for real-world deployment.

This paper uses "predictive performance" to refer to metrics such as accuracy and compute performance to refer to latency/energy/power metrics. Unless mentioned otherwise, performance refers to predictive performance. We use ViT to refer to the vision transformer proposed by Dosovitskiy et al. [6]. We use "CV transformer" and "NLP transformer" to refer to the broad family of transformers in CV and NLP areas, respectively.

Comparison with related surveys: Our paper is significantly different from earlier works in terms of the variety of methods discussed and in-depth analysis of optimization techniques for transformer inference. Tay et al. [55] limit their coverage to architectural innovations for minimizing computation overheads, e.g., minimizing the quadratic complexity, parameter sharing for small model size, and methods related to sparsity and pooling. Fournier et al. [56] provide a high-level summary of optimizations such as pruning, quantization, NAS, knowledge distillation, hashing, low-rank factorization and sparse attention, and then focus on transformer-specific techniques. They do not discuss hardware architectures for transformers.

The survey by Kim et al. [57] focuses primarily on pruning, quantization, and NAS. However, it has only a limited focus on hardware architecture methods. It also lacks classification or taxonomy for the techniques described in their paper. Their paper discusses both CNN and transformer methods, applying CNN-related methods to Transformers. It does not cover lightweight architecture design, knowledge distillation and binarization methods. Unlike their paper, our review article exclusively concentrates on transformers and also summarizes vision transformer-related pruning and quantization methods. Further, different from all these surveys, our survey also provides figures that summarize the quantitative characteristics of well-known transformer models (refer Figures 1, 10, 11, 21, 29 and 31). By providing a quantitative comparison of attributes of various categories of transformer networks, these figures will provide significant insights to the readers.

II. Background on Transformer Networks

The transformer model [2] learns global dependencies in the input through the attention mechanism in a pairwise correlation manner. The model, depicted in Figure 2, has N identical encoder and decoder modules. The primitive modules in these two units are Input and Output Embedding, Positional Embedding, MHA and Pointwise Feed-Forward Network (FFN). In this paper, the vanilla transformer refers to the transformer model with both encoder and decoder units.

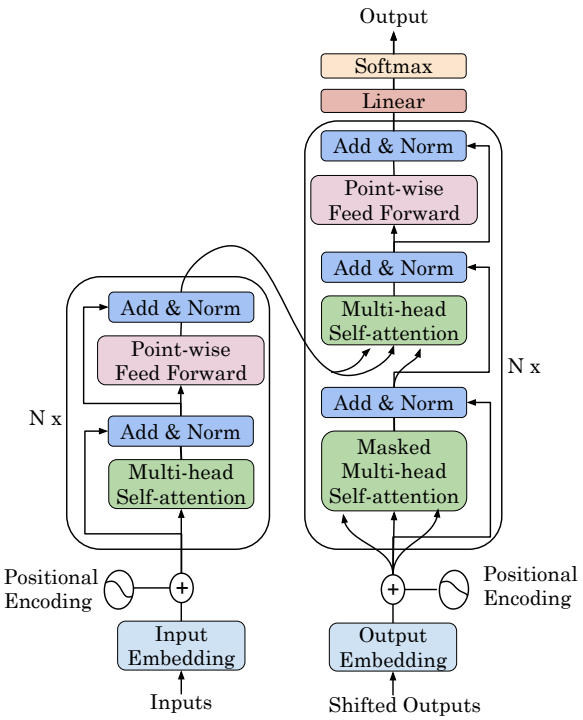


Fig. 2. Transformer architecture [2]

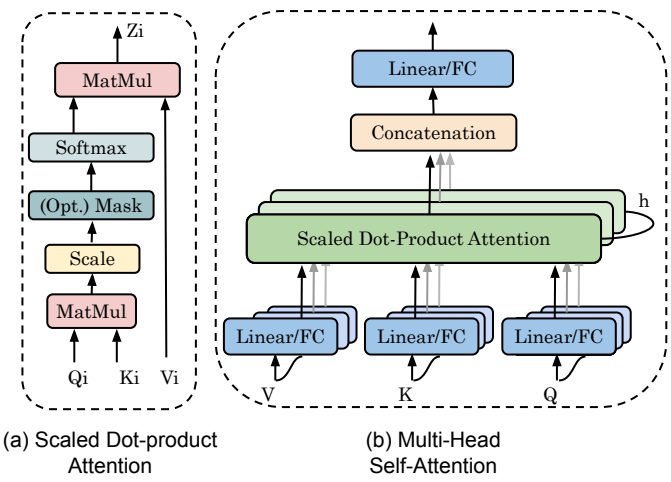


Fig. 3. Attention mechanism in transformers

A. Basic Modules

1) Embedding Layer: The embedding layer translates the tokens into a dense vector representation sequence, which are fed to the attention mechanism.

2) Positional Embedding: Since transformers lack recurrence or convolution operations, they need a mechanism to remember the relative positional information of the words in the input sequence [58]. In the input sequence, the positional information is induced using sin and cosine functions at even and odd positions.

3) Self-Attention: Transformer architectures rely on the self-attention mechanism, which exhibits better model parallelism than recurrent layers and requires minimal inductive bias compared to a convolution network. This mechanism enables the model to focus on different parts of the input sequence dynamically, establishes pairwise correlation and models long-range dependencies between the elements of the input data sequence. In self-attention, the model calculates the attention weights for each position in the sequence, which reflects the importance of each position with others. This allows the model to attend to different parts of the sequence depending on the input. The input of the attention module is fed to three distinct fully connected (FC) layers, which are learned during training to produce Query (Q), Key (K), and Value (V) tensors. The scaled dot-product attention (A), as given in Equation 1, represents the influence of each word in Query with respect to other words in the Key matrix.

$$\mathrm{A} = \text{softmax}\left(\frac{QK^T}{\sqrt{D_k}}\right) \tag{1}$$

The Query and Key are multiplied in an element-by-element manner to produce a score matrix, which is divided by $\sqrt{D_k}$, the square root of output dimensions of the Key matrix to alleviate the gradient vanishing problem. The softmax function boosts high score values and dampens lower score values. The attention score is finally obtained by multiplying the attention and value matrix, as given in Equation 2. The schematic of self-attention is depicted in Figure 3(a).

$$\text{Attention } (\mathbf{Q}, \mathbf{K}, \mathbf{V}) = AV \tag{2}$$

4) Multi-Head Self-Attention (MHA): The MHA module includes several "heads", each concurrently computes attention operations. As depicted in Figure 3(b), the input to the MHA module is replicated across all the heads. The input (X) to a head (head_i) is processed across three FC layers (W^{Q_i}, W^{K_i}, W^{V_i}) to obtain one set of Query (Q_i), Key (K_i), and Value (V_i) vectors on each head, as per Equation 3.

$$Q_i = \mathrm{XW}^{\mathrm{Q}_i}, K_i = \mathrm{XW}^{\mathrm{K}_i}, V_i = \mathrm{XW}^{\mathrm{V}_i} \tag{3}$$

The output (Z_i) of each head is computed using Q_i, K_i, and V_i vectors through the self-attention mechanism, as per Equation 4.

$$head_i = \text{Self-attention}(\mathrm{Q}_i, \mathrm{K}_i, \mathrm{V}_i),\ \mathrm{i} = 1,2,..\mathrm{h} \tag{4}$$

The independent outputs from all the heads $\{\text{head}_1, \text{head}_2, ... \text{head}_i\}$ are concatenated depthwise and linearly transformed using an FC layer, as per Equation 5 to produce the output of MHA module.

$$\text{MHA}(Q, K, V) = [head_1; ...head_h] * \mathbf{W}^O \tag{5}$$

5) Pointwise Feed Forward Network (FFN): FFN or multilayer perceptron (MLP) unit is a series of two fully connected (FC) layers with ReLU [59] or GELU [60] activation function. FFN learns position-specific information with respect to different sets of input sequences. The output of MHA is fed to pointwise FFNs, which is further processed using a normalization (Norm) operation.

B. Encoder and Decoder

The vanilla transformer proposed by Vaswani et al. [2] consists of encoder and decoder modules. The encoder processes the input sequence to generate a fixed-length representation, which contains the essential details of the input data. The decoder utilizes the encoder context and the attention mechanism to generate an output sequence. This is used for sequence-to-sequence tasks such as machine translation.

1) Encoder: The encoder and decoder modules are built by stacking identical layers, each with two sub-layers: MHA and FFN, as shown in Figure 2. The input to the first MHA module is the positional embedding, and that to subsequent modules is the output of the previous layer. The output tensor of MHA is processed further through a Normalization layer [61] and fed to the FFN block to enhance the expressiveness of the input sequence. The output vector of the FFN unit is added to the output of MHA using a residual connection and normalized to generate the encoder output.

2) Decoder: The decoder follows a similar structure to the encoder and is built using an identical stack of three sub-layers. The first sub-layer is a masked MHA unit. Its operation is equivalent to MHA, except that the future positions in the sequence are masked as they are yet to be predicted by the network. The second sub-layer is a multi-headed cross-attention unit, where the output of the encoder is mixed with the output of the first sub-layer (masked MHA). This cross-attention scheme utilizes the previously generated sequence from the encoder and focuses on essential information in the sequence. The third sub-layer is an FFN, which learns position-specific information of the processed sequence, followed by an FC layer.

C. Family of Transformer Architectures

Several transformer-based large models have been recently developed. The most prominent ones are BERT [62] and GPT [63] models. These models learn universal language representation from a large unlabeled dataset and distill the knowledge to a downstream application on the labeled data. The pre-trained BERT or GPT is fine-tuned on a specific downstream application. NLP tasks can be divided into two categories: **(1) Discriminative** tasks summarize an input sequence or classify a sentence. The BERT model is widely used for these kinds of tasks. **(2) Generative** tasks use a GPT model to summarize the input sequence and generate new tokens.

1) BERT: The early language models were designed to process text data sequentially in a unidirectional manner: from right to left or from left to right. By contrast, BERT predicts the missing data based on both the previous words and the following words in the input sequence; hence, the name bidirectional. The BERT model consists of only the encoder module of the original transformer. It masks 15% of the words in input sequence data, as shown in Figure 4(a). The hyperparameters of the BERT model are **(1)** the number of encoder layers (L), **(2)** hidden size (H), and **(3)** the number of attention heads (h). The BERT-base and BERT-large have the following hyperparameters: {L = 12, H = 768, h = 12}, and {L = 24, H = 1024, h = 16}, respectively.

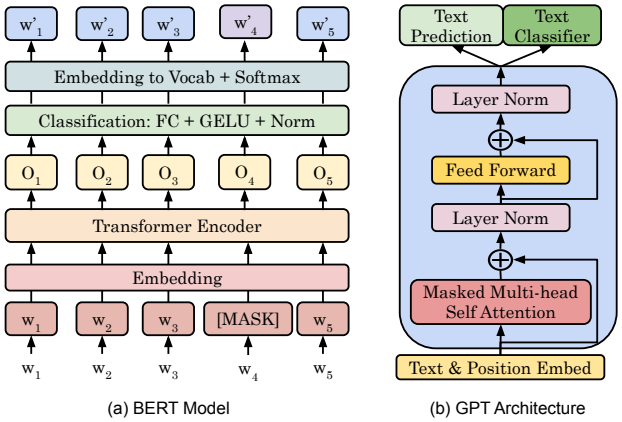


Fig. 4. (a) BERT and (b) GPT architectures

2) GPT: GPTs [63, 64] are large language models (LLMs), which are pretrained in an unsupervised manner on diverse text data to perform predictive tasks. GPT retains only the decoder containing positional encoding, masked MHA, FFN, and normalization operation, as illustrated in Figure 4(b). The variants of GPT include GPT-1, GPT-2, GPT-3 [18], etc. The GPT model is used in various real-world applications, such as ChatGPT [3].

D. Vision Transformer

The vision transformer ViT [6] has opened up a new area of research, focusing on using self-attention modules for computer vision tasks. Vision transformer models have many advantages over CNNs, such as a large receptive field, higher capacity to learn complex features, low inductive bias, etc. Unlike CNNs, which learn local representations through their spatial inductive bias, transformer models learn global representations through the self-attention mechanism. They also effectively model long-range interdependencies and can process multi-modal data such as images, videos, speech, and text. The ViT model, depicted in Figure 5(a), consists of three main modules: (1) Patch Embedding, (2) Position Embedding, and (3) Transformer Encoder.

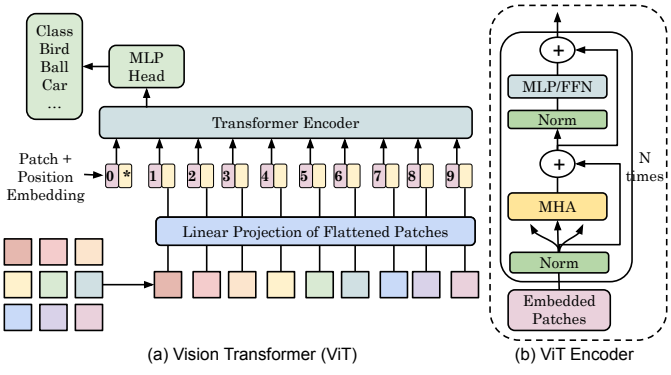


Fig. 5. (a) Overall architecture of ViT (b) ViT encoder

1) Patch and Position Embedding: The input to a ViT is a 3D image of dimension (H×W×3), which is transformed

into a flattened sequence of 2D patches. The ViT model splits the input image into several non-overlapping patches, each of size p×p, to treat them as token embeddings. For instance, consider an image of dimension (H, W, IC), where H, W, and IC represent the height of the input image, the width of the image, and the input channel size, respectively. The resolution of each 2D patch is (p, p), and the image is transformed into a vector of dimension (h*w, p*p*C), where H = h*p and W = h*p. The flattened projection is processed through an FC layer and passed to the subsequent operations in the transformer. The position of each element plays an essential role in better learning global information. Therefore, a 1D learnable position embedding is linearly added to the patch embeddings to preserve the spatial positional information [6].

2) Transformer Encoder: The vision transformer retains only the encoder module of the vanilla transformer, similar to BERT. The encoder extracts features from the input activation map. It establishes long-range dependency among the patches through the self-attention mechanism. In the encoder module of ViT, the normalization operation is applied before MHA and FFN units, as illustrated in Figure 5(b). The FFN module is a sequence of two Fully Connected layers whose output is added to the tensor before the second normalization layer through a residual connection. The final layer of ViT is an FC layer, which predicts output probabilities.

III. Motivation and overview

This section describes the motivation, necessity, and challenges faced in developing optimization methods for transformers.

A. Motivation for optimizing transformer models

We now discuss the necessity of optimizing large-scale transformer models:

1) Model size reduction: Large language models are highly demanding in terms of memory and computing resources, making them challenging to deploy in real-time applications. For example, BERT-base and BERT-large models have 110M and 340M parameters, respectively. Similarly, computer vision models have huge model size, e.g., the original ViT-base model consists of 86M trainable parameters [65]. Techniques like SparseGPT [66] can help remove 100 billion parameters without any accuracy loss. Larger models also provide higher scope for compression. In other words, for a fixed target sparsity, larger models experience a much smaller accuracy drop than their smaller counterparts. For instance, the most extensive models from the OPT and BLOOM families can be pruned to 50% sparsity with a minimal increase in perplexity [66]. Therefore, model compression techniques can allow for storing large models in limited storage capacity.

2) Performance benefits: Model compression can improve hardware efficiency on several metrics, such as latency, energy and power. The inference of a large-sized transformer requires a significant amount of computing time. A smaller model can generally be quickly loaded and executed, leading to low inference latency. For example, MobileBERT [67] is a compressed version of the BERT-base model, and it runs 5.5× faster than the BERT-base model on Pixel 4 mobile phone. Also, smaller models require less memory to store and run, which can benefit resource-constrained environments such as edge devices. Running smaller models requires less energy than running larger models, which can extend the battery life of mobile devices and reduce power consumption in data centers.

B. Challenges for optimizing transformer models

Although essential, the optimization of transformer models presents several challenges.

1) Need of Computing Resources: Developing and implementing transformer optimization techniques require significant computational resources, particularly during the finetuning phase. Finetuning a compressed or optimized model involves retraining the model on a smaller dataset, which can require several iterations of training and validation.

2) Wider distribution of weights: Mao et al. [68] illustrate the challenges in transformer pruning by comparing the weight distribution of the ResNet model on the CIFAR10 dataset with the transformer model on the WMT dataset. Their analysis revealed that the weight distribution of the transformer network is wider than that of the ResNet model, indicating that the weights of the transformer tend to be larger than those of a CNN model. This difference in weight distribution presents a significant challenge for pruning transformer models as the process requires careful consideration of the complex interdependencies among the weights. Therefore, pruning transformer requires more sophisticated techniques than CNN.

3) Simplification prohibits generalization: ML models need to generalize well to new and unseen data. While simplification and compression lead to performance improvement on the target dataset, they can result in poor performance on a dataset from a different domain or having different characteristics. This is because a compression technique may remove weights trained for generalization.

4) Hardware-related challenges: Transformer models use hardware-unfriendly operations that hinder their efficiency and are difficult to implement on specialized hardware. Unlike CNNs, which rely on linear operations, transformer models employ a more complex architecture with many nonlinear operations, including attention mechanisms, softmax and multi-headed attention [69].

IV. Knowledge distillation

Knowledge distillation (KD) [45] is a widely used model compression technique where the knowledge is transferred from a large pretrained teacher model to a small student model, so it can replicate or mimic the teacher model's behavior. KD methods have been effective in compressing large transformer models, such as DistilBERT [47], TinyBERT [70] etc. The distilled models are smaller, faster, and have comparable accuracy to the teacher model. Also, they can enhance the accuracy of the small networks on applications that need complex representations.

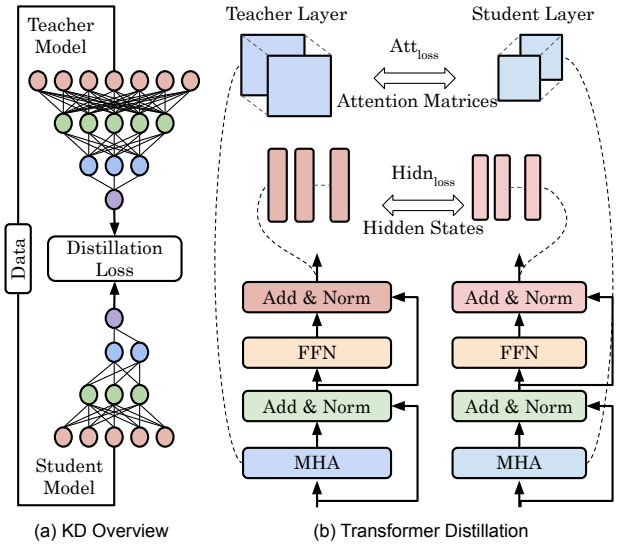


Fig. 6. (a) Overview of KD [70] (b) transformer distillation

A. Overview of Knowledge Distillation Methods

Typically, the distillation methods utilize the teacher model's predictions to guide the student model's training. The process first creates a large neural network, and the task is to make a smaller transformer network approximate the function learned by the larger network. The student model is trained to predict both the correct output and the soft targets produced by the teacher model. The soft target here refers to the probabilities the teacher produces when the predictions are made for a given input. This is done by minimizing the distillation loss between the targets produced by the teacher model and the predictions produced by the student model. Figure 6(a) depicts the distillation method overview.

The distillation process usually employs a linear combination of two loss terms, with a hyperparameter controlling the balance between them. The hyperparameter controls the softness of the teacher's output probabilities, with higher values producing softer targets, making it easier for the student network to learn. The first term is usually the standard cross-entropy or any other loss function depending on the target task, and the second term measures the difference between the output probabilities of the student and the soft targets generated by the teacher network. In general, several loss functions exist to measure the difference between student and teacher models, such as Kullback–Leibler (KL)-divergence, Mean Squared Error (MSE) and Cosine Similarity.

We now review KD methods used to compress the large-sized BERT and ViT models. Table I provides a classification of these methods.

B. Methods based on task-awareness

The KD methods can be broadly divided into two categories based on the level of task-specificity of the knowledge transferred from the teacher model to the student model. They are summarized below.

TABLE I
CLASSIFICATION OF TRANSFORMER KD METHODS

Distillation loss	
KL divergence	[52, 67, 71–74]
MSE	[70, 75–77]
Cross-entropy	[76]
Cosine similarity	[47]
Based on task-awareness	
Task-specific	[52, 71, 74, 76–78]
Task-agnostic	[47, 67, 72, 73, 79]
Learning granularity	
Layer-wise	[67, 70, 72, 76–78]
Output-wise	[47, 52, 71, 72, 74, 75, 79]
Attention-wise	[80]
Network Type	
Transformer/BERT	[47, 67, 70–73, 75, 76, 79]
Vision transformer	[52, 74, 78]

1) Task-agnostic KD: The task-agnostic KD refers to distilling "generic" knowledge, i.e., without considering any specific task, which can be useful for several downstream applications. Homotopic Distillation (HomoDistil) [72] is a task-agnostic distillation method that combines iterative pruning and layer-wise (attention-wise and hidden layer-wise) transfer learning. The student model is initialized from the teacher model and is iteratively pruned until the target width is reached. The iterative pruning method removes the least important parameters throughout the distillation process based on the importance of the parameters with respect to the final score.

2) Task-specific KD: Task-specific distillation transfers knowledge to a small model for the same downstream application. This distillation method is advantageous for obtaining the best performance for the specific task. In contrast, task-agnostic distillation is suitable for transferring only general knowledge and may not obtain the best performance on the target task. DeiT [52] is the first distillation method for ViT. The authors train a student transformer model to match hard labels provided by a pre-trained CNN teacher network on the target Imagenet dataset. The authors utilize only the teacher and student model's final output while ignoring the intermediate layers' information in both networks.

C. Methods based on distillation granularity

The distillation granularity refers to the level at which information transfer happens between the teacher and student network. Figure 6(b) shows that the granularity can be network, layer or token. We now discuss them.

1) Network-level Distillation: The network/model-level distillation transfers knowledge only at the model output level. In this method, the student network is trained to match the output of the teacher model by considering the training to minimize the loss between teacher and student models. This technique is also known as prediction-layer distillation, as the student model is trained to match the predictions.

DistilBERT [47] is a pretraining method based on network-wise KD [45]. It generates a small general-purpose language model that can be finetuned on a wide range of applications. DistilBERT combines language modeling, distillation

and cosine-distance losses. DistilBERT retains 97% of BERT's language understanding capabilities, while having 40% lower model size and 60% lower latency on the Intel Xeon E5-2690 CPU. DistilGPT2 [47] uses the same approach under the supervision of GPT2 and generates a compressed version of the GPT model. DistilGPT2 obtains similar performance as the GPT2 model with only 84M model parameters, as opposed to 124M parameters in the GPT2 model.

TinyBERT [70] is designed using a mixture of task-agnostic and task-specific KD methods. It is a two-stage distillation method, where the first stage transfers general domain information from a large pretrained BERT model to obtain a small-sized general TinyBERT model. The general TinyBERT model acts as a teacher in the second stage and is further finetuned or distilled on the target dataset to obtain a task-specific TinyBERT model. TinyBERT with only four self-attention layers can match 96.8% predictive performance of the teacher BERT-base network on GLUE benchmark while being 7.5× smaller. UVC [74] is a unified compression framework to achieve pruning, layer skipping, and KD in a single, constrained optimization loop. Specifically, its prunes heads in the MHA unit and inner dimension in the FNN block. The original uncompressed ViT network provides the soft labels during the KD process.

2) Layer-level distillation: The layer-level distillation refers to transferring knowledge at the level of individual layers. In this method, the student model is trained to produce similar outputs of selected layers as the teacher model. Hidden state-level transfer learning is a type of layer-level learning that aims to minimize the loss between the hidden states of teacher and student networks. The hidden state represents the output of MHA and FNN modules of the encoder or decoder.

Sun et al. [67] propose a network called Inverted-Bottleneck BERT (IB-BERT). It enhances the original BERT model by adding the linear layers in each self-attention module, as shown in Figure 7. The IB-BERT model acts as a teacher model, and the knowledge is distilled to a smaller version, MobileBERT, progressively over multiple steps in a task-agnostic fashion. The knowledge from IB-BERT is transferred to MobileBERT layer-wise, i.e., the attention level and hidden layer-wise independently, as depicted in Figure 7. The distilled MobileBERT model is 4.3× smaller than the BERT-base network and 5.5× faster on a Pixel 4 mobile phone.

DynaBERT [76] first trains a width- and depth-adaptive teacher model. Then, based on this teacher model, it dynamically adjusts the width and depth of the student model to minimize the target hardware latency using KD. As illustrated in Figure 8, DynaBERT is a two-stage KD process. First, the knowledge is transferred from the large model to a width-adaptive subnetwork and then from this intermediate model to a depth-adaptive model. The distilled models achieve better language capabilities than BERT-base, RoBERTa and TinyBERT models with less latency on GPU and CPU devices. One observation from the adaptive distilled models is that the width direction is more robust to model compression than the depth direction.

3) Attention-based distillation: The attention-based distillation trains the attention matrices of the student network

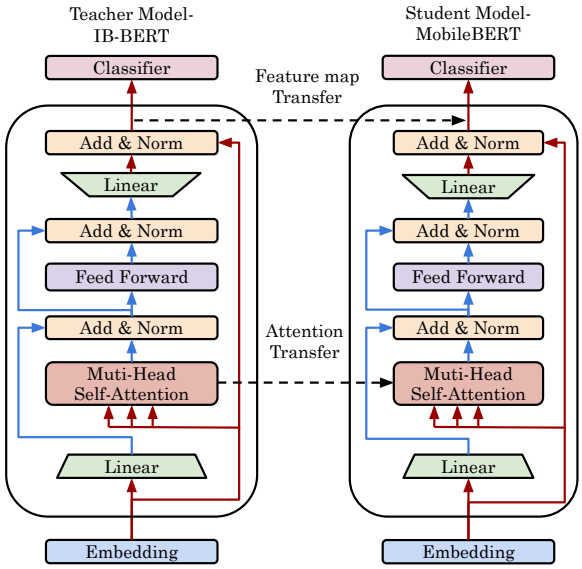


Fig. 7. KD approach used by Sun et al. [67] to distill knowledge of IB-BERT to MobileBERT network

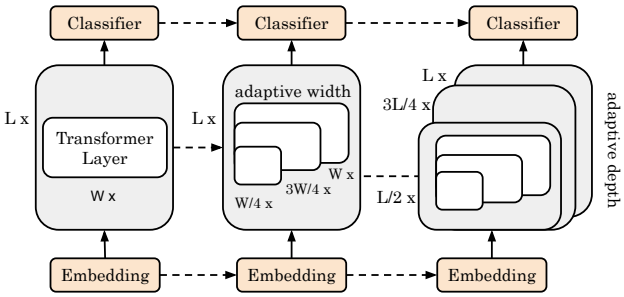


Fig. 8. KD approach used in DynaBERT technique [76]

from the teacher network, such that it transfers linguistic information. The motivation for this method comes from BERT's capability to learn attention weights in such a way that it captures rich linguistic knowledge, which includes syntax and coreference information [70]. Minilm [73] is a deep self-attention distillation technique where the student is trained to mimic the self-attention behavior of the last layer of the BERT teacher. However, within the last self-attention layer, the task is to minimize the KL divergence between the QKV attention matrices. The attention matrix transfer learning process can be beneficial for task-specific distillation scenarios.

4) Embedding-layer distillation: In addition to model-level, attention-level and hidden states, the knowledge from the teacher embedding layer can be transferred to the student's equivalent layer to learn the embedding layer. Manifold learning-based distillation [82] methods use inter-sample information to support layers with mismatched dimensions. Hao et al. [78] utilize patch-level information and develop a fine-grained manifold distillation method to transfer the patch-level manifold information between teacher and student ViTs. The tiny model is trained so that it mimics the patch-level manifold space of the teacher model using three manifold matching loss terms.

Although useful, KD methods suffer from problems such

TABLE II
CLASSIFICATION OF PRUNING TECHNIQUES BASED ON SALIENCY OF NETWORK PARAMETERS: ZERO/FIRST/SECOND-ORDER

	Approach	Pros	Cons
Zero-order	prunes weights based on local importance score (e.g., magnitude) of each weight in the model [79]	computationally efficient	Sub-optimal for large networks due to ignoring global importance of weights
First-order	Considers the impact of each parameter on the model accuracy, e.g., weights moving away from zero are considered important [81]	More accurate in high-sparsity regimes due to considering the gradient information	computationally expensive due to requiring gradient computation for every weight.
Second-order	uses the approximations of the loss curve to guide pruning. Considering loss curvature helps establish a relationship between weights and loss function. [49]	Most accurate due to computing the Hessian matrix (or its approximations)	Very expensive as it requires Hessian computation

as limited generalization, interpretability and overfitting. The distilled student model may not generalize well to new and unseen data when the student tries only to mimic the teacher and ignores the full distribution of the target data. KD can lead to overfitting if the student model is overtrained and tries to fit the teacher model too well.

V. PRUNING

Neural network pruning is a method to reduce the size and computation complexity by removing redundant weights and activations. The pruning algorithms force the weights/nodes/neurons/heads to be zeros as much as possible during inference runtime. In this section, we classify methods based on saliency, sparsity pattern and transformer granularity.

A. Overview of pruning techniques

The general methodology of most pruning methods is first to train a neural network to achieve the best accuracy possible. The second step in this process includes identifying and removing the least important parameters based on magnitude or contribution to the overall model performance. The third step is to finetune the pruned model to recover the accuracy. The second and third steps are iteratively performed until there is an accuracy loss.

B. Pruning taxonomy based on saliency quantification

The pruning techniques can be divided into zero, first and second-order based on how they quantify the saliency of network parameters. Table II describes the three methodologies, along with their pros and cons.

An example of a first-order technique is AxFormer [83]. For large transformers, iteratively performing pruning and finetuning leads to overfitting the training data for the downstream applications. They solve this problem using a hierarchical greedy scheme that needs no additional finetuning. They first find the baseline loss of the transformer by finetuning it on a downstream application. Then, the loss (say K) is computed by removing an element. If K is below the lowest loss encountered so far, the element is pruned. To avoid overfitting, they prune an element only if it reduces the loss of at least half of the samples in the validation set. This ensures effective generalization.

Their pruning technique works hierarchically: it first looks at self-attention and FFN blocks, and only, if required, it analyzes their building blocks, such as neurons and attention heads. This approach prunes bulky blocks quickly and speeds up subsequent iterations. To further narrow down the search space, if a block (say, self-attention) is found to be highly significant, they exclude all the heads in that block from further consideration. For effective pruning, it is crucial to analyze the elements in the right order. Towards this, they note that the lower layers of BERT extract phrase-level and surface features; intermediate layers find syntactic features, and deeper layers focus on semantic features. Deeper layers are required only for capturing long-range dependency. The depth of analysis required by each task is different; e.g., local context is sufficient in sentiment extraction since sentiments change quickly. In fact, syntactic and semantic knowledge is usually not necessary. As such, they inspect from the last layer towards the first layer since the last layers are unimportant or harmful for sentiment analysis.

In the transformer, the use of soft attention facilitates end-to-end training. However, by accounting for only the top-N (say N=30) attention values, the transformer can focus on the most important phrases of the input. They replace hard attention with soft attention in the layers where hard attention reduces the validation loss. Hard attention can sometimes enable better representation by focusing on just one input token. Hence, hard attention is especially useful for capturing phrase-level information in lower layers. Their technique leads to smaller, faster and more accurate models. Their technique can also further improve Q8BERT and DistllBERT models. Also, their models are relatively insensitive to the choice of random seed initialization. Finally, their technique has small latency since it only requires multiple iterations on a small validation set and no finetuning or retraining.

An example of the second-order pruning technique is oBERT [49], which approximates the Hessian function to measure the importance of model parameters. The pruned models attain 8.4× inference speedup with less than 1% accuracy drop and 10× speedup with less than 2% accuracy drop on the Intel Xeon Platinum 8380 CPU platform.

C. Classification based on the matrix sparsity pattern

A neural network can be pruned at different levels, resulting in different sparsity patterns. The methods are classified into unstructured, semi-structured and structured methods. The techniques are described in Table III and illustrated in Figure 9. We summarize a few transformer-specific works based on this classification below:

TABLE III
CLASSIFICATION OF PRUNING TECHNIQUES BASED ON THE SPARSITY PATTERN

	Approach	Example
Unstructured	It prunes the weight matrices of a model irregularly, resulting in unstructured weight/activation matrices. An example of it is element-wise pruning.	[81, 84, 85]
Semi-structured	An example of semi-structured sparsity is N:M sparsity, where the weight matrix is divided into groups, each of size M, of which N elements are pruned.	[86–89]
Structured	It prunes at the component-level, e.g., neurons, channels, heads, columns, rows or entire layers, instead of individual weight parameters. This leads to more regular networks that gain performance even on general-purpose hardware.	
	1. Row/Column: remove redundant rows/columns in weight matrices	[90, 91]
	2. Head-wise: it is a row-wise structured pruning technique that removes the redundant heads in MHA	[48, 92]
	3. Layer-wise: It prunes individual layers of a network	[93]
	4. Block Pruning: it first groups a weight matrix into a 1D (Figure 9c) or 2D (Figure 9d) blocks and prunes the entire block	[94]

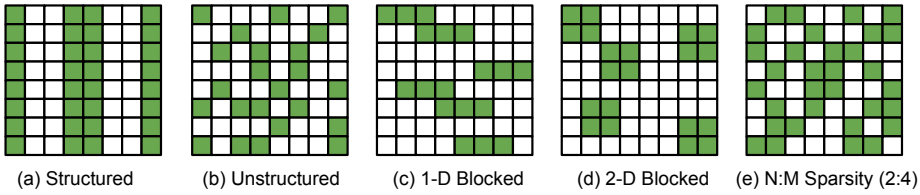


Fig. 9. Types of sparsity

1) Unstructured Pruning: The irregular pruning methods significantly reduce model parameters due to the lowest level of pruning granularity. However, unstructured sparse patterns require specialized hardware architectures and sparse libraries to take advantage of the significantly compressed model. Although 97% of the network parameters can be pruned, it is hard to obtain substantial inference speedup on many hardware platforms [81]. Gordon et al. [85] apply magnitude-based pruning [41] to compress BERT, where weights close to zero are pruned. The authors observe that low pruning levels (30-40%) do not affect pretraining loss, while medium pruning levels hinder useful pretraining information from being transferred to downstream applications. Additionally, the pretraining loss depends on the downstream application in the case of high pruning levels.

Gradual magnitude pruning (GMP) [95] is a process of gradually pruning the weight parameters with low magnitude during the training process. Sparse*BERT [96] applies GMP on LLMs and shows how pruned models can transfer between domains and applications. They show that models pruned on a particular large-scale dataset and applications on the general domain language can be utilized on new domains and small-scale datasets without significant hyperparameter tuning. They can obtain similar accuracy as unpruned LLMs.

Prune-OFA [79] creates unstructured sparse pre-trained BERT models that can be finetuned on the target downstream applications at high sparsity ratios. This method consists of a teacher preparation step for initializing the student model and a student pruning step which is finetuned for the downstream task through a knowledge distillation approach. Prune-OFA also introduces a pattern lock to prevent the zeros in the model from being updated while finetuning the network.

PLATON [97] is another example of an unstructured pruning method. It captures the uncertainty of importance scores based on the absolute difference between the importance score at the current pruning iteration and the moving average of the previous iterations. This method retains weights with low important score values and high uncertainty. On a wide range of transformer models, such as BERT-base [62] and ViT-B16 [6], PLATON can compress the model by up to 90% while increasing the accuracy by 1.2 percentage point.

2) Semi-structured Sparsity: This type of sparsity pattern is more efficient than unstructured pruning and has been implemented in commercial hardware, e.g., the tensor core in Nvidia A100 GPU can accelerate the 2:4 sparsity pattern, illustrated in Figure 9(e), by a factor of 2 [86]. N×MTransformer [88] models N:M sparsity as a constrained optimization problem and optimizes the downstream tasks while considering the hardware constraints. The authors use the Alternating Direction Method of Multipliers (ADMM), a popular technique for non-convex optimization problems with multi-objective constraints. N×MTransformer prunes Q, K, and V matrices, attention output and fully connected layers in the transformer, and the sparsified model is 1.7 points more accurate than the SOTA N:M sparse language models.

Fang et al. [87] propose a network-hardware co-design framework to generate a series of N:M (3:4, 2:4, 1:4) sparse transformer models for deployment on a diverse set of FPGA platforms and a dedicated hardware architecture to support this specialized sparse implementation. The set of N:M sparse transformers are generated using inherited dynamic pruning (IDP), resulting in 6.7 percentage point increase in accuracy.

Chen et al. [98] propose three sparse vision transformer exploration methods to obtain compressed models. The first method, Sparse Vision Transformer Exploration (SViTE), dynamically extracts sparse subnetworks and explores sparse connectivity during the training. Structured Sparse Vision Transformer Exploration (S^2ViTE) structurally prunes and grows the attention heads as structured sparse models are more hardware-friendly. The Sparse Vision Transformer Co-

Exploration (SViTE+) co-explores data and architecture sparsity and determines the most important patch embeddings. The end-to-end exploratory methods improve the accuracy of DeiT-small by 0.28% while compressing at least 50% weights.

3) Structured pruning: Structured pruning methods prune at the granularity of entire layers/filters/channels/heads, leading to a sparse matrix with a structured pattern. WDPruning [99] is a structured pruning technique to reduce the width of FC and MHA layers and the depth of the overall network simultaneously. The width of the weight matrices is pruned using a set of learnable parameters, which are used to adjust the width of the matrices dynamically. On the other hand, the depth of the model is pruned by shallow classifiers based on the intermediate data of the self-attention blocks. The pruning results on DeiT-base [52] show that the throughput can be improved by 15% for an accuracy drop of 1%.

D. Classification based on Pruning granularity

In this subsection, we focus on pruning the trained weights of a transformer model. Based on the algorithm and hardware requirement, a transformer can be pruned at different granularity levels, such as element-, layer-, head-, and line-wise.

1) Element-wise pruning: The element-wise pruning method is analogous to zero-order, which picks the individual element in a transformer as the pruning granularity, resulting in an irregular sparse matrix. The importance of each weight can be measured based on different criteria such as magnitude, output activation values, or scores calculated by other functions. Transformer.zip [84] performs iterative magnitude pruning [41], which prunes all the parameters below a certain threshold in each pruning iteration.

2) Row/Column Pruning: Row/Column is a line-wise structured pruning technique to remove redundant rows/columns in the weight matrices of a transformer. Row pruning refers to removing individual attention heads, while column pruning removes output features. Both techniques prune less important parts of the self-attention unit while maintaining the regular structure of the model. CoFi [90] learns the pruning mask of all operators in a Transformer: FFN layers (Z_{FFN}), FFN intermediate dimensions (Z_{int}), MHA layers (Z_{MHA}), Attention heads (Z_{head}), Hidden dimensions (Z_{hidn}). This framework achieves 10x speedup and close to 95% sparsity across several datasets while preserving 90% of the accuracy of the transformer. VTP [91] target output feature of the linear projections, i.e., FC layers, in a ViT model by learning the sparsity mask of each output feature based on L_1-norm.

TPrune [68] is a combined row and column-wise transformer pruning technique for resource-constrained environments. This method divides the weight matrix into several sub-blocks with the same shape and then utilizes the row and column-wise lasso penalty. The row-wise and column-wise l2-regularizer terms are added to the loss function, and the model is trained to learn the structured sparse representations. The regularizer here is the square root of the sum of squares of weights along a dimension. This way, the least important rows or columns are automatically pruned, as gradient descent aims to minimize the combined loss function. The individual structurally pruned subblocks are concatenated to form the final weight matrix. The pruned transformers achieve 1.16–1.92× speedup for the same model accuracy on mobile devices.

UP-ViT [100] is a unified framework to structurally prune all the important dimensions of ViT blocks, such as MHA, FFN, normalization layers, and convolution channels in ViT variants. The importance of each channel is calculated by first dividing the ViT model into several individual uncorrelated components and evaluating the performance difference after removing each channel in every component. The authors apply the UP method on several SOTA ViT models, such as DeiT, PVT and achieve acceptable accuracy performance tradeoffs on compressed models.

3) Block Pruning: Lagunas et al. [94] compute the importance of each block in the attention layers based on its contribution to the overall model performance and prune the least important blocks. HMC-Tran [101] is a tensor-core aware pruning (TCP) to exploit sparsity in a coarse-grained manner using block pruning technique (Figure 9(d)). The authors first divide the weight matrix into p×q blocks, say 16×16, and prune the entire block whose l2-norm is less than a predefined value *prec*. TCP attains a speedup of 3.68× with 92% sparsity on the BERT-base model on V100 Tensor core GPU, while the baseline SVD achieves only 3.56× speedup.

4) Head Pruning: Head pruning is a row-wise structured model compression method that removes the redundant heads in the multi-head self-attention module. Michel et al. [48] show that a few layers in a transformer can be reduced to as low as a single head. The authors use a first-order proxy method to determine the importance of each head and prune them iteratively. The experiments on Vanilla Transformer and BERT show that the models can be compressed up to 20–40% without any quality loss. Voita et al. [92] first analyze the intrinsic properties and determine each head's importance to conclude that specific heads take specific roles. The authors then develop a gating mechanism to prune half of all the heads with less than 0.25 BLEU loss.

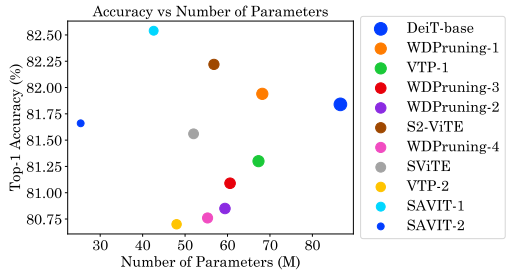
5) Layer-wise Pruning: Layer-wise pruning is a structured pruning technique that uses individual layers as the pruning granularity to reduce the depth of the overall transformer network. LayerDrop [93] selects a sub-network from the original Transformer model by learning the retention rate for each layer during training, and only the layers with high impact are preserved during the inference runtime.

6) FFN Pruning: Pruning redundant weights in an FFN layer is extremely important as this layer account for close to 2/3rd of the total parameters in a Transformer model (excluding the embedding parameters). Ganesh et al. [102] showed that MHA and FFN layers take almost similar time on GPUs even though the former layer account for 1/3rd of the parameters, while FFNs become a bottleneck on CPUs. VTP [91] prunes channels in such a way that it focuses more on the FFN unit than MHA weights.

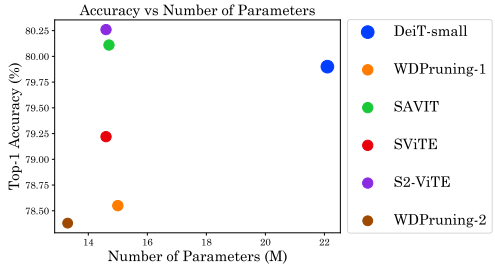
E. Quantitative comparison of pruning techniques

Figures 10(a), 10(b), 10(c) compare the accuracy vs number of parameters of pruning methods on DeiT-base, DeiT-small and DeiT-tiny networks [52], respectively. We obtain

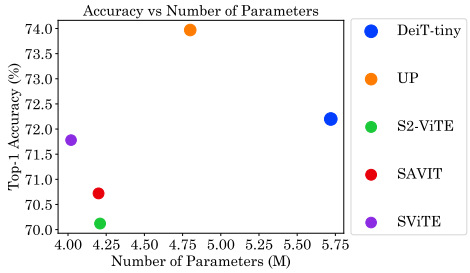
the accuracy numbers from the corresponding papers. The plots show that certain pruning techniques can reduce the number of parameters while improving the accuracy of the DeiT model, while a few methods remove parameters with some compromise in accuracy. For example, WDPruning [99], S^2ViTE [98], SAViT [65] methods increase the accuracy of the compressed model with less number of model parameters on DeiT-base model. On the other hand, other methods, such as VTP [91], come with a drop in accuracy. Therefore, the accuracy of the pruned models depends on the pruning method and finetuning pipeline.



(a) DeiT-Base



(b) DeiT-Small



(c) DeiT-Tiny

Fig. 10. A comparison of pruning techniques: SAVIT [65], VTP [91], WDPruning [99], SViTE [98], UP [100] on (a) DeiT-base, (b) DeiT-small (c) and DeiT-tiny networks [52]

F. Token/Patch Pruning

In the previous subsection, we discussed pruning methods pertaining to the weights of a transformer model, while this subsection focuses on token/patch pruning, which is analogous to activation pruning in the CNN model. Token pruning [103] reduces the computation complexity of a transformer by removing redundant tokens or words from the input vocabulary, whereas patch pruning removes less important patches in the embedding of a ViT model. This pruning can be applied at different stages of a transformer model, such as at embedding, intermediate and final classification layers.

Learned Token Pruning (LTP) [104] adaptively prunes less effective tokens as the sequence passes through different network layers. The tokens below a certain threshold, learned during the training process, are pruned in every layer, allowing the length of the pruned sequence to vary with respect to the input sequence. Luo et al. [105] proposed a token pruning method for vision transformers by using the attention score as a natural indicator to determine the importance and prune tokens. An attention-based pruning module is inserted between the self-attention layers. The integrated weight parameters are fused with MHA to estimate the importance of each token and prune the tokens in the layer accordingly.

Yang et al. [106] dynamically monitor the tolerance of tokens and adapt the precision of Q/K/V vectors. They note that the noise tolerance of a token depends on its importance. They sort the tokens based on their importance scores. There is negligible impact on BERT accuracy when adding Gaussian noise (equivalent to quantization) in the tokens with very low importance. However, doing this to highly important tokens greatly impacts accuracy. They further note that pruning 19% least important tokens causes only a 0.2% drop in accuracy, whereas pruning 24% tokens degrade accuracy by 5%. Hence, the pruning ratio needs to be carefully controlled, and accuracy loss due to pruning needs to be compensated.

Their technique divides the tokens into three types: high-precision (e.g., top 15% most important), low-precision (next 70%) and pruned (last 15%). They are stored with 8b, 4b and 0b, respectively. They regard pruning as 0-bit quantization, which unifies both these techniques. The exact ratio of tokens of each type is decided based on Bayesian optimization. The pruned tokens are consolidated in a single representative token (RToken), obtained by weighting the tokens based on their importance score and then summing up these values. This RToken (which can be 4b or 8b) is concatenated with 8b and 4b tokens and fed to the transformer. At the end of the transformer block, these pruned tokens are updated and concatenated with the output non-zero-bit tokens. This output is fed to the next transformer block. Processing this RToken adds only minor overhead but avoids the accuracy loss due to completely pruning unimportant tokens. This approach allows more aggressive pruning for the same accuracy loss.

Their hardware accelerator uses a variable-speed systolic array [107] to support 4b and 8b matrix multiplication. Due to the dataflow constraints of SA, the PEs (processing elements) performing 4b*4b and 8b*4b operations have to stall for one cycle and two cycles, respectively. Due to this, the PEs remain under-utilized. To deal with this issue, they group similar-precision tokens together and place low-precision tokens in the front. This reduces stall cycles since the cases of Q and K^T having different precisions are reduced.

1) Uniform vs Non-uniform Token Pruning: The uniform token pruning methods use a single pruning configuration for all the tokens throughout the network for a given dataset. Nevertheless, the input sequence can vary with respect to different tasks and datasets. Therefore, applying the same pruning percentage can potentially under-prune short sequence

or over-prune long sequence [103]. The non-uniform token pruning techniques adapt the pruning percentage based on the characteristics of the input sequence. SpAtten [108] is an example of a non-uniform token pruning method that assigns the pruning rate proportional to the input sequence length.

2) Static vs. Image-Adaptive Patch Pruning: Static token pruning methods [98, 104] prune the number of input tokens by a fixed ratio for different images. They neglect the fact that each image's information varies in region size and location. The image-adaptive token pruning methods [109] remove the surplus tokens based on the image characteristics to attain a per-image adaptive pruning rate. Therefore, the latter method can achieve a higher overall model compression ratio than the former method. AS-ViT [110] is an adaptive sparse token pruning method that uses a set of learnable thresholds and MHA to prune the redundant tokens. The attention weights of the self-attention unit evaluate the token significance with a few additional operations, and the learnable parameters are inserted within the ViT model, distinguishing important tokens from uninformative ones. The learnable threshold parameters are optimized in such a way that they can balance accuracy and model complexity, thereby generating different sparse combinations for different input sequences.

HeatViT [69] is an image-adaptive token pruning method for efficient and accurate ViT inference. The authors designed a token selector consisting of an attention-based multi-head token classifier and a token packager to classify tokens accurately and consolidate non-informative tokens. The pruning rate is enhanced by carefully analyzing the inherent computational patterns in ViTs, as opposed to static pruning. The hardware efficiency is further improved by employing 8-bit fixed-point quantization.

3) Quantitative comparison of token pruning techniques: In Figure 11, we compare various token-pruned models' accuracy and floating point operations (FLOPs). As pruning tokens does not alter the network/weight structure, the number of parameters remains the same as the baseline DeiT-small model. However, the number of multiplications and additions is reduced due to token/activation pruning. The size of each circle in the figure corresponds to the relative FLOPs of the token pruned model with respect to the baseline DeiT-small model. All the pruning methods reduce the total number of FLOPs. A few techniques like ViT-Slim [111] improve the baseline DeiT-small accuracy and reduce the FLOP count. DyViT [112] method achieves the best compression with respect to FLOPs but comes with a drop in accuracy.

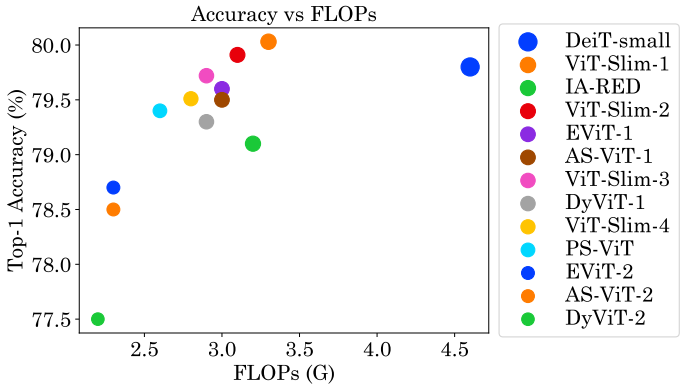


Fig. 11. Comparison of token pruning methods: PS-ViT [104], DyViT [112], EViT [113], AS-ViT [110], ViT-Slim [111], IA-RED [109]. The baseline is DeiT-small [52] model.

G. Post Training Pruning (PTP)

The conventional pruning algorithms require finetuning the pruned network and/or jointly learning the pruning configurations. This is required to retain the accuracy lost due to trimming weights, thereby requiring a significant amount of retraining time. Post-Training Pruning (PTP) does not require any additional retraining and maintains the same baseline accuracy. For instance, Kwon et al. [114] propose a PTP method for transformers based on the Fisher information matrix as a criterion to identify redundant heads/filters. The framework requires only 3 minutes on a single GPU to remove heads in MHA and filters in FFN layers. It achieves a 1.56x speedup on inference latency for BERT with less than 1% accuracy loss. The PTP methods can be divided into static and dynamic, as explained below.

1) Static PTP: The static PTP methods identify and prune the least important parameters in the model, irrespective of the input token sequence. The model is pruned only once and is used for inference for all input sequence lengths. Frantar et al. [66] propose a static PTP method to compress giant language models, such as GPT, and show that it is possible to remove 50% of the weight parameters without significantly compromising model perplexity. They develop SparseGPT, a one-shot and layer-wise solver based on closed-form equations by approximating sparse regression solver and is efficient enough to produce a sparse model only in a few hours of GPU time. The proposed method achieves 60% unstructured sparsity on OPT175B [22] and BLOOM-176B [115] models with minimal accuracy loss under 4.5 hours. SparseGPT can be further extended to N:M sparsity (2:4 and 4:8) with some additional accuracy loss compared to the unoptimized model.

2) Dynamic PTP: Adaptive inference or dynamic PTP refers to the ability of a pretrained transformer to dynamically reduce and adjust the layer length during inference based on the input sequence/token without requiring any additional fine-tuning. The intuition behind this technique is that each input sample is different in terms of complexity, and using a fixed-size model for all input samples may not be computationally optimal. Hence, adaptive inference methods adaptively skip part of the layer computations according to the input sample to obtain the best performance. EBERT [116] is an example of such a method that dynamically prunes the redundant heads in the MHA unit and structured computations (output channels) in the FFN unit for each input sample at inference time. The authors employ two predictor networks (two feed-forward, one batch norm and one ReLU layer), one for MHA and one for FFN unit. The predictor network generates a $\{1, 0\}$ mask equal to the size of the number of heads in MHA and the number of output channels in the FFN layer. The BERT model and the

randomly initialized predictor network are jointly trained. The goal is to learn the predictor network to determine the most important components in the BERT model.

H. Hardware-aware pruning

To realize the full performance benefit of pruning, there is a need to customize pruning to different hardware platforms.

Fan et al. [117] execute the BERT-large model on CPU and GPU platforms and show the percentage latency of attention layers, linear layers and remaining computations. For an input sequence length (L) of 256, linear layers dominate the execution time, whereas, for L=1024 and 2048, the attention layers become dominant. Hence, both these layers need to be accelerated. They classify the sparsity patterns into five basic patterns: random, low-rank, block-wise, sliding window and butterfly (BF). Of these, the butterfly pattern is the only one which (1) allows structured data accesses, (2) simultaneously benefits both global and local context (3) benefits both attention and FFN layers. Butterfly matrices are universal representations of structured matrices having a simple recursive pattern. They can approximate linear transformations. The low-rank sparsity requires sequentially reading the rows and columns, which leads to poor hardware efficiency. The sliding-window pattern only studies local context, and hence, it needs low-rank sparsity to compensate for the accuracy loss. Block-wise sparsity engines require additional algorithmic transformations.

They propose two types of building blocks: ABF-block and FBF-block. The ABF block has attention as the backbone and compresses all the linear layers using butterfly factorization. It has three BF linear layers (BFLLs) for producing Q, K and V matrices. Then, there is an MHA layer and another BFLL for extracting relationships between tokens. Finally, there is a BF FFN consisting of two BFLLs. The FBF block replaces attention with a 2D FFT (fast Fourier transform) layer and hence, lower parameter and computation count. The mixing of input tokens by FFT enables subsequent BF FFN to compute longer sequences. By virtue of FFT, this block uses a unified BF pattern, leading to better hardware efficiency. However, the use of FFT degrades accuracy. Their proposed design uses N_1 FBF blocks and N_2 ABF blocks to address these tradeoffs, as shown in Figure 12.

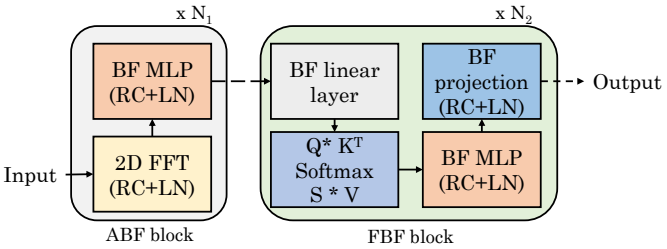


Fig. 12. Accelerator proposed by Fan et al. [117] (RC= residual connection, LN=layer-normalization, BF = butterfly)

Their accelerator has multiple BF engines, which can be programmed at runtime to accelerate either FFT or BF linear layers. It uses multiplexers and demultiplexers to choose the correct input and provide the correct output. This allows the reuse of add/subtract/multiply units. The butterfly pattern requires different inputs at different stages. Hence, both row-major and column-major patterns lead to bank conflict. They propose a custom data layout that shifts down the first element of every column by a certain number of rows. This layout removes bank conflicts in reading data in the first two stages of the BF pattern. They use double-buffering to overlap memory access with computation. Since FFT computation involves complex data, they concatenate the lower (higher) portions of two input buffers to create the first (second) ping-pong storage. Their algorithmic optimizations reduce the model size and FLOPs with no loss in accuracy.

Zhang et al. [118] note that even with structured pruning, the shape of pruned weight matrix may differ in different encoders of various heads in an MHA. They propose compressing the transformer model in a weight-shape-aware manner so that the weight matrices of Q, K, V, O, FFN1 and FFN2 layers are of similar shapes. This improves the utilization of FPGA buffers and MAC (multiply-accumulate) arrays. Their compression technique first finds the weight importance based on the "winning ticket hypothesis" methodology. It shows that the sub-model, created by removing the lowest-magnitude weights and training from the original initialization, can achieve similar accuracy as the original unoptimized model. They use LayerNorm to find the importance of every weight column. One by one, in every encoder and decoder, they attach LayerNorm to any MatMul (matrix multiplication) containing weight. Then, they train the transformer till the convergence of γ factors in the newly attached LayerNorms. The final value of γ shows the importance of each weight column. The scaling factors γ of all the LayerNorms are stored in a new model. In LayerNorm, the scaling elements are the same for different rows but different for every row element. Thus, using LayerNorm with MatMul, γ and β show the scaling factor for each column. Since γ is more dominant than β, γ is taken as the column-importance factor. Then, weight columns with γ values below a threshold are pruned.

Then, a two-stage pruning strategy is used. (1) Coarse-grain pruning removes weights with the same ratio throughout the transformer model. Pruned weight matrices are of similar shape. (2) Fine-grain pruning, which removes redundant weights without accuracy loss. In both stages, pruning and training are performed alternately. While performing MatMul, they compute the output in a column-wise manner. A single-size element-wise vector multiplication and addition substitute the MAC of various sizes. Modern FPGAs have high bitwidth (e.g., 27b * 18b) DSPs, which provide no extra advantage for INT8 computations. They use the double-MAC technique $(((a <<) + b) * c)$, which accomplishes two multiplications in a single operation. In their dataflow, the weight term is used as the shared term c. Column-wise computation naturally avoids multiplications with zero operands. Their technique compresses the transformer by 95 times and achieves high throughput on FPGA.

Fang et al. [89] present a technique for networks with N:M sparsity. A transformer having N:M sparsity requires both sparse-dense and dense-dense MatMul. They present a unified MatMul engine for accelerating both these MatMul. It has H planes of $K * K$ SA. Each PE has a MAC, MUXes,

registers and non-zero (NZ) value selection unit. The MAC unit multiplies two 16b numbers and accumulates with a 32b partial sum. NZ value selection unit is activated only for sparse-dense MatMul. Based on the input bitmask, it loads only NZ weights and the corresponding activations. Figure 13 shows dense-dense and sparse-dense MatMul for a 1:2 sparsity pattern. Clearly, for sparse-dense MatMul, trivial computations are avoided, improving hardware efficiency and performance.

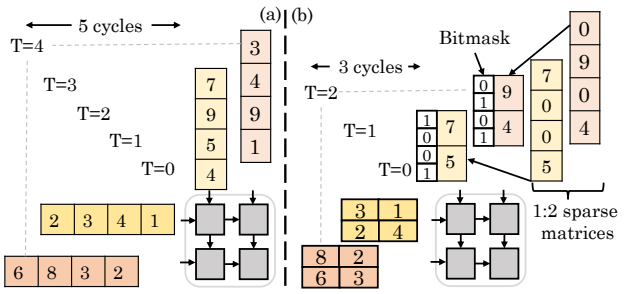


Fig. 13. Multiplication of (a) dense (b) sparse matrices using systolic array [89]

They note that for a fixed sparsity ratio (i.e., fixed (M-N)/N), setting N to 1 provides comparable accuracy as N=2, and hence, they set N=1. The accuracy loss remains negligible even with 75% sparsity; therefore, they set N:M value to 1:4. They set $H = 4$ and $K = 16$ and thus, their accelerator has 1024 MAC units. Compared to Lu et al. [119], which uses 4096 MAC units, their design has comparable inference latency and much higher throughput per MAC unit. On using N:M of 1:8, the latency reduces further.

Li et al. [120] note that transformers that use composite sparse attention, such as Longformer, are inefficient on GPUs since different sparse patterns have different locality patterns. Previous works store the sparse matrices using either coarse-grain format (e.g., block coordinate format) or fine-grain format, e.g., coordinate (COO) or compressed sparse row (CSR), which is inefficient for all sparse patterns. Hence, composite sparse attention takes three-fourths of the execution time on Longformer. They propose novel GPU kernels for accelerating such networks. Based on the spatial locality, they divide sparse patterns into two categories. (1) coarse-grain: all types of blocked patterns, local and dilated. (2) fine-grain: random, global and selected. For these categories, they use BSR (block sparse row) and CSR formats, respectively.

While feeding the input, they create metadata for these formats and load them to the GPU. (1) BSR metadata is produced based on the window size and remains fixed for the dataset. (2) CSR metadata is produced based on global indexes of crucial token locations. This metadata needs to be revised for each iteration. Then, coarse- and fine-grain kernels are used to execute corresponding patterns. SDDMM (sampled dense-dense matrix multiplication) and SpMM (sparse-dense matrix multiplication) are processed parallelly in two streams using coarse- and fine-grain kernels. For fine-grain kernels, they adapt Sputnik kernels and use CUTLASS kernels, which are more efficient than Sputnik kernels. They fuse scale and mask operations with sparse softmax. They propose two coarse-grain kernels that use BSR for (a) SDDMM and (b) SpMM.

(a) It assigns every row block in the output BSR matrix to a single thread block. One thread block handles the entire blocked GEMM. For C = I1*I2, every thread block computes the NZ blocks in a row by reading the blocks from I1 and I2. They decompose blocked GEMMs into hierarchical tiled GEMMs at thread-block, warp and thread levels. (b) This kernel uses a blocked 1D tiling method. It is similar to (a), except that the output row block is not fully processed by one thread block. Instead, separate thread blocks are used for 1D tiles of the output matrix. Similar to (a), further levels of tiling are also used.

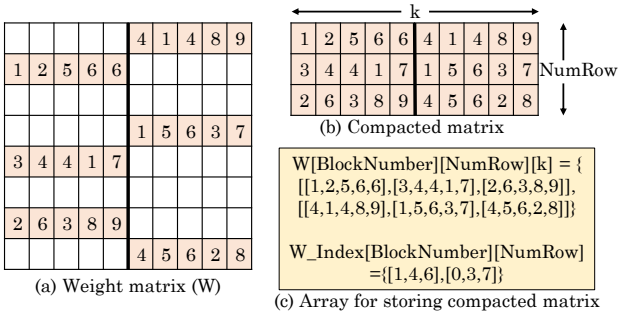
They propose a sparse softmax kernel for computing the outputs of both coarse and fine-grain kernels. Since softmax reads all the row elements, the presence of overlapped coarse/fine-grain patterns in the same row leads to inaccuracies. Hence, they invalidate the overlapping regions. Then, a row-level softmax is done following scheme (a) above. The output row block has NZ elements from both coarse- and fine-grain patterns. They sequentially scan the row using BSR metadata and CSR metadata to process NZ elements in coarse- and fine-grain patterns, respectively. Then, the results of these scans are combined to get overall results. Their technique improves the performance of Longformer and QDS-Transformer models on state-of-art GPUs. They achieve higher performance than techniques using either coarse- or fine-grain methods.

I. Storage formats for sparse matrices

Some researchers have proposed specific formats for storing different sparse matrices.

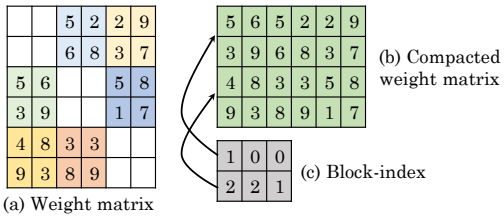
Qi et al. [121] propose techniques for optimizing the transformer model. They compare "block-balanced pruning" (BBP) [122] with "block-wise pruning" (BW) [123]. BBP prunes at row/column granularity in every block of a matrix (refer Figure 14(a)), whereas BW prunes at the granularity of a block. On applying these to the transformer, BBP provides superior accuracy than BW at nearly all sparsity ratios. BBP is a fine-grained pruning scheme that retains more crucial information. Hence, they choose it as their compression scheme. They propose a "compressed block row" (CBR) format for storing matrices resulting from BBP. Based on the compacted matrix, CBR uses two arrays, as shown in Figure 14(b)-(c). (1) A 3D array to save non-zero elements. (2) block indices of non-zero sub-rows. Compared to COO and CSR formats, their CBR format needs lower memory. This is because it stores only the non-zero row index and not the column index of each non-zero element. Their pruning technique reduces latency on an FPGA.

Peng et al. [124] present a "column balanced block-wise pruning" (CBBWP) scheme for bringing the best of block-wise and bank-balanced pruning. It prunes blocks with low L2 norm in each column so that every column is left with the same number of blocks. Figure 15 shows their proposed storage format. It needs only one index pointer for every block, saving memory. CBBWP enables parallelism within and across the blocks. Their FPGA accelerator optimizes MatMul with the CBBWP scheme.



(a) Weight matrix (W) (b) Compacted matrix (c) Array for storing compacted matrix

Fig. 14. (a) Sparse matrix resulting from BBP (b) compacted matrix (c) CBR format [121]



(a) Weight matrix

Fig. 15. (a) Resultant matrix from column-balanced block-pruning (b) compacted matrix and (c) block-index [124]

VI. Quantization

Quantization refers to reducing the precision of the model parameters (weights and activations). This method reduces the precision of these numbers to lower bit widths, such as 16-bit or 8-bit integers. This section briefly overviews transformer quantization methods, taxonomy, and implications on hardware performance.

A. Overview of quantization

There are multiple ways to quantize a network while preserving the model performance. The general quantization methods can be classified into: static vs. dynamic, uniform vs. mixed precision, Post Training Quantization (PTQ) vs. Quantization-aware Training (QAT). Table V provides high-level definitions of these different methods and serves as a background for the rest of the transformer-specific methods.

Table IV provides classification of quantization methods based on the methodology used (PTQ vs QAT) and bitwidth assignment within the network (unform vs mixed precision).

TABLE IV
Classification of Quantization methods based on PTQ vs QAT and Uniform vs Mixed

Method	PTQ	QAT
Uniform	[125, 126]	[50, 86, 127]
Mixed Precision	[128–132]	[133]

B. General quantization procedures

1) Quantization function: The most basic quantization function is linear quantization [44], which represents the floating-point range using a fixed number of discrete levels. The range is divided into equally spaced intervals evenly distributed around a central value. For instance, the range is uniformly distributed between -128 and 127, with zero at the center for 8-bit signed quantization. This method is useful for data with a symmetrical distribution, such as weight tensors in neural networks [138]. The quantization process is represented in Equation 6.

$$Q(r) = \lceil \frac{r}{S} \rfloor + Z; S = \frac{r_{max} - r_{min}}{2^{b-1} - 1} \quad (6)$$

where $\mathbf{r}$ is the floating-point value and Q(r) is the corresponding quantized representation of r, $\mathbf{S}$ and $\mathbf{Z}$ represent the scale parameter and zero-point respectively. r_{max} and r_{min} denote the maximum and minimum values of the floating point range. The rounding function is given as $\lceil \cdot \rfloor$, and b constitutes the quantization bit width. Besides linear quantization, other weight/activation quantization functions include Dorefa [42], PACT [139], QIT [140], LSQ [141], etc. Q8BERT [50] is a QAT method that uses the linear quantization scheme [44] in the forward pass and Straight-Through Estimator (STE) in the backward pass.

Li et al. [133] apply "symmetric linear quantization" on the ELBERT model. Then, they compute clipping range W_{clip} as $sign(W) * min(|W|, 2^i - 2^{-d})$. Here i and d are the numbers of integer and fraction bits, respectively, in W_{clip}. After this, the quantized weight is computed as $\lfloor 2^d W_{clip} + 0.5 \rfloor \times 2^{-d}$. They perform 8-bit quantization for weights and activations of MHA and 16-bit quantization for those of FFN. This allows storing all the weights on-chip. This quantization strategy incurs less than 1% accuracy loss. They use LUTs (lookup tables) to compute logarithms and multiplications in the entropy calculations of the early-exit strategy. The rest of the design and optimizations are similar to that of Lu et al. [119]. Implementation of their technique on FPGA incurs lower latency and energy consumption over a GPU implementation.

Liu et al. [142] first propose FQ-BERT to quantize all the tensors (weight and activation matrices, including biases) and operations (softmax and normalization) and then develop an efficient accelerator to deploy the quantized BERT models. The FQ-BERT method is based on the linear symmetric quantization [44] where the quantized 4-bit weight and 8-bit activation model suffer less than 3.6% accuracy drop while attaining a compression ratio of 7.94×. The accelerator consists of processing units (PUs), a softmax core, a layer normalization core and on-chip buffers. The PU consists of several processing elements which support the multiplication of different weight and activation precisions, such as (8-bit W × 4-bit A) for XW^Q and (8-bit W × 8-bit A) for QK^T. The softmax core approximates the expensive softmax function using a lookup table instead of computing the exponential values during inference runtime. The table requires only 256 entries as the input and output activations are quantized to 8-bit integers. The layer normalization core comprises a 3-stage pipelined SIMD (single instruction multiple data) to utilize the parallelism in element-wise norm operation. The low-precision model accelerated on Xilinx ZCU102, and ZCU111 FPGA achieves a performance-per-watt improvement of 28.91× and 12.72× over Intel i7 CPU and NVIDIA K80 GPU, respectively.

TABLE V
COMPARISON BETWEEN DIFFERENT CATEGORIES OF QUANTIZATION TECHNIQUES

Static: The statistics of the pre-trained model, such as value ranges in a layer, are collected during the offline phase over a calibration dataset. These values remain constant during the inference.	**Dynamic:** It dynamically calculates the quantization parameters of activation tensors during model execution. Hence, this process is expensive as quantization is performed for every new input sample
Uniform: It quantizes all the layers with the same bitwidth [134]. It is simple to implement but the performance is sub-optimal.	**Mixed-precision:** It assigns different bitwidth to different tensors or layers [135] within a network. It attains better accuracy, but finding the optimal bitwidth for each layer/tensor is combinatorial. For example, Li et al. [133] quantize attention weights to 8 bits and FFN weights to 16 bits.
PTQ: It quantizes a pretrained model without requiring additional finetuning steps. It can either use a calibration dataset or can be performed without any such dataset [136] in a data-free manner. It avoids the need for finetuning, but degrades accuracy for low precisions.	**QAT:** It finetunes the quantized model by simulating the effect of the quantization function by using Straight-Through Estimator (STE) [137] to approximate gradients through the non-differentiable quantization function in backpropagation. This makes the model robust to quantization noise but requires an additional expensive training pipeline

2) Matching Full Precision Model: In this subsection, we discuss quantization methods that quantize the model while achieving the same accuracy as the original model. This is similar to the knowledge distillation method, where the quantized model learns the patterns from the floating point network to minimize the overall loss between both versions.

Liu et al. [128] formulate PTQ as an optimization problem to find optimal quantization intervals using Pearson correlation coefficient and ranking loss. The quantization scheme for MHA and FFN modules is learned differently within the transformer. The precision of the MHA module is learned by an attention map ranking loss, and the precision of the FNN unit is learned using the cosine similarity. The main aim of these learned methods is to maximize the similarity between the outputs of the full-precision and quantized network.

PSAQ-ViT [143] is a data-free quantization framework for CV transformers, which does not require the calibration dataset to reduce the precision of a trained transformer model. The authors utilize the properties of the self-attention module and analyze the general difference in its processing of Gaussian noise and real images to generate realistic samples to estimate the quantization parameters. This framework uses a relative value metric to optimize the Gaussian noise to approximate the real images and then calibrate the quantization parameters. The experiments on benchmark models demonstrate the effectiveness of PSAQ-ViT, outperforming real-data-driven methods.

PSAQ-ViT-V2 [129] also utilizes a student-teacher learning methodology, where the goal is to minimize the KL divergence between the pretrained full precision model and the quantized network. CPT-V [130] is based on contrastive loss, which learns the data representation that is not variant to changes in specific attributes. The contrastive loss also minimizes the distance between the quantized and full precision predictions in a self-supervised approach for a given mini-batch. It requires only 1000 calibration images to determine the best quantization scales for each layer in the ViT model.

Yuan et al. [125] observe that the activations after softmax and GELU operations differ from the traditional Gaussian distribution. Also, the commonly used metrics, such as MSE and cosine distance, are not efficient in finding optimal quantization scale parameters. Therefore, the authors propose PTQ4ViT by employing twin-uniform to minimize the quantization error and Hessian-guided method to learn the scales of FC layers. The scaling factors are determined in such a way that the distance between the output before and after quantization is minimized. The experiments on ViT, DeiT, and Swin show that PTQ4ViT obtains near-lossless accuracy on the ImageNet dataset.

3) Dealing with outliers: Dettmers et al. [131] note that the outliers in activation matrices in a few layers break the quantization of LLMs. They quantize outliers to FP16 and other activations to 8 bits to resolve this issue, thereby improving accuracy but bringing challenges in the implementation.

The GOBO technique [126] quantizes model weights and embeddings. In every layer, ~99.9% weights follow Gaussian distribution. Hence, their technique divides weights into two groups: Gaussian and outliers (0.1%). They calculate the mean and standard deviation of the layer's weights. Then, if the probability of a weight belonging to this distribution is below a threshold, it is considered an outlier. The outliers are stored as it is (FP32). Not storing outliers leads to large accuracy loss, but storing more than 0.1% outliers provides a marginal improvement in accuracy. The Gaussian group weights are quantized to just eight FP32 centroids (with $<$1% accuracy loss) or sixteen FP32 centroids (with no loss). Then, only a 3b index to the dictionary is required for each Gaussian group weight. They reduce the number of multiplications by expressing ax+bx as (a+b)x. Their technique allows memory compression by 10$\times$. Their technique is faster and more energy efficient than the Tensor core-like design.

Mokey technique [132] quantizes both weights and activations. In the BERT-Large model, activations contribute more than 50% of the memory footprint for token lengths above 512. Unlike weights, activations cannot be quantized in the offline stage, and they are also spread more widely than weights. Their quantization approach works in three steps (1) They generate a dictionary using "agglomerative clustering", which progressively merges nearby clusters to bring the cluster count to the desired value. It leads to more accurate models than K-means clustering. Their technique creates a bell-shaped pattern with a mean of zero and a standard deviation of one. On this pattern, they repeatedly apply agglomerative clustering to quantize it to 16b FX dictionary of centroids, called "golden dictionary" (GD). The same GD is used for all the models. GD is symmetric around zero, so it requires storing only half the entries. (2) Their technique profiles the model to collect activation samples. Using them and the pre-known weight tensors, they adjust the GD to fit every tensor. Specifically,

if the target tensor has mean and standard deviation as m and s, they compute $GD * s + m$.

For every input tensor, their technique creates two dictionaries for each layer: a Gaussian group of values close to the mean, which covers >98% weights and >95% activations, and an outlier group of the remaining values. Although individual activations depend on the input, their layerwise distribution does not change much. (3) The weight tensors are encoded as indexes to their dictionaries. During inference, their technique outputs FX16 activations, which are converted into indices to corresponding dictionaries before storing them in memory. All weights and activations are quantized to 16-entry dictionaries of FX16 centroids, which require only 4-bit indices.

For further simplification, they fit an exponential curve to model the GD since the near-mean values of a Gaussian distribution follow an exponential function of the form $a^Q + b$, where Q is an integer. Then, instead of computing 256 possible products, they need to compute only 15 exponent sums based on the property $a^q * a^p = a^{q+p}$. This is shown in Figure 16. An actual MAC operation is required in the rare case when either weight or activation is an outlier. Their technique outperforms previous quantization techniques.

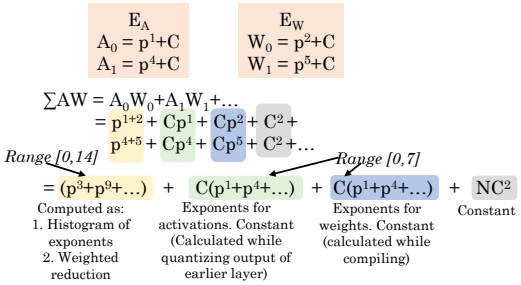


Fig. 16. Computation of output activations in Mokey technique [132]

4) Combining pruning and quantization: Joint-way compression applies two or more compression methods, such as pruning and quantization, to gain additional savings. Most CNN model compression techniques, including pruning and quantization, are usually a two-step finetuning process. First, the model is pruned and retrained, and then, it is quantized to low precision and finetuned. However, training the compressed model twice requires high computation time. Wang et al. [127] propose a technique where quantization and pruning steps are carried out in the same finetuning step on downstream tasks using the pretrained models. The activations are quantized using PACT [139] technique, while the weights are quantized using statistics-aware weight binning (SAWB) quantizer [144], which minimizes the quantization error utilizing first and second-order moments. Their technique achieves 16x speedup over the unoptimized transformer models across language and vision tasks by employing 4-bit quantization and 50% weight pruning. Mishra et al. [86] first perform 2:4 sparsity-aware finetuning on a pretrained transformer, followed by 8-bit PTQ of the pruned weights. The resultant vanilla transformer and BERT models are 8x smaller than the original model, with no loss in accuracy.

The authors of SpAtten technique [108] note that natural languages have many redundancies due to adverbs, articles, prepositions, etc. They assess token importance and then prune unimportant tokens based on attention probability values. More tokens are pruned for longer sentences since they generally have higher redundancies. Token pruning optimizes both attention and FC layers. They further note that the correlations tracked by some of the heads in MHA are redundant. They assess the impact of each head on the output and then prune unimportant heads. Both pruning operations are done in a cascaded manner: A pruned element is removed from all the subsequent layers, and thus, deeper layers need to process fewer tokens/heads. This is shown in Figure 17. Token and head pruning decrease sentence and feature lengths, respectively. While conventional activation pruning decides based on activation magnitude, their technique decides based on attention probabilities accumulated over layers.

MNNFast and A^3 [145] do not reduce DRAM accesses, and hence, they are useful for compute-bound discriminative models (e.g., BERT) only and not only for memory-bound generative models (e.g., GPT-2). Whereas SpAtten reduces memory accesses and, thus, optimizes both types of models. A^3 prunes QKV vectors of a token in one head, and MNNFast prunes V vector; hence, they bring down computation of attention layers, but not FFN layers. In SpAtten, once a token is pruned, those computations in subsequent layers are also avoided, and thus, it optimizes FFN layers also.

They propose progressive quantization of attention inputs to reduce memory accesses further. They observe that when a few tokens are dominant, the quantization error is low, and just MSB (most significant bit) is sufficient, whereas, for a smooth distribution, both MSB and LSB (least significant bit) are required. Hence, they perform more aggressive quantization when some attention probabilities are dominant. They first bring MSBs of attention inputs and compute attention probabilities. If the highest probability is below a threshold, it indicates a flat distribution. Then, LSBs are also fetched, and probabilities are recomputed. Effectively, this technique uses more bits for harder inputs. Overall, they reduce memory accesses by performing extra computations, which benefits memory-bound models (e.g., GPT-2). This technique is not used for compute-bound models (e.g., BERT). Since attention layers frequently use softmax, which can reduce quantization errors, the quantization technique has negligible impact on accuracy.

They further propose a hardware accelerator. To support pruning, they design a highly parallel top-K engine, which finds K most influential heads or tokens in $O(n)$ time, whereas a sorting engine would take $O(n \log n)$ time. The engine supports the splitting and concatenation of LSBs and MSBs. SpAtten is evaluated across 30 well-known benchmarks, such as GLUE, SQuAD, and Wikitext-2, on BERT and GPT-2 networks. The co-designed pruned transformer and accelerator reduce the memory access by 10.0× with negligible accuracy loss and achieve significant speedup and energy savings on a wide range of platforms, including Raspberry Pi ARM CPU, TITAN Xp GPU, Xeon CPU, and Nano GPU.

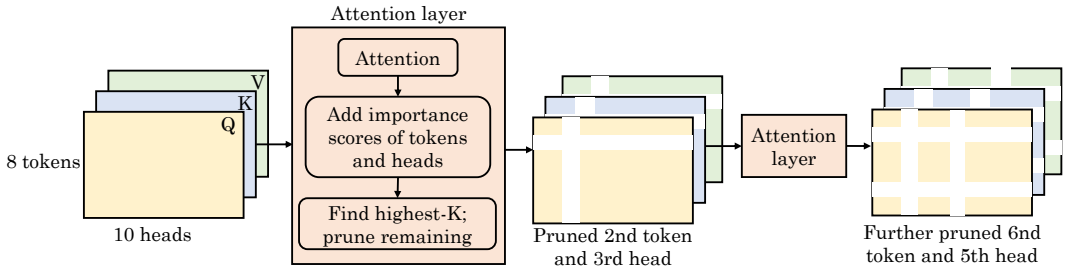


Fig. 17. Cascaded pruning in SpAtten technique [108]

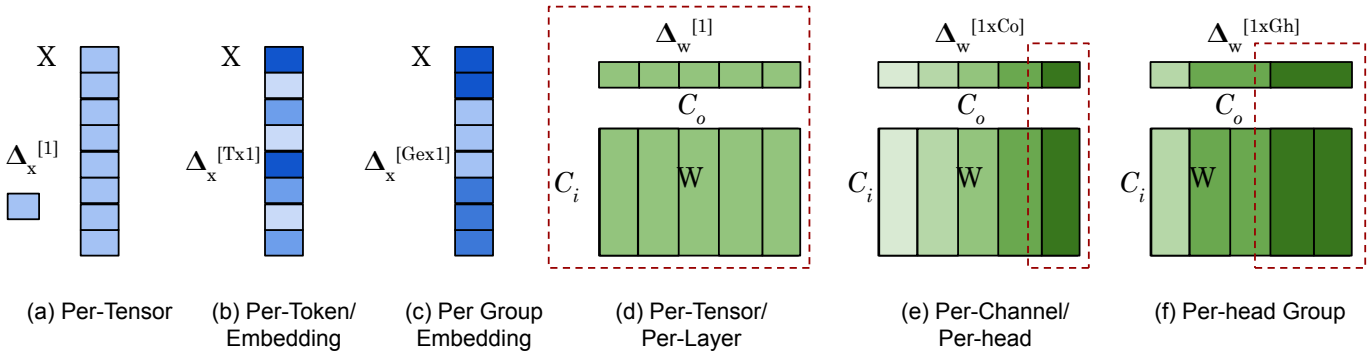


Fig. 18. Quantization techniques. Δ: Quantization parameter, C_i: Input Channel Dimension, C_o: Output Channel Dimension, T: Token Dimension, Ge: Number of Embedding Groups, Gh: Number of Head Groups

TABLE VI
QUANTIZATION GRANULARITY

Granularity	Remark
Per-Tensor	Quantizes each tensor in the network differently. It accounts for different distributions of weights and activations.
Per-Channel	It quantizes each channel of a tensor differently. It is more accurate than per-tensor but requires more metadata and quantization steps during inference.
Per-Head	Each head in the MHA has a different quantization scale. In language/vision tasks, some words/regions are more important, leading to different patterns of self-attention applied to different parts of the input.
Per-Token/Patch [146]	It quantizes each element in the token/patch vector with different quantization scales
Weight-group [147, 148]	Each group within a single weight matrix has a different scale value. Especially useful when the embedding has a complex or non-uniform weight distribution.

C. Classification based on the granularity of quantization

A transformer model can be quantized across different layers and levels of granularity, such as head-wise, channel-wise, embedding-wise, etc. The main challenge with transformer quantization is the difference in the range of parameters between different MHA and FFN layers, and therefore using similar scale parameters can lead to results in strong outliers. Also, previous works have shown that accuracy of a ViT model degrades by quantizing the layer norm and softmax operators [128]. Table VI summarizes the methods based on their granularity, and Figure 18 illustrates these methods.

1) Per-Tensor Quantization: Per-tensor quantization scheme quantizes different tensors (weight and activation tensors) with different quantization scale parameters. This scheme can provide high accuracy as weight and activation tensors can have varying distributions.

2) Per-Channel Quantization: Per-channel quantization is a commonly used method for CNNs, where each channel of a weight filter/activation map is quantized to different precision or scale. Per-channel transformer quantization involves independently quantizing each dimension in each weight tensor matrix with different scale values. This method is more accurate than the per-tensor quantization as it considers each channel's statistical properties, giving more fine-grained control over each channel. However, this technique can be computationally more expensive than per-tensor as it requires storing more quantization parameters and quantization steps during inference.

3) Per-Head Quantization: Per-head quantization is a precise technique that is particularly useful in the case of a model with different attention head distributions, where each head in an MHA module is quantized with a different scale or precision. This process benefits transformer models or text/image inputs with varied data distribution. In language tasks, a few words/tokens in a sentence exhibit more importance than words to understand the overall meaning of a sentence better, leading to different patterns of self-attention applied to different parts of the input. In vision applications, a few regions in the input image may have more variability in color or texture than other sets of pixels, leading to different feature distributions and requiring the model to learn features

using a varied weight distribution.

4) Per-token/Per-Embedding/Per-Patch Quantization: The per-token and per-patch quantization methods refer to quantizing the sequence vector for language task and patch sequence for vision application, respectively, with different quantization scale values. This fine-grained optimization for the specific characteristic in the token/patch can lead to better accuracy, although the number of additional quantization parameters increases with an increase in the embedding size.

ZeroQuant [146] is based on multiple contributions in hardware-friendly quantization grouping scheme, layer quantization learning method and optimizing the inference backend to achieve speedup on different devices. This method applies dynamic group-wise quantization for the weight matrix and token-wise quantization for the activation tensor. The optimized inference backend reduces the cost of quantization and dequantization operations to achieve speedups on INT8-supporting tensor core GPUs. ZeroQuant proposes a knowledge distillation for Mixed Precision Quantization, where the transformer is quantized layer-by-layer to Int4/Int8 precision. The method achieves optimal accuracy and speedups for BERT and GPT-3 (350M parameters) but cannot maintain accuracy for a large-scale GPT model with 175B parameters.

5) Group Quantization: Group quantization is a process of partitioning the weight and activation parameters within a layer into several groups and quantizing them with different precision or scale. Per-embedding group quantization [148] technique divides the embedding weights into groups, and each group is quantized differently using a different set of quantization parameters. This type of group quantization is particularly useful when the embedding has a complex or non-uniform weight distribution. It allows the quantization parameters to be tailored to the specific characteristics of each group. This technique can enhance the transformer model's accuracy by allowing the quantization parameters to be customized for each group of embedding weights. Q-BERT [147] divides the layer parameters into groups, typically 128, and quantizes based on the importance of each group using second-order Hessian approximation in the range of 4 to 16. Q-BERT [147] is a group-wise, mixed-precision quantization scheme that uses the second-order Hessian information [149] to evaluate the sensitivity of the different tensors on the overall accuracy. Their technique achieves 3-bit and 8-bit weight and activation bitwidth, respectively.

D. Classification based on resultant bit-width or datatype

The easiest and most common quantization technique to preserve accuracy is to use integer arithmetic for linear operations while retaining floating-point precision for non-linear operations. However, this mixed precision approach requires the support of custom hardware with a large area for processing floating-point precision into integer-only hardware, thus introducing overhead for interaction between integer and FP precision. Several PTQ methods chose to leave softmax and activation functions in floating point precision as quantizing to low precision may lead to a significant drop in model accuracy.

1) Integer-only Quantization: Integer-only quantization methods propose efficient methods to quantize every layer, tensor, and operation, both linear and non-linear, to integer precision without affecting the accuracy. The int-only quantization methods eliminate the need for quantization and dequantization steps and enable entire network inference in a uniform integer domain.

I-BERT [150] quantizes all the layers in a BERT model, GELU, Softmax, and LayerNorm operations to Int8 precision and achieves 2.4-4x speedup over FP32 BERT model on Nvidia T4 GPU. I-ViT [151] utilizes Dyadic quantization [152] for int-only quantization of linear matrix multiplication. FQ-ViT [153] is a fully quantized model incorporating a log2 quantization and an integer softmax operation. Specifically, the authors introduce Log-Int-Softmax (LIS) to quantize the attention maps to 4 bits and replace the multiplication with a bitshift operator. Rock et al. [134] quantize all tensors and operations, including LayerNorm, SoftMax and GELU, of BERT to uniform Int8 precision. Their technique leads to only a minor drop in accuracy.

SwiftTron [154] is an accelerator designed for quantized networks supporting common transformer operations. On a high-level, SwiftTron hardware consists of the following units, which are executed in integer precision: (i) MHSA unit for attention and softmax computation, (ii) FFN unit for MLP and GELU computation, and (iii) LayerNorm unit for element-wise multiplication and square root operation. Figure 19 depicts the accelerator's core structure, which consists of five MatMul blocks, each one for QKV, QK^T and AV matrix computation. The network parameters are quantized using the traditional quantized function while a re-quantization circuit (Req) is introduced in the accelerator to quantize the output activations back to 8-bits from 32-bits using Dyadic Numbers concept [152]. The hardware is synthesized in a 65 nm CMOS technology, and the RoBERTa-base model [155] requires only 1.83 ns while occupying an area of 273 mm^2 and consuming 33.64 mW power.

2) Binarization/Ternarization: Binarization and ternarization are extreme forms of quantization, where the model parameters are represented using only two and three values, respectively, thereby significantly reducing memory consumption. While binarization requires only $\{1, -1\}$ values, ternary weights are either $\{-1, 0, 1\}$, and hence the memory reqiurement decreases by $32\times$ and $16\times$. respectively. These techniques often go hand-in-hand with PTQ or QAT methods and can be very well utilized for low-power devices such as MCU. Although binarization or ternarization can be very effective in significantly reducing the storage space, they can also impact the model accuracy due to extreme forms of low-precision implementation. While the floating point representation offers the highest level of accuracy, extreme quantization is the most efficient for low-power computational platforms.

Although binarization methods have achieved acceptable accuracy on Convolutional architectures [156, 157], they cannot be directly applied and generalized on transformer models. For instance, BiBERT [158] showed that directly binarizing the model parameters of BERT can cause a drop of 20 points in GLUE dataset [159]. BiBERT proposes a bi-attention

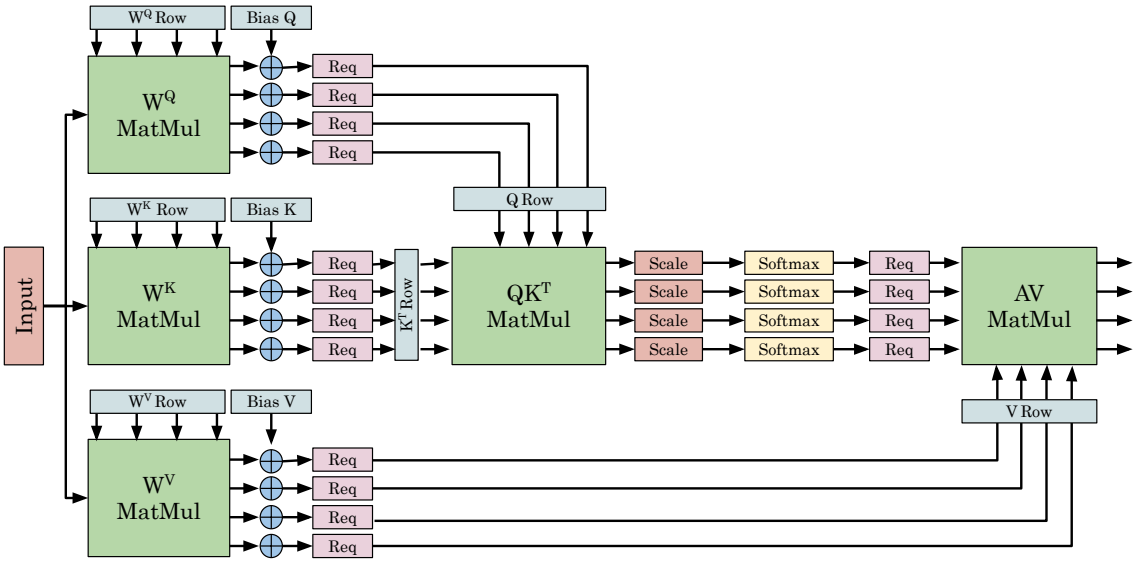


Fig. 19. Overall accelerator design in SwiftTron technique [154] (Req= requantization)

module, depicted in Figure 20, where the attention activations are binarized using a binarization function. The bi-attention module also replaces the softmax function with a bool function that binarizes the attention values to $\{0, 1\}$. The ignorance of softmax can cause quantization errors and damage the attention activations as the model is trained with softmax operation in the MHA modules throughout the network. The authors also propose Bitwise-Affine Matrix Multiplication (BAMM), which used to support the computation between the binarized attention score (B_A) and binarized vector (B_V) during inference.

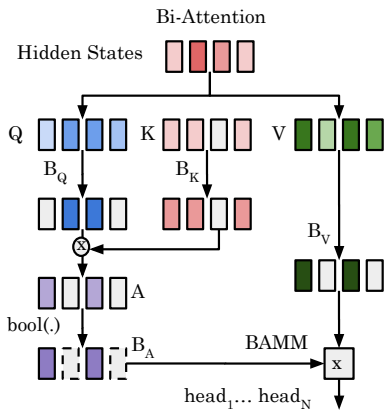


Fig. 20. Bi-attention [158]

Binarized Transformer (BiT) [160] is a multi-stage distillation-based binarization technique, where the authors first distill the information from a full precision model to a medium quantized model (lower precision but not binarized network). The intermediate reduced precision network then acts as a teacher model for the binarized student transformer network. This two-step process ensures good transferability between the large-sized full precision model and the binarized student model, shown on the BERT model in the GLUE benchmark dataset. The authors of XTC [161] first conducted a study to estimate the importance of hyperparameters and training strategies from several previous works. They finetune many existing compressed BERT models and conclude that longer training epochs and smaller learning rate values can aid quantization compression. Based on the observations, they propose methods to quantize BERT while achieving acceptable model performance and 50x size reduction. BinaryBERT [162] proposes a ternary weight splitting method, where a ternary model is chosen as a proxy, and the goal is to bridge the gap between a binary and full precision model. Binary Ensemble BERT (BEBERT) [163] is a technique to overcome the limitations of previous binarization methods, which integrates several Binary BERT models in an ensemble fashion using AdaBoost. It incurs a minimal accuracy loss of 0.3%

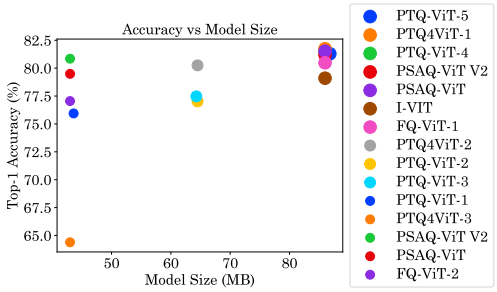
3) Emerging Numerical Formats: The emerging numerical formats such as Bfloat16 [164], Microsoft Floating Point (MSFP) [165], Tensorfloat32 [166], and FP8 [167, 168] provide better computational efficiency and accuracy for several transformer workloads. Bfloat16 is a 16-bit floating-point format developed by Google, consisting of eight exponent and seven mantissa bits. This precision offers significant speedups for BERT models while maintaining comparable accuracy to FP32. MSFP is a floating-point datatype for efficient cloud inference. MSFP shows promising results on the BERT model as it incurs 3× lower cost compared to Bfloat16 with less than 1% drop in accuracy. TensorFloat32 (TF32) is a precision format developed by Nvidia, which provides an easy path to accelerate FP32 on A100 and H100 GPUs. This format uses one sign bit, 8-bit exponent (same as FP32 precision) and 10-bit mantissa (same as FP16 format). Finally, FP8 is an 8-bit floating-point format that is mainly used for low-power inference, which allows a larger dynamic range at the expense of precision [168]. The exponent and mantissa bits for FP8 can be set dynamically. For example, FP8 can take any of the following two formats: (1-bit sign, 5-bit mantissa, 2-bit exponent) or (1-bit sign, 4-bit mantissa, 3-bit exponent). All

these formats discussed here offer several benefits over the traditional floating-point precision, enabling faster and more efficient inference with reduced memory bandwidth.

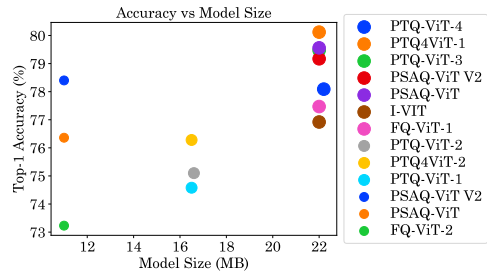
Table VII provides classification of quantization methods based on the bitwidth to which weights or activations are quantized. It can be observed from the table that majority of the methods use 8 bits for transformer quantization as this level provides the least accuracy loss while reducing the precision.

TABLE VII
CLASSIFICATION OF QUANTIZATION METHODS BASED PRECISION

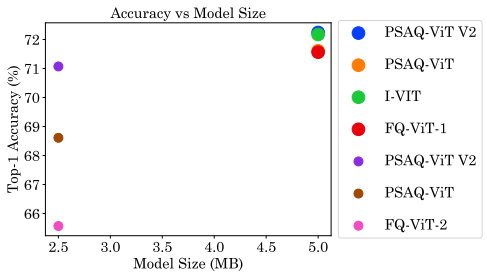
Precision	References
Binary/Ternary	[158, 160–163]
3-bit	[126, 130]
4-bit	[125–130]
6-bit	[125, 128]
8-bit	[50, 86, 125, 127–131, 133]
16-bit	[133]



(a) DeiT-Base



(b) DeiT-Small



(c) DeiT-Tiny

Fig. 21. Comparing accuracy and model size of DeiT-base, -small and -tiny models on the following quantization techniques: PTQ-ViT [128], PSAQ-ViT [143], PSAQ-ViT V2 [129], FQ-ViT [153], PTQ4ViT [125] and I-VIT [151].

E. Quantitative comparison of quantization techniques

In Figures 21(a), 21(b) and 21(c), we compare the model size and accuracies of several quantization methods on DeiT-base, tiny and small backbone configurations, respectively. The methods we compare on DeiT are PTQ-ViT [128], PSAQ-ViT [143], PSAQ-ViT V2 [129], FQ-ViT [153], PTQ4ViT [125] and I-VIT [151].

The model size of the quantized model after quantizing the weight and activation parameters to the same bitwidth using different quantization methods remains the same. This is because they differ only in the quantization function. However, the accuracy of a quantization method depends on the quantization process in terms of how well the knowledge is transferred from a full precision model to a quantized model or how well the method handles the outliers. For example, PSAQ-ViT V2 transfers the knowledge better than PSAQ-ViT as the teacher-student model in the former method is more efficient than the patch similarity in the latter method. It transfers the attention distribution from high-precision weights to low-precision ones well. Ultimately, the model performance of quantized models depends on the method used.

VII. EFFICIENT TRANSFORMER DESIGN

The complexity and computational demands of transformer models hinder their deployment in resource-constrained environments such as embedded systems. Therefore, researchers have developed lightweight transformer models, which still achieve high accuracy. These models can bring the benefits of deep learning to a wider range of tasks and devices. This section reviews the design principles of a few such techniques. Table VIII provides a classification of these methods.

TABLE VIII
CLASSIFICATION OF LIGHTWEIGHT NETWORK DESIGNS

Reducing complexity of attention from quadratic to linear	Applying attention across channel dimension instead of spatial dimension [171], using element-wise operation in attention computation [172], replacing softmax attention with ReLU attention [173] or linear function [170] or low-rank factorization of attention [174]
Reducing number of parameters	Reducing number of heads [174], depthwise convolution [171], group convolution and parameter-free layers [175]
Hybrid convolution-attention networks	replacing 3×3 CONV in the bottleneck residual block with MHA [176], introducing local self-attention into convolution [177], replacing MBConv with MobileViT block in MobileNetV2 [51]
Fusing local and global features	[51, 177, 178]
Domains	NLP [170, 173, 174], CV [51, 171, 172, 175–179]

A. Methods for NLP

To retain long-term/global information, the self-attention operation updates each token's representation by attending to all other tokens in the sequence. However, this requires a quadratic computation cost with respect to the token sequence length for that layer. In addition, transformers perform batch-wise matrix multiplication to learn global representations in MHA. These operations incur high overhead. The goal of efficient transformer variants is to avoid such costly operations and replace quadratic-complexity MHA operations with linear-complexity operations.

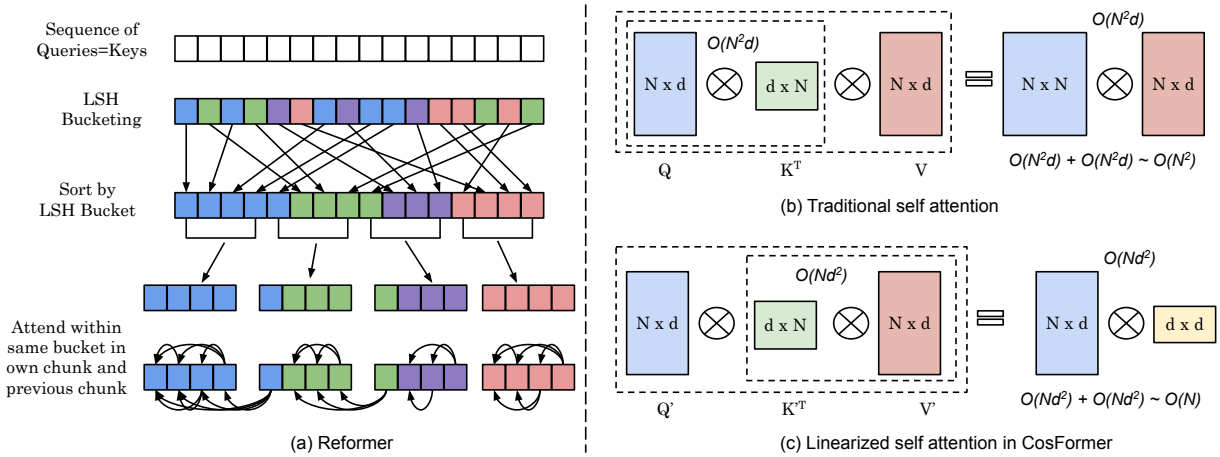


Fig. 22. (a) Locality-sensitive hashing method in Reformer [169]. (b) Self-attention operation in the classical transformer with O(N^2) time complexity (c) The execution methodology of attention with O(N) complexity in CosFormer [170]

Reformer: Kitaev et al. [169] utilizes locality-sensitive hashing (LSH) method to lower the time complexity of traditional self-attention from $\mathcal{O}(N^2)$ to $\mathcal{O}(NlogN)$, where N is the sequence length. The general idea of LSH is to reduce the dimensionality of the data while preserving the similarity between the data points of high dimension. LSH attention involves using locality-sensitive hashing to efficiently find the nearest neighbors among the keys. This is achieved by assigning each vector to a hash using a hashing scheme that ensures nearby vectors get the same hash with high probability and then employing random projections to create multiple hashes for each vector. The tokens within each chunk are attended among themselves, resulting in $\mathcal{O}(NlogN)$ complexity. The entire schematic of LSH in Reformer is summarized in Figure 22(a). The authors also use reversible residual layers, where the activations are stored only once during training, instead of L times, where L is the number of layers in the model. The resultant transformer model is efficient regarding both latency and memory due to these two techniques.

Linformer: Wang et al. propose Linformer [174], which decomposes the dot-product attention operation into small chunks of attention multiplications through linear projections. Therefore, the quadratic self-attention can be executed using a low-rank factorization of the original attention. The number of heads is reduced compared to the baseline transformer, and the decrease in the number of heads in MHA is compensated with long input sequences. While the transformer inference latency increases with an increase in sequence length, the latency of the Linformer remains relatively flat. Thus, for long input sequences, Linformer provides significant speedup over the transformer.

Performers: The authors of Performers [173] introduce a new approach called FAVOR+ to estimate the softmax self-attention for models with longer sequence lengths. The Performers method requires linear space and time complexity. Therefore, this method is more efficient in terms of both latency and memory compared to quadratic $\mathcal{O}(N^2)$ transformer [2] and $\mathcal{O}(NlogN)$ Reformer [169] attention methods. The authors utilize positive random features to estimate the softmax function with a positive feature map, which is essential for ensuring stable training. The Performer model outperforms other models regarding speed, memory usage, and performance. The authors further demonstrate that it is not necessary to approximate softmax to obtain good results. Instead, they use ReLU attention to achieve superior performance when training from scratch.

CosFormer: Qin et al. [170] propose Cosformer, which replaces the quadratic softmax attention operation with a linear function. The features are passed through ReLU to enforce the non-negative property before computing the similarity scores to avoid aggregating negatively-correlated contextual information. The attention weights are re-weighted using the cosine function to enhance local correlations, which usually contain relevant token information for NLP tasks. The quadratic attention operation can be achieved in linear complexity using the matrix product property as follows: $\left(\phi\left(Q\right)\phi\left(K\right)^{T}\right)V = \phi\left(Q\right)\left(\phi\left(K\right)^{T}V\right)$. Figure 22(b) depicts the multiplication in a traditional transformer, where Query (Q) and Key (K) are first multiplied, resulting in an attention matrix, followed by multiplication with Value (Q). Figure 22(c) depicts the linear complexity attention in CosFormer, where first the Key and Value matrices are multiplied, followed by the attention-key dot product. The time complexity in both cases is depicted in Figures 22(b) and (c).

B. Methods for Computer Vision

While efficient transformer design methods for NLP tasks focused predominantly on optimizing attention, efficient models for vision applications aim to curate lightweight models that can run efficiently on resource-constrained devices. In this subsection, we focus on such efficient variants in the computer vision field.

MobileViT: MobileViT [51] combines the strengths of standard convolution and attention mechanism and presents a new approach for better local-global context fusion. The authors replace the traditional Mobile Inverted Residual Convolution (MBConv) layer in the upper stages of the MobileNetV2

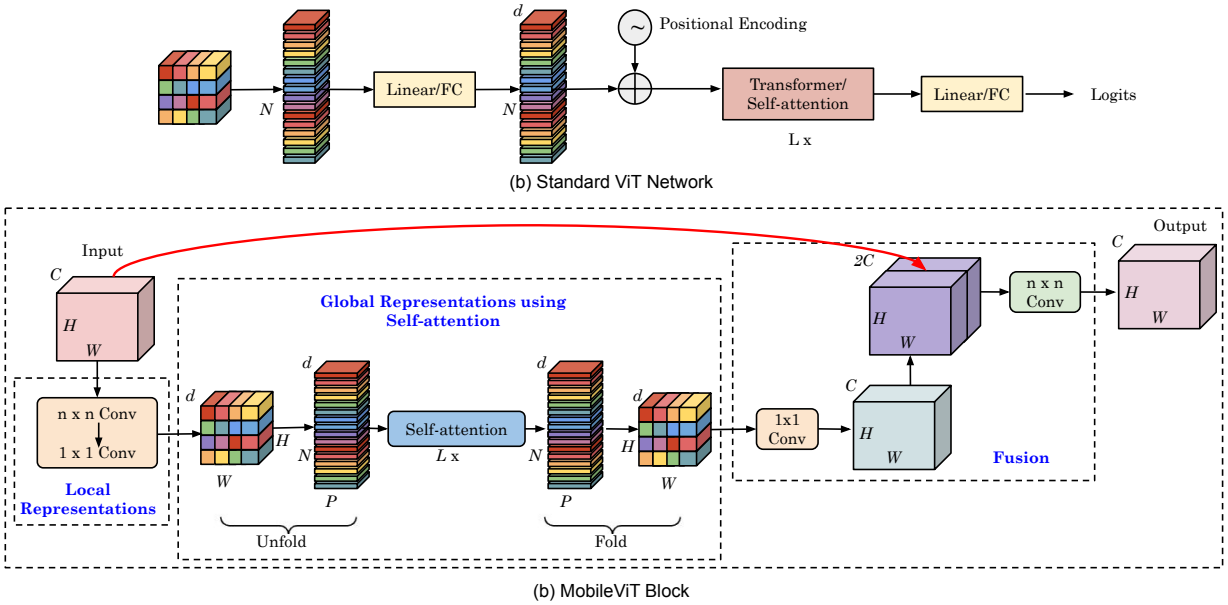


Fig. 23. (a) Standard ViT model proposed by Dosovitskiy et al. [6]. It first flattens the input image into several patches. These resultant patches are processed using a standard FC layer, followed by a series of self-attention modules. (b) MobileVit Block [51]. This block replaces a few MBConv modules in MobileNetV2. The input is a feature map from the previous layer. The local features are extracted using an n × n Convolution layer, and global information is modeled using a transformer. The local and global features are fused together using residual connections.

model [180] with the MobileViT block to obtain better global representation. Using a self-attention mechanism, the MobileViT block substitutes the local processing in convolutions with global processing. The local representations and the global information are concatenated to generate enhanced local-global representations. The hybrid unit helps to learn better representations with fewer parameters and a simple training procedure. This modified approach surpasses lightweight CNNs with a similar parameter budget on the ImageNet dataset. Although MobileViT reduces the number of parameters and increases accuracy, the MHA module is still a performance bottleneck as it requires quadratic complexity. The MHA module is directly inherited from the original vanilla transformer. Each module's sequence of operations and details are summarized in Figure 23.

MobileViTV2: MobileViTV2 [172] enhances the MobileViT network by introducing a separable self-attention layer. This enhanced, separable module uses an element-wise operation for self-attention computation, thereby bringing down the MHA complexity from quadratic to linear with respect to the patch sequence length. The MobileViTV2 model replaces the expensive batch-wise matrix multiplication with a context-aware element-wise operator, as depicted in Figure 24. The separable self-attention layer is explained in detail in Figure 25, which requires only $\mathcal{O}(N)$ complexity. Although MobileViTV2 has more parameters than MobileViTV1, its latency is lower. The MobileViTV2 network achieves 75.6% accuracy on the ImageNet dataset, performing better than MobileViT by 1% and running 3.2× faster on iPhone12.

Mobile-former: The Mobile-former network [178] employs parallel MobileNetV2 and Transformer modules with two-way bridges (Mobile→Former and Mobile←Former). It seeks to achieve a bidirectional fusion of local and global feature

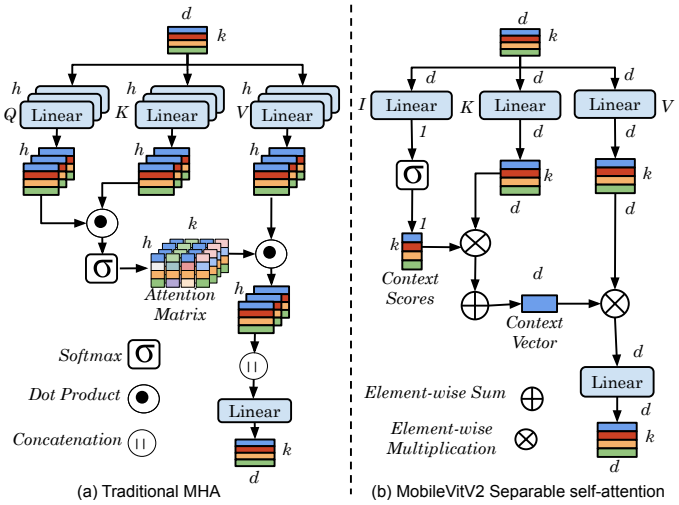


Fig. 24. (a) Traditional self-attention in MHA. (b) Separable self-attention in MobileViTV2 [172], which replaces the expensive batch-wise matrix multiplication with element-wise linear operations.

representations at each level in the network. The input to the MobileNetV2 is the input image, while the transformer module takes a few learnable tokens as input, which are used to encode the global features of the input image. The mobile block is an MBConv block from MobileNetV2 [180], where the ReLU is replaced by dynamic ReLU [181]. Mobile→Former employs lightweight cross attention to fuse the local representations with global features. The Former block is a standard transformer module that consists of MHA and FFN. The Mobile←Former unit bridges from global to local features. The Mobile-Former network attains an accuracy of 77.9%

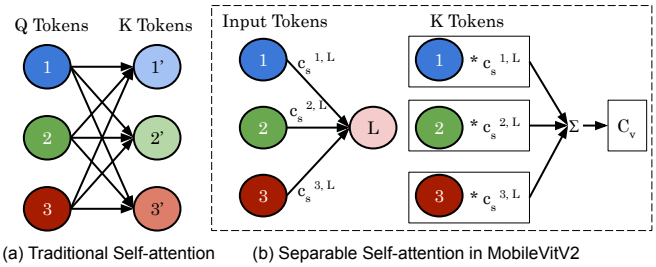


Fig. 25. (a) Computation in the traditional self-attention Each token in Query attends to all other tokens in Key, resulting in O(N^2) complexity. (b) Comparison of separable attention [172] with self-attention [2]. In this enhanced module, the input is processed using three branches Input (I), Key (K) and Value (V). The input I maps each token in input X to a scalar through an FC layer of Weights W_I. These weights serve as the latent node L. The separable attention computes context scores only with respect to this latent token L, resulting in O(N) complexity. The context scores c_s are used to compute context vector c_v, which is equivalent to the attention matrix in self-attention.

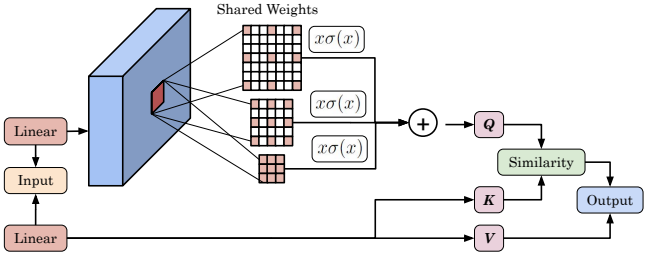


Fig. 26. Recursive Atrous Self-Attention (RASA)

with 294M FLOPs and exceeds MobileNetV3 accuracy by 1.3 percentage points on the ImageNet dataset. The network also outperforms DETR [8] by 1.1 AP on the object detection task.

LVT: Lite Vision Transformer [177] propose two new layers, viz., Convolutional Self-Attention (CSA) and Recursive Atrous Self-Attention (RASA) in the transformer architecture. CSA is used in the first stage of LVT, whereas RASA is used in the last three stages of LVT. They note that the CONV layer is more effective in extracting low-level features. They introduce local self-attention into a 3*3 CONV. Compared to transformers, this leads to enhanced low-level features, improving the generalization capabilities. RASA uses multi-scale context with a single kernel for computing similarity between Q and K, as shown in Figure 26. RASA processes the features recursively using "atrous self-attention" as the activation function. This allows RASA to improve model depth without increasing the parameter-count.

TFormer: TFormer [175] network proposes techniques for making CV transformer transmission-friendly. It employs many *parameter-free* on-the-fly operations along with traditional attention multiplication. Specifically, the authors of TFormer proposed a hybrid layer and partially connected feed-forward network (PCS-FFN), which replace the MHA module and FFN unit, respectively. The hybrid layer comprises only parameter-free layers, such as max and average pooling, whereas the PCS-FFN layer is based on group convolution to reduce the model parameters. These ideas reduce the size of the trained TFormer model that is transmitted from the cloud to the inference hardware. The TFormer models outperform DeiT [52] and PVT [182] in terms of the number of parameters and model accuracy on a wide range of tasks.

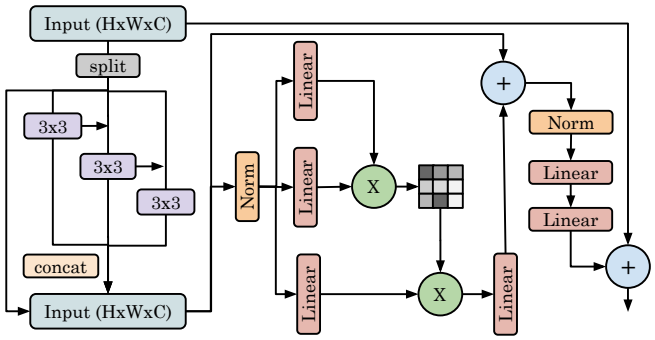


Fig. 27. EdgeNeXt [171]

EdgeNeXt: EdgeNeXt [171] is a lightweight architecture with hybrid convolution and attention modules. The authors propose a "split depth-wise transpose attention" (STDA) unit to split the input tensor into multiple channel groups. This module utilizes N×N depthwise convolution and pointwise operations for spatial mixing and channel mixing, respectively. EdgeNeXt uses cross-covariance attention that applies attention operation across the channel dimension instead of the spatial dimension to reduce the complexity from quadratic to linear. The network also employs adaptive kernel sizes, where small kernel sizes are used in the early stages, and large sizes are used in the latter part of the Transformer model. Figure 27 illustrates the proposed EdgeNeXt module and the network attains 71.2% accuracy on the ImageNet dataset with only 1.3M parameters.

BoTNet: BoTNet network [176] seeks to improve the global representation of feature maps. Bottleneck Transformer (BoT) module is obtained simply by replacing the 3×3 convolution in the bottleneck residual block with MHA. Figure 28(a) illustrates the bottleneck module in the ResNet50 network [183], while Figure 28(b) depicts the enhanced BoT module, which is obtained by replacing the middle convolution with a self-attention unit. The overall BoTNet model is constructed by replacing the last three bottleneck modules of a standard ResNet50 model with the BoT module without changing other hyperparameters. BoTNet achieves 84.7% accuracy on the ImageNet dataset and is 1.64 times faster than EfficientNet on the TPUV3 accelerator.

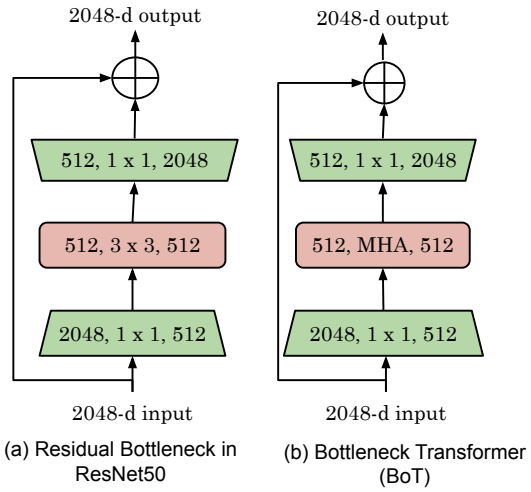


Fig. 28. (a) Bottleneck Module in the traditional ResNet Family [183]. (b) BoTNet [176] module, which replaces the 3×3 Convolution with MHA

C. Quantitative comparison of lightweight CV transformer techniques

In Figure 29, we compare the ImageNet top-1 accuracy and parameter-count of vision transformer models summarized in this section with the baseline transformer models, viz., DeiT-tiny [52], ViT-small [6], T2T-ViT [184] and CNN models, viz., MobileNetV1 [185], MobileNetV2 [180], MobileNetV3 [186]. A positive correlation (not necessarily linear) exists between the number of parameters and accuracy. Larger models generally achieve higher accuracy scores, although there are diminishing returns as the model gets larger.

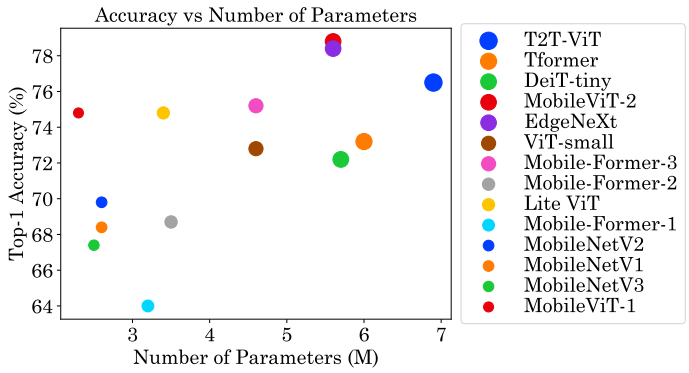


Fig. 29. Comparison of ImageNet top-1 accuracy and number of parameters of lightweight vision transformer models: MobileViT [51], MobileViTv2 [172], Mobile-former [178], LVT [177], TFormer [175], EdgeNeXt [171]

This indicates that increasing the number of model parameters can improve the model's ability to learn and generalize to new data. There is a wide variation in accuracy scores across different models, even when they have a similar number of model parameters. This suggests that other factors, such as the model architecture and training process, are crucial in determining model performance. Also, adding more parameters beyond a certain point may not significantly improve accuracy. For instance, T2T-ViT [184] has the highest number of parameters among the compared models, though the accuracy is less than that of MobileViTV2. Some models, such as MobileViT, achieve high accuracy with relatively small model sizes, indicating they are more efficient at learning from the available data.

VIII. Neural Architecture Search

Neural architecture search (NAS) is a rapidly evolving research field that automates a neural network's end-to-end manual design process for a given task and dataset. Hardware-aware NAS (HW-NAS) is a class of NAS that focuses on automatically searching accurate and hardware-efficient models [187]. The searched transformer networks through NAS and HW-NAS often outperform manually designed models in accuracy and compute performance on the target hardware [188].

This section first provides a brief overview of NAS techniques targeting the transformer family and then summarizes a few works that utilize NAS for model compression. Table IX presents a classification of transformer NAS methods.

TABLE IX
Classification of transformer NAS methods

Search method	
Reinforcement Learning (RL)	[189]
One-shot/Differentiable	[190–197]
Evolutionary	[198–200]
Once-for-all Search	[46, 53, 201–205]
KD	[206–208]
Accuracy Predictor	[209]
Training-free search	[210, 211]
Search Parameter	
Head Number	[46, 190, 194, 201, 202, 209, 210, 212]
QKV Dimension	[46, 202, 210, 212]
FFN Dimension	[46, 190, 195, 201–203, 210, 213, 214]
Embedding Dimension	[46, 190, 195, 202, 210]
Network Depth	[46, 191, 192, 202, 210, 214]
Kernel & Channel size	[189, 191, 194, 199, 203, 204, 209]
Network Type	
Transformer (enc-dec)	[53, 195, 198, 209, 211]
BERT	[196, 215–217]
Computer Vision	[46, 190, 191, 195, 200–203, 210, 212, 213]

A. Overview of Neural Architecture Search

A NAS method typically consists of three components: **(1)** search space, **(2)** search strategy and **(3)** evaluation phase. The first step is to efficiently curate the search space consisting of all possible architectures. The transformer search space typically consists of the architectural hyperparameters, such as Q-K-V FC dimensions, the number of heads in the MHA unit and the inner dimensions of the FC layer in the FFN/MLP module at each level of the network. The depth of the transformer model, i.e., the number of encoder or decoder layers, is also considered in the search space. The vision transformer model includes the patch size and patch embedding size in the search space, while the hybrid attention-convolution search space considers the kernel and filter size of the convolution operation. The search strategy automatically discovers top-performing architectures from the predefined search space for

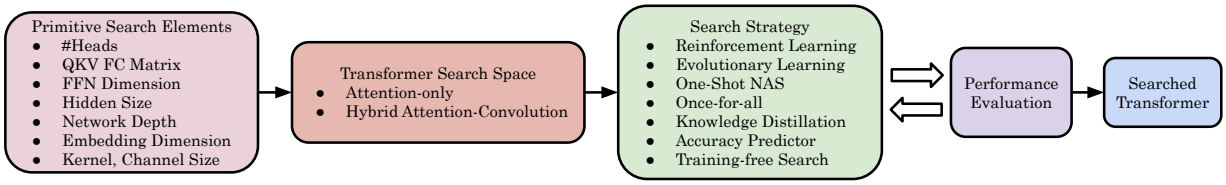


Fig. 30. Overview of transformer Neural Architecture Search Methodology. The main step in the NAS process includes building primitive search elements, and search space followed by applying a search methodology to search for an efficient transformer model

a dataset. The search algorithm outlines a methodology to find an optimal model from a pool of all possible networks. The evaluation phase is the most critical step, which evaluates the performance of the predicted architecture. It compares different neural architectures predicted by the search algorithm to properly guide the search process toward finding an optimal model. The flow of a transformer NAS algorithm is illustrated in Figure 30.

Since a model searched for one hardware platform may not be optimal for other platforms, HW-NAS methods [187] include the performance metrics of the underlying hardware platform in the search method as a multi-objective optimization function. For instance, Hardware-aware Transformers (HAT) [53] showed that the NLP transformer specialized for GPU runs faster on GPU than CPU and vice versa for similar validation accuracy.

B. Classification based on Transformer Search Space

The transformer search spaces can be classified into two types based on the operations present in the primitive element set.

1) Attention-only Search Space: The search elements in the self-attention-only search space include hyperparameters of the transformer attention module, such as the number of heads, FFN dimension, and QKV FC matrix sizes. HAT [53] and AutoFormer [46] are examples of such search methods based on the vanilla transformer and ViT, respectively.

2) Hybrid Attention-Convolution Search Space: The hybrid attention-convolution search space consists of attention and convolution parameters within the transformer backbone network. Although convolution operations are primarily used in vision applications, they are also utilized in a few NLP applications, such as text classification. TextNAS [218], GLiT [203], and BurgerFormer [204] are examples of hybrid search space, which includes the kernel and channel size of the convolution along with the self-attention parameters.

C. Classification based on Search Method

The search algorithm is designed to find the best-performing architecture from the predefined set of primitive operations without significant human intervention. The search strategy has evolved greatly over the last few years, which includes reinforcement learning (RL) [219], One-shot/Differentiable [220], Evolutionary [221], Once-for-all [222], Random search [223], low/zero cost proxy search [224], Bayesian Optimization [225]. We now describe the search methodology and summarize a few transformer-centric search methods.

1) Reinforcement Learning Search (RL-NAS): The pioneering NAS method based on RL [219] consists of an RNN model as a controller which interacts with the environment of all possible neural architectures. In each search iteration, the controller predicts the best architecture that is likely to produce good model performance, i.e., accuracy, and the predicted model is trained end-to-end. UniNet [189] is an RL-based method to search for the optimal combination of convolution, MHA and MLP-mixer layer [226], along with their depth and channel dimension throughout the transformer backbone network. As-ViT [213] utilizes the RL-NAS method to find the base ViT architecture, which is further scaled up to meet the accuracy and computation budget requirement. Compiler-aware architecture search [227] is an RL-NAS method to search for a BERT model network that achieves good accuracy and has low latency on mobile CPU and GPU platforms.

2) One-shot/Differentiable Search: The one-shot search methods reduce the computation time by creating a supernetwork of all possible neural architectures based on a predefined search. This approach uses weight-sharing to reuse the same weights for multiple architecture combinations, allowing the search algorithm to train and evaluate using a single big network instead of individual small models. DARTS [220] is a one-shot method that uses a learnable architectural parameter for each operation in the search space, formulating the search process in a differentiable manner. The final searched architecture is obtained by sampling the best operation at each network level. DARTSformer [193] addresses the problem of applying DARTS directly on the transformer search space. They show that the memory consumption of the transformer supernetwork increases with hidden size. The authors combine DARTS with "reversible networks" [228] to search without running out of memory. This is achieved by reconstructing the input of a reversible network layer from its output during backpropagation, requiring only the output of the last layer to be stored. This reduces the memory burden on the supernetwork and allows for larger hidden sizes and more candidate choices. The searched network for machine translation performs better than the vanilla transformer.

Planer [197] is a differentiable search method [229], which takes a transformer model and a target latency value to produce a sparsely-activated optimized network that meets the latency budget. The search space includes different combinations of FFN layers, number of heads and mixture-of-expert (MoE) layers. MoE layers consist of multiple expert layers, where each path predicts a specific subset of inputs. The output prediction accuracy is enhanced by combining the outputs

from all the paths. The transformer models searched using Planer achieve 2× speedup over vanilla transformer on a GPU while maintaining the baseline accuracy.

You Only Compress Once BERT (YOCO-BERT) [230] first constructs a search space of 10^{13} architectures, featuring all combinations of a BERT model. The optimal model for a given performance constraint is then searched using a novel stochastic nature gradient optimization method. ViT-Slim [111] is another one-shot framework to search for an efficient architecture over three important modules - MHA, FFN and patching mechanism.

3) Evolutionary Learning Search: The evolutionary learning search algorithms [221] use the principle of natural evolution, such as selection and mutation, to find optimal neural architectures. The genetic algorithm (GA) in evolutionary learning is an iterative process of evaluating selected individuals according to a fitness function and generating a new set of architectures using the characteristics of best-performing models from the previous generation. Initially, a population is randomly generated by sampling different architectures from a large pool of networks in the search space. Each individual is a specific neural architecture trained on the target task to determine fitness. The weaker networks have less chance of surviving in the current generation as they compete with candidates of a higher fitness function. The next generation of top-k networks is obtained by mutation or crossover of top individual models in the current generation of networks. Although this search methodology is very effective, it requires a large amount of computation time and resources.

Evolved Transformer (ET) [198] and Primer [199] are examples of evolutionary learning methods to find optimal encoder-decoder and decoder-only self-attention networks, respectively. ET allocates more computing resources to promising models and finds efficient cell/graph architecture stacked multiple times to form the encoder and decoder units. Primer resolves the hurdles of the search method in Evolved Transformer and uses a small proxy dataset to search for optimal models, which are then transferred to the large target dataset. The searched ET and primer models achieve superior validation metrics than the vanilla transformers on language tasks.

Real-time Style Transfer [200] is a hardware-aware CV transformer search method. The submodules of the ViT backbone for style transfer are searched using evolutionary search. The searched network is at least 2.1× faster than the baseline model for style transfer on Xiaomi Redmi 10 mobile and Raspberry Pi 3 embedded devices.

4) Once-for-all search: Once-For-All (OFA) [222] is a two-step method that combines the one-shot approach and evolutionary learning search process. The OFA method first trains a supernetwork of maximum dimensions along all the hyperparameters, i.e., MHA heads, FNN dimension, kernel and filter size. During the second step of the evolutionary search process, the predicted models are sampled from the pretrained supernetwork and validated to obtain validation metrics without additional finetuning. This search method avoids training of every sampled network as weights for the sampled models are retained from the trained supernetwork. Several transformer methods rely on the OFA technique by training a large transformer supernetwork and applying an evolutionary search for the optimal number of heads and FFN dimension.

HAT [53] is an HW-NAS method where the search space includes key transformer hyperparameters, namely, the number of heads, MLP dimension, and the number of encoder/decoder blocks. The search space is elastic, so the encoder/decoder module at each stage can choose a different set of hyperparameters. The search methodology follows the once-for-all technique, where a supernetwork of the highest dimension is initially trained, and different submodels are sampled to perform the evolutionary search. The search process incorporates the target latency for different devices, viz., Intel Xeon CPU, Raspberry Pi ARM CPU, and Nvidia TITAN Xp GPU platforms. The searched model for Raspberry Pi-4 embedded device on the WMT'14 translation task achieves a 3× speedup with 3.7× fewer trainable parameters over the baseline vanilla transformer. The searched subtransformers reveal that GPU platforms prefer shallow models with wide layers, while ARM CPU picks deep models with thin layers to obtain optimal hardware performance.

Several vision transformer search methods, such as AutoFormer [46], ViT-ResNAS [201], NASformer [212], BurgerFormer [204], etc., also rely on OFA technique to search for an optimal model. The OFA method first trains a large Supernetwork of maximum dimensions and samples different-sized models from the supernetwork to specialize for different performance metrics, thereby significantly reducing the search cost. The supernetwork comprises maximum dimensions, such as QKV, number of heads, embedding dimensions etc. The objective function of evolutionary search considers different performance-related metrics in different ViT methods, such as accuracy, model size and latency. The searched CV transformer models outperform SOTA CNN and manually designed self-attention-based models, such as DeiT [52], regarding the accuracy and the number of parameters.

5) NAS using Knowledge Distillation: Knowledge distillation methodology can accelerate the NAS process by transferring the acquired knowledge from a large teacher network to the student model. This allows the NAS algorithm to evaluate the performance of the predicted subnetwork using the teacher model. LightHuBERT [206] first trains a once-for-all BERT supernetwork of maximum dimensions from scratch using the loss function of the pre-training distillation. During the search process, the subnetworks with different sizes of embedding dimension, number of heads, FFN inner dimension and network depth are sampled and evaluated using the teacher model. The search process culminates when a network with desired performance is reached. On several speech recognition tasks, the searched LightHuBERT achieves comparable predictive performance as the baseline teacher model HuBERT [231], even though it has 29% fewer model parameters. AutoDistill [207] is a distillation-based search method that utilizes multi-objective Bayesian Optimization (BO) to learn a small model by considering several objectives, constraints and hardware performance. The authors include layer-wise, progressive knowledge transfer, and a model pre-training distillation into the search process.

6) Accuracy predictor based NAS: The search methods based on accuracy predictors utilize an ML model as a surrogate model to predict the accuracy of a predicted network during the search process. The auxiliary model is previously trained on pre-collected samples of architecture-accuracy pairs. LightSpeech [209] is a search method to find the optimal text-to-speech model based on the accuracy predictor. The search space is a hybrid attention-convolution backbone consisting of the number of heads and kernel size in the convolution operation. The searched transformer is 6.5× faster than the baseline model on Xeon CPU E5-2690 v4 with similar voice quality metrics.

7) Training-free Neural Architecture Search: The training-free NAS methods rely on a set of performance evaluation strategies based on the model architecture or gradient information for quickly estimating the model accuracy. During the search iteration, the estimated accuracy is used without training the model, thereby significantly reducing the search time. For instance, Abdelfattah et al. [232] devise a zero-cost proxy method, where they assign a score to the neural architecture at initialization which is indicative of the validation accuracy of just a single minibatch of data. LiteTransformerSearch [211] is a training-free search method to find optimal transformer architectures for resource-constrained hardware platforms. The authors establish a strong relationship between the validation accuracy and the number of model parameters of the decoder layer in the transformer, thereby substituting the decoder parameter count as a proxy during the search process. The authors integrated the zero-cost proxy metric into the evolutionary search algorithm, where the accuracy is obtained by the surrogate model and latency is computed from the target hardware platform. The searched transformers attain a speedup of 1.3× and 1.5× on Intel Core i7 CPUs and Nvidia Titan Xp GPUs, respectively, over the baseline transformers while achieving similar perplexity. TF-TAS [210] is a zero-cost proxy method that efficiently evaluates different configurations of the CV transformer model. The evaluation is based on the synaptic diversity and synaptic saliency of MHA and MLP units, respectively. The search time to find the optimal transformer is just 0.5 days, while the supernetwork training-based search requires 24 GPU days.

D. Application of NAS for Model Compression

The algorithmic development of NAS methods led to applying the search strategies to solve combinatorial problems automatically. In this subsection, we review the use of NAS methods for model compression, mixed-precision quantization and other use cases.

1) NAS for Automated Pruning: The NAS-based pruning methods apply the search methodology to automatically remove the redundant parameters, providing an alternative to manual pruning algorithms. Several BERT-centric pruning methods, such as AdaBERT [215] and NAS-BERT [216], compress the large model into a small model on downstream tasks. AdaBERT [215], LightHuBERT [206], AE-BERT [233] are examples of BERT compression methods using NAS principles. NAS-BERT [216] is a task-independent compression strategy to reduce the size of a BERT model, while AdaBERT [215] compresses a pretrained BERT in a task-dependent manner utilizing differentiable search [220]. AE-BERT [233] is an automated compression methodology to find a submodel for a target pruning ratio from a pretrained BERT.

2) NAS for Mixed-Precision Quantization Search: The challenge in mixed-precision quantization is assigning the optimal bit-width of each layer such that model size is reduced without accuracy loss. If an "L"-layer transformer can be quantized to one of the "b" possible bitwidth values, there exists b^L different quantized configurations. It is practically impossible to finetune every combination to find the optimal mixed-precision quantized model. Therefore, the problem of mixed-precision quantization can be reformulated as a NAS problem, thereby utilizing the search principles of NAS algorithms. AQ-BERT [196] is a mixed-precision quantization search method to assign different bit-width/precision to different encoder layers of a pretrained BERT model and different precisions to different sub-groups within a layer. The automatically searched and compressed BERT networks show their effectiveness on standard GLUE benchmarks and commodity hardware in terms of accuracy and latency.

3) NAS for Hybrid Operator Search: NAS methods can also be utilized to search for hybrid operators within a transformer backbone network. We review two case studies to demonstrate the usefulness of NAS in such scenarios. Liu et al. [234] include the conventional attention ($\mathcal{O}(n^2)$ complexity) and linear attention ($\mathcal{O}(n)$ complexity) from Cosformer [170] in the search space. They propose a search methodology to find the best type of attention at each layer of the transformer network such that the models are balanced in terms of time complexity and accuracy. Compared to baseline transformers with quadratic complexity, their searched networks have comparable accuracy on NLP and vision tasks and much better compute efficiency.

The networks based on convolution and attention operations achieve high accuracy but involve expensive multiplications. To boost hardware efficiency, researchers have proposed networks that perform only addition [235] or bitwise-shift operations [236]. Nevertheless, such non-multiplication networks attain inferior accuracy. ShiftAddNAS [195] employs a NAS method that searches the combination of multiplication or non-multiplication operators at every layer of the backbone transformer network to balance model accuracy and hardware efficiency. The searched transformers on WMT'14 En-Fr and WMT'14 En-De NLP datasets outperform the baseline transformers and HAT models in terms of latency, energy, and BLEU score. The searched CV transformer model also outperforms the ResNet50 and other CV transformers on the ImageNet dataset.

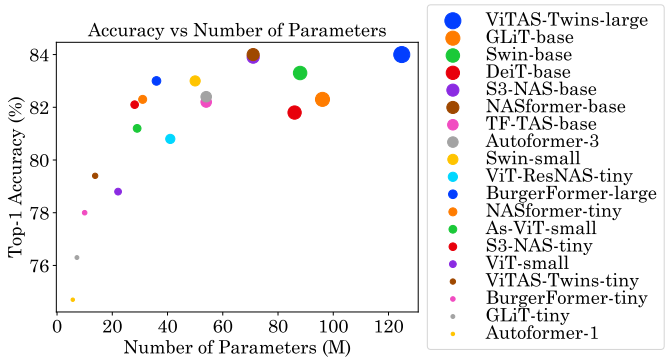


Fig. 31. ImageNet accuracy vs. model size comparison of the searched CV transformer models with manually designed models. The searched CV transformers include: ViTAS [190], GLiT [203], S3-NAS [202], TF-TAS [210], NASformer [212], AutoFormer [46], ViT-ResNAS [201], BurgerFormer [204], As-ViT [213]

E. Quantitative comparison of searched CV transformers

In Figure 31, we summarize the parameter count and top-1 accuracy on the ImageNet dataset of the searched models. Based on the plot, it appears that a model must possess a large number of parameters in order to achieve a high level of accuracy. However, the trend is not linear, rather increases with increasing parameter count and saturates after a point. Methods such as NASformer [212] have better accuracy and fewer parameters than manually designed models such as Swin [7] and DeiT [52]. This highlights the strengths of these search methods.

IX. Hardware Optimization Techniques

Hardware optimization techniques for transformers play a crucial role in achieving efficient and high-performance computing. These techniques involve improving the design and architecture of hardware systems for efficient inference of the neural network models to optimize performance and energy efficiency. Table X shows key ideas of these techniques. In this section, we discuss hardware optimization techniques, such as pipelining (Section IX-A), optimizing matrix-multiplication operations (Section IX-B) and skipping redundant or trivial operations (Section IX-C). We also review dataflows to exploit reuse (Section IX-D), and block-circulant matrix to compress weight storage (Section IX-E).

A. Pipelining

Pipelining allows overlapping computation with data transfer or overlapping different sub-computations. It helps achieve load-balancing and is especially helpful for deep networks such as transformers, which have multiple encoder and decoder layers.

PipeBERT [253] is a pipelining technique to accelerate BERT models on big.LITTLE processors. This processor has two clusters, big and LITTLE, each with four cores. They divide the BERT network into two subgraphs and map one subgraph each on big and LITTLE clusters. Mapping is realized using the "affinity" functionality of CPU. They propose a latency-aware binary search algorithm to achieve a load-balanced division of the subgraph. They first observe the ratio, say R, of the throughput of BERT on big and LITTLE clusters. Let the total number of operations in the entire graph be Z. They start with the initial allocation of [Z-Z/R, Z/R] operations to [big, LITTLE] clusters. Then, the latencies of both clusters are recorded. If they differ by more than a threshold, half of the slower cluster's operations are moved to the lighter cluster. This process is repeated till the latency difference falls below the threshold. Their binary search algorithm completes much faster than naive binary search and brute-force search. The subgraphs on two clusters operate in a pipelined fashion. On the HiKey970 system, their technique achieves much higher throughput than executing on four big cores and a much lower "energy-delay product" than the best single-cluster execution.

Length Adaptive Co-design: Peng et al. [243] note that when the sequence lengths of the inputs in a batch vary greatly, e.g., in SQuAD v2.0, the maximum and average lengths are 975 and 171, respectively. The padding of inputs creates unnecessary overheads. They predict relative attention scores at low precision. Multiplication is realized through an LUT, e.g., with 4b integers, only 256 entry LUT is required. Then, $Q'K'^T$ is computed. After this, sorting is done to choose top-

TABLE X
Classification of Hardware-Level Techniques

Reducing computations	by skipping layers or input/output channels [237], exploiting patch locality [238], performing two multiplications in one go [118, 239], early terminating negative computations [239], avoiding trivial operations [240]
Predicting attention score using	LRT of Q and K [241], cosine of Hamming distance between two vectors [54], accounting for only the largest positive and negative products [145], ternary (0/-1/1) quantization of key matrix [242]
Loop-optimizations	unrolling [121], reordering [121], fusion [243]
Systolic array	[89, 119, 238, 244–246]
What is stationary in dataflow	output-block [247], output [245], weight [238, 245], key [248], sparse attention scores [246]
LUT for	log [133], multiplications [133, 243], reciprocal [54], exponent [89, 238], softmax [239], cosine values [54], adder-tree [247], inverse square root [119], GeLU [249]
Approximate computing	computing multiplication of large values exactly and small values approximately [239], truncating mantissa bits [240]
GD of weights	exploited for quantization [126, 132] and trivialization [240]
Intelligent data storage	storing encoder/decoder layers on-chip and embedding layer off-chip [250], storing weight matrices in HBM and remaining data in DDR [249]
Memory optimizations	matrix reordering to expose reuse in dilated window attention [245], splitting/compressing weight matrices to make them SA-friendly [119] or FPGA buffer-friendly [118], custom data-layout to remove bank-conflicts [117], formats for storing sparse matrices [121, 124]
Others	FFT [117, 117, 250, 251], tiling [245, 248, 249], early-exit [133], block-circulant matrix [250, 251], double-buffering [117, 120, 238, 252], load-balancing [241, 244, 246, 249], multi-FPGA accelerator [249], agglomerative clustering [132]
BERT	[54, 242, 246, 253]
NLP	[83, 89, 101, 106, 108, 117–119, 124, 126, 132, 133, 145, 239, 239, 241, 243, 245, 250]
CV	[237, 238, 240, 241, 245, 247, 248]

L (i.e., most dominant) attention scores. For these L values, exact MatMul and softmax are done to obtain QK^T. This is more efficient than computing the attention scores of all the elements. They split the encoder into three coarse-grain pipelines: (1) linear layer using MatMul and approximate attention computation. (2) attention computation and (3) FFN. Stages 1 and 3 utilize on-chip memory and loop allocation to overcome the memory wall. Stage 2 is further subdivided into a three-stage pipeline for trading resource utilization with data locality. Various attention operators are fused into a single loop using the fine-grain pipelining capability of FPGA. The reconfigurability of FPGA allows fusing loops with diverse iteration counts, whereas GPU can only fuse selected loops.

Variable input sequence lengths lead to stalling of the pipeline. They propose a sequence-length aware coarse-grain pipeline, which adjusts the resource allocation according to the requirement of a stage. It works based on the observation that with sparse attention, all operators have linear complexity. It sorts the inputs of a batch in decreasing order of length. Transformers usually have multiple encoder layers, and the inputs sequentially go through those layers. Their technique patches the pipeline stages of different sequence length inputs and different encoder layers. This removes pipeline bubbles and improves hardware efficiency. Pipelining and data-prefetching facilitate the overlapping of computing with data transfer. Their technique leads to better throughput than padding and cutting schemes. Their technique provides higher throughput for a marginal accuracy loss than CPU and GPU implementation and higher energy efficiency than a GPU design using CUBLAS GEMM.

B. Optimizing matrix-multiplication

Transformer computations involve the multiplication of large matrices, and hence, optimizing matrix multiplication can significantly boost computation efficiency.

Lu et al. [119] present a hardware accelerator for MHA and FFN blocks in the transformer. The computation of FFN is shown as $FFN(x) = ReLU(xW_1 + b_1)W_2 + b_2$. They note that all four tensors have a shape of $[batchSize, s, d_{model}]$. Both MHA and FFN blocks involve GEMM computations. In all the heads, the GEMM computations of linear sublayers can be done by a systolic array (SA) of size $s \times 64$. However, the use of this SA is still a challenge for large matrices such as W_1, W_2 and W_G. They note that in well-known transformers, $d_{model} = 64h$, and $d_{ff} = 4d_{model}$. Based on this, they divide the weight matrices W_1, W_2 and W_G as shown in Figure 32. Then, most GEMMs can be performed using an SA of size $s \times 64$, where s denotes the max sequence length. The use of a single SA saves hardware resources.

The $s \times 64$ SA has s rows and 64 columns, which produces the output matrix column by column. Then, the bias and the residual input are added using two sets of s adders. The softmax calculations are done in parallel to the calculations of "value" to enhance the hardware usage of this SA. They ensure that the softmax produces the output before the SA computes the "value" so that the overall latency is dictated by the SA and layerNorm block. In the softmax block, they avoid LUT and multipliers by using the "log-sum-exp" strategy, which uses linear approximation for logarithmic and exponentiation functions. Both MHA and FFN blocks need to compute layerNorm; hence, layerNorm is on the critical path. They minimize its latency by starting their computation early. They implement their technique on FPGA and show that it provides high speedup compared to the GPU.

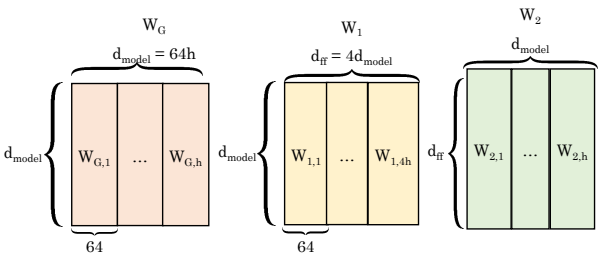


Fig. 32. Division of weight matrices in the technique of Lu et al. [119]

DFX: Hong et al. [249] present a multi-FPGA accelerator for text-generation workloads such as GPT. The GPT-2 model produces tokens in two stages: summarization and generation. These stages have different properties. The parallel computing capability of GPU makes it effective for summarization; however, the sequential nature of generation renders GPU ineffective. Acceleration of GPT-2 requires accelerating not only attention and FFN but LayerNorm, LM head, residual and token embedding. While residual and LayerNorm account for only 0.1% of operations, they account for 22.8% of the GPT-2 latency. This highlights the need for a custom accelerator. Further, given the massive size of GPT-2, a single FPGA device is insufficient, and hence, they connect a cluster of four FPGAs with a single CPU host.

In pipelining, all the workers process one input completely, leading to large latency. Instead, they use intra-layer model parallelism, which reduces MatMul latency and incurs low synchronization delay. Every FPGA (worker) handles weight matrices at head granularity in MHA and column granularity in FC layers. Unlike GPU cores, which depend on massive parallelism, their compute core is optimized for processing individual tokens. Their ISA includes (1) matrix instructions for performing matrix-vector operations, (2) vector instructions for performing vector-vector and vector-scalar operations. The vector instructions execute LayerNorm and softmax, (3) DMA instructions and (4) router instructions. An FPGA has 8GB HBM and 32GB DDR with bandwidths of 460GB/s and 38GB/s. The weight matrices are kept in HBM since they are frequently accessed. The remaining infrequently accessed data are kept in DDR.

In conventional attention operation, the key is transposed; however, they transpose 'value' since it is read column-wise but written row-wise. Thus, transpose is performed inherently during the write operation, not the read operation. To hide the latency of the transpose operation, they start it in advance while Q and K are being computed. Both matrix and vector operations are mapped to DSPs, whereas non-linear operations are performed using DSP, LUT and combinational logic. GeLU is realized using piecewise linear approximation and

an LUT. A parallel tree of comparators is used for finding the largest value of a vector. A router is used to synchronize different FPGAs over a ring network. They evaluate their technique using four Alveo U280 FPGAs and show that it is superior to four V100 GPUs in performance, energy efficiency and cost efficiency.

C. Skipping redundant, ineffectual or trivial computations

A large fraction of computations performed in transformers are either repeated (e.g., due to patch locality), have no impact on the final output (because only high attention scores decide the final result) or are trivial (e.g., multiplication with zero or one). Identifying and avoiding such computations can boost efficiency significantly.

Since softmax is a normalization approach, what matters is only the relative values of attention scores and not the absolute values. Based on this, some works [241, 243, 246] quantize K and Q from the FP32 value to a low-precision (1b or 4b) integer format. Since quantization and exponential are monotonic functions, the relative ordering of attention scores is maintained. Compared to 16-bit dense attention, 4-bit computations incur only 1/16X overhead. A few works predict attention scores based on low-rank transformations (LRT) of Q and K [241].

ELSA technique [54] reduces computations in attention operation. Without performing full computations, they seek to select, for every query, the key that would lead to high attention scores. They compute approximate self-attention in three steps (1) They compute Hamming distance between k-bit hashes of two vectors (say key and query) to guess the angle between them. The hash function is computed by using a structured orthogonal matrix. (2) A higher value of the cosine of the angle shows a higher value of dot-product between them and hence, the higher similarity between them. (3) The relevance of a key to a query is ascertained by comparing the approximate similarity to a threshold. Then, irrelevant relations are filtered out, and the exact dot-product is computed only for significant relations. While computing exact dot-product requires d multiplications, their algorithm requires only two multiplications, a cosine function and Hamming distance calculation. A LUT of cosine values can further reduce the computations. Based on the approximation acceptable to a user (s), their technique decides the threshold for each sub-layer. For this, the target model is run with the training set, and for a sub-layer, the keys whose softmax-normalized attention score is more than s/n are considered relevant, where n is the count of input elements. With only 1% accuracy loss, their proposed accelerator provides high speedup and energy efficiency gain compared to the GPU.

DOTA [241] technique reduces computation wastage. Although 90%

Long-sequence transformers operate on large GEMM/GEMV computations with configurable hidden dimensions. Hence, they identify the building blocks of each computation and propose a single architecture for efficiently handling various layers. They propose a reconfigurable MM engine (RMME) to support multi-precision calculations at row granularity. RMME is a 32*16 array of PEs that can support 16b fixed-point, INT2, INT4 and INT8 calculations. RMME initially estimates $\tilde{S}$ in INT2 or INT4 precision and then selects significant attention scores. For the sake of hardware-friendliness, it selects an equal number of attention connections in all the rows of the attention matrix. Then, only significant attentions are computed in 16b fixed-point precision. The QK^T results are converted to FP before computing the softmax function to avoid overflow. The softmax output is again quantized to perform $A * V$ in fixed-point. Their technique provides large gains over a GPU.

The A^3 technique [145] proposes two approximation schemes for accelerating attention computations. (1) Since key and value matrices are acquired at the time of knowledge understanding and not query response, they preprocess the key matrix to reduce the number of operations required and query response time. Specifically, for a d-dimensional query, the dot-product between a query and a key matrix row is a sum of multiplications of each dimension. Their key idea is that if the multiplication of a single dimension is a large positive number, the final dot-product result is also expected to be large. If this value is a large negative number, the final result is unlikely to be a large positive number.

Their technique seeks to estimate a "rough-score" for each row in a greedy fashion. This is illustrated in Figure 33. In J^{th} iteration, their technique checks the Jth largest and smallest number in the result (key*query) and adds them to the rough score of their row. This is repeated for Z iterations. The rows with positive rough-score have large positive elements and vice versa. Then, their technique further processes only the rows having positive rough-score. They sort the key matrix during knowledge comprehension to reduce critical path delay. Then, when the query arrives, the above technique is applied. The preprocessing overhead is easily amortized in networks where many queries use the same key matrix (e.g., 320 in BERT).

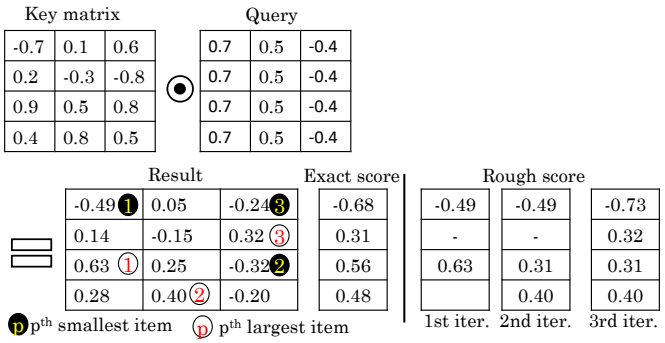


Fig. 33. Approximate computation of attention score ($Z = 3$) in the technique of Ham et al. [145]

(2) They compute the attention score of the above-selected rows exactly and further note that the softmax function transforms score values into weights, such that small scores lead to small weights. These small weights do not impact the model validation accuracy. In fact, they arise because of using a differentiable version of the argmax function, which is required in training, not inference. Hence, these near-zero weights can be treated as zeros. Based on this idea, only if a row's score is

within $T\%$ of the top row's score, it is passed to the softmax function. They quantize FP inputs to FX and use different bitwidths in different layers. They also propose an accelerator for their technique, achieving magnitude-order performance and energy efficiency gains over CPU and GPU.

Chen et al. [242] present modifications to the A^3 technique[145]. They store a key' matrix, such that near-zero/positive/negative numbers in key matrix becomes 0/1/-1 in the key' matrix. On multiplying key' with the query, the relative ordering of attention scores does not change, as illustrated in Figure 34. Their technique allows finding Y rows with positive rough-score. Further, while Ham et al. [145] perform exact dot-product for these rows and then further shortlist top-K rows, Chen et al. [242] select top-K rows from the Y rows selected based on their rough-scores. Their technique causes a 1% accuracy loss on the BERT model while boosting energy efficiency.

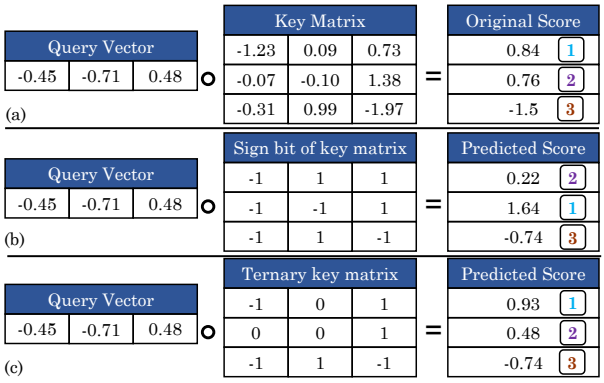


Fig. 34. Ternary pruning [242]

Wang et al. [239] present three approximate-computing techniques for accelerating transformer computations. (1) To avoid the ineffectual computations due to weakly-related tokens, they present a PE that calculates small values with high errors for saving energy but computes large values exactly since these strongly-related tokens have a high impact on the final attention score. It uses an inexact multiplier that gauges the magnitude from the MSB and then, for small values, gates the LSB computations. Although this multiplier produces a high error, the softmax operation reduces the magnitude of those errors.

(2) In $Q * K^T$ computation, near-zero values from softmax become zero after N-bit quantization. This sparsity pattern depends on X_{max}, which varies for every row. To deal with this issue, they present a "bidirectional asymptotic speculation engine". It uses diagonal-prior computing to find row-wise X_{max}. It first performs positive MACs and, on observing sparsity, terminates negative computations. A token is strongly related to its neighboring tokens. Based on this locality property of attention, they first compute eight scores on the matrix diagonal to find an estimate of X'_{max}. An update-unit iteratively generates a new estimate of X_{max} and compares it with X'_{max}, to finally obtain actual X_{max}. From this, it predicts sparsity and avoids redundant computations.

(3) For the near-zero probability values, many MSB values are zero. Due to them, in $P*V$, many partial products are zero. To address this issue, they present a scheduler that reorders operands to merge two operations in a single multiplication. This removes zero-valued partial products. They fabricate a die that takes only 6.8mm^2 area, 272mW power and achieves 14.28TOS/W at 510MHz. Their processor outperforms GPU and previous accelerators.

Dynamic ViT Inference: Sreedhar et al. [237] note that recent vision transformers [7, 8, 254–256] use CONV backbones and CONV layers in the encoder. They evaluate the impact of selective skipping of computations in CONV and attention layers on the predictive performance of object detection and semantic segmentation applications. In DETR and deformable DETR [8, 257] object detection models, the transformer accounts for nearly 15% of execution time, whereas ResNet-50 accounts for the remaining time. Especially at higher batch-size, the latency of ResNet-50 increases further. In Segformer and Swin-tiny semantic segmentation models, ~70 and ~90% of total computations are in CONV layers. However, CONVs account for only 26% of latency on GPUs since CONV is implemented efficiently on GPUs. MatMul and softmax operations of attention account for 27% of latency. Thus, both CONV and MatMul must be accelerated in the overall semantic segmentation application.

They study the resilience of semantic segmentation models towards computation bypassing techniques. Here, they modify the model execution graph to execute subsets of the original model using (1) skipping a layer, (2) reducing the input or output channels of a layer (3) decreasing sampling scale factors in spatial-downscaling attention. The Pareto-optimal sub-models are identified that achieve higher accuracy with lower resource consumption. In Segformer-B2, scheme (3) degrades accuracy without reducing latency. Rather, Pareto-optimal points are obtained by changing the number of encoder layers in every stage and the input channels to CONV layers. This technique reduces mIoU (mean intersection over union) by 6 percentage points on the ADE20K dataset while bringing a 17% reduction in latency. This bypassing approach without retraining can save up to 25% of execution latency. However, saving 50% latency requires retraining the model. The Swin-Tiny model does not have similar resilience as Segformer-B2. They further propose a dynamic inference technique, which performs computation skipping by using an LUT to store Pareto-optimal configurations and their latency saving. During inference, a configuration is searched from this LUT based on the target and resource constraint. Their technique populates this LUT during inference and avoids the need for pruning+retraining.

DiVIT technique [238] is based on the observation that since neighboring patches of ViT have high locality, many computations in self-attention operation are redundant. It proposes a "delta encoding" technique to exploit patch locality, shown in Figure 35. In the transformer, an input feature x is multiplied by the weight w. If a neighboring patch is x', then $\delta x = x' - x$ shows the delta patch. In a bit-serial multiplier, the multiplication workload depends on the number of non-zero bits. With patch locality, δx has fewer non-zero bits than x'. Based on this, delta encoding replaces $x' * w$ with $x * w + \delta x * w$. By reusing the value of x*w from a neighboring patch, their

technique can significantly accelerate multiplications in MHA without losing accuracy.

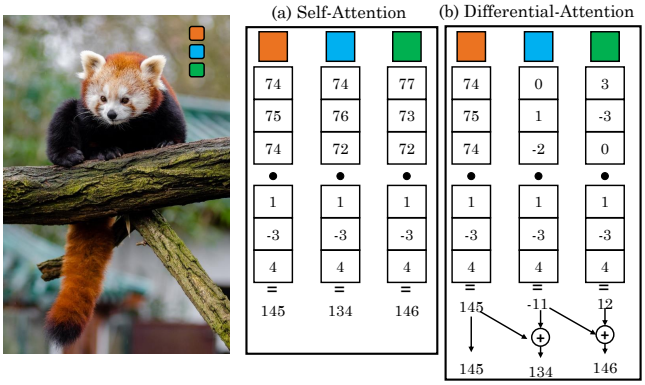


Fig. 35. Delta encoding in technique of Li et al. [238]

Their proposed accelerator computes differential attention based on the "bit-pragmatic architecture" [258], which processes only non-zero bits of input features in a bit-serial manner. Their PE array computes the input patch features in a bit-serial manner. The offset generators change the patch features into powers of two. In every cycle, a weight is multiplied with a power of two using a shifter. Then, an add or subtract operation is performed, depending on the sign. Their engine can parallelly compute 32 patches while reusing the same group of weights.

They use a 32*32 PE array, where neighboring patches are assigned to neighboring columns. Thirty-two channels of patch features are assigned to every column. Instead of multipliers, this engine uses shifters and a 32-input adder tree. The PEs in the same row reuse the same group of 32 weights. Specifically, the first column works on the raw values of the patch. After finishing, it passes the result to the second column, which works on the second patch feature in a delta computation manner. This continues until the last column (number 32), which passes its result back to column 1 for computation on the next set of 32 patches. This pipelining operation hides the latency of accumulating delta results. An SA handles the MLP units. Their accelerator shows high speedup and energy efficiency gains over CPU and GPU.

AxBy-ViT technique [240] avoids trivial operations in ViT. In the operation a*b, if a or b is zero, it is termed fully trivial, whereas if a or b equal ± 1, it is termed semi-trivial. They note that the operands in input-embedding, MHA, MLP and FC computations show Gaussian distribution such that more than 50% operands are concentrated around zero. Also, a small fraction of values in input-embedding and FC are ± 1. They note that exactly matching the operands with 0 or ± 1 finds the scope of trivialization in only 0.1% of the operands.

They propose two approximate matching techniques that increase this scope by 1000×. (1) Assuming FP32 operands, values having biased exponents lower than a threshold (θ) are considered zero. For example, if θ is 121, then any $|x|$ below 0.015625 is approximated to zero. Say, for 0.01171875, the biased exponent is 120, which is smaller than 121, and hence, it is approximated to zero. This can be seen from the fact that $0.01171875 < 0.015625$. (2) To match a number with ± 1, they truncate certain mantissa bits and then compare the number. For example, on truncating 19 (K) bits, 1.015625 has the same representation as 1.0. However, on truncating only 16 bits, its representation is not similar to that of 1.0. By approximating 1.015625 to 1.0, a computation such as 1.0001*6 reduces to 1*6, which is trivial. On increasing θ and K, the error increases exponentially. On setting θ to 121, MHA and MLP show an accuracy drop below 3%, but input-embedding and FC show a larger accuracy drop. Thus, different blocks in ViT have different error tolerance.

D. Dataflows for exploiting reuse

In a hardware accelerator, it is important to choose the right dataflow to exploit reuse and improve hardware utilization. For example, input stationary (IS) and weight stationary (WS) dataflows reduce the energy of reading inputs and weights, respectively, whereas output stationary (OS) reduces the energy of accessing the partial sums.

Output Block Stationary (OBS): Zhao et al. [247] note that since CNN kernels show substantial weight-sharing, using WS dataflow is advantageous for CNN accelerators. However, transformer models show much less weight reuse than CNN, rather they show a comparable amount of input and weight sharing. They propose output-block stationary dataflow, which divides the matrix-multiplication operation into several smaller matrix multiplications. To enable reuse, it uses two levels of broadcasting. Consider the matrix-multiplication $P = A \times B$. (1) They divide A and B into $b \times b$ block matrices and compute the resultant block P_{ij} as $\sum_k A_{ik} B_{kj}$. To compute four output block-matrices shown in Figure 36, they broadcast four blocks, viz., A_{ik}, $A_{(i+1)k}$, B_{kj}, and $B_{k(j+1)}$. (2) Inside a block-matrix, they broadcast the rows of A_{ik} and columns of B_{kj}, as shown in Figure 36. In comparison with OS, the two-level broadcasting approach of OBS lowers the memory accesses for input and weight. Further, it reduces the requirement of output writing bandwidth. However, OBS has a slightly lower hardware utilization (96% vs 89%).

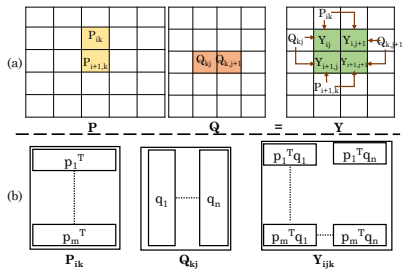


Fig. 36. In OBS dataflow [247], broadcasting at (a) block and (b) vector level

The partial sums of the output are accumulated in the DSP using an LUT-based adder tree. For a block size of b, a block-multiplication unit requires b^3 DSPs. At $b = 1$, OBS dataflow reduces to OS dataflow. Given the limited number of DSPs, b value can be at most 8. They observe that $b = 8$ leads to lower energy consumption than $b = 4$, and hence, they choose $b = 8$. They experimentally compare WS with OBS for the "transformer-in-transformer" (TNT) network. In TNT,

an "outer TE" embeds an "inner TE", and they extract patch- and pixel-level attention, respectively. The outer TE accounts for more than 83% of operations, and OBS provides nearly 84% utilization for this. OBS lowers memory access by more than 6.7× and power consumption by one-third.

Sanger [246] is a technique to accelerate sparse attention models. Unlike ReLU, which leads to exact-zero activations, the softmax operation does not output exact zero at any position. Hence, the authors use a threshold to zero out small attention weights. This creates an attention mask with unstructured sparsity. Also, the dynamic sparsity patterns are unstructured and hence, hardware-unfriendly. To deal with this issue, they first vertically split the attention mask into sub-matrices of the same width as the vertical input ports of the PE array. Then, sub-rows with all zero elements are removed. After this, to keep comparable non-zero elements in each sub-row, those with more non-zero elements than PE array row-width are split into multiple rows. This creates structured blocks with an almost equal load. This process is illustrated in Figure 37.

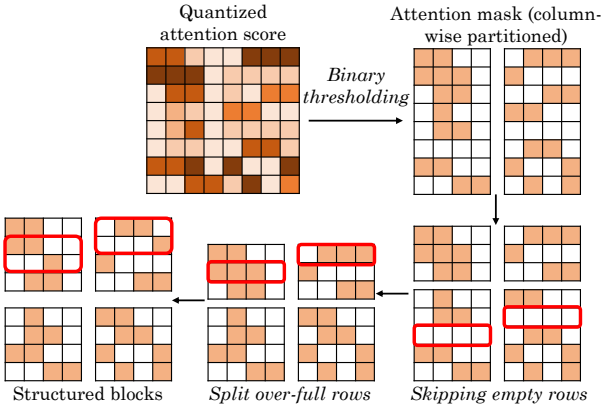


Fig. 37. Encoding of attention matrix into fine-granular structured blocks [246]

Still, the exact distribution of non-zero elements can be random. This creates non-uniform data accesses in SDDMM and SpMM operations. The output of SDDMM is the score matrix, which is fed as input to SpMM. Since Q/K/V matrices are dense, but the score matrix is sparse, it leads to decoding overheads. They propose a "score-stationary" dataflow, which stores sparse scores in PE till the end of computations. In a 4*4 block, with every row containing two non-zero values, they use a 4*2 PE array. Then, bubbles are inserted, corresponding to the zero locations, representing a ghost PE. This alleviates decoding overheads. For this, the input data registers are dynamically connected with the PEs based on the attention mask. Further, this unifies SDDMM and SpMM operations and hence, saves chip area since a single SA can be used for both of them. Still, there are some differences between these two operations; hence, they reconfigure the SA between them. Their technique can prune the network to 0.08-0.27× model size without accuracy loss and attains high speedup over CPU, GPU and previous attention accelerators.

SALO [245] is a technique to speed up attention for tasks with long input sequences. Their accelerator has a data scheduler and a spatial engine. The sparse attention pattern is characterized by the window size (w) of sliding window attention and dilation (d) of dilated window attention. The spatial engine has a PE array that maximizes data reuse by computing sliding window attention using diagonal connections. The engine also has a global PE row/column for computing global attention.

Their datapath seeks to optimize data reuse between (R1) various queries within sliding window attention and between (R2) sliding window attention and global attention. For (R1), they note that if q_i attends to key vectors $[k_{i+a}, k_{i+a+1} \dots k_{i+b}]$, then, q_{i+1} attends to key vectors $[k_{i+a+1}, k_{i+a+2} \dots k_{i+b+1}]$. Hence, $b - a$ key vectors are reused. Their engine streams input k and v vectors diagonally and q vectors horizontally, as shown in Figure 38. An incoming key vector k_{i+b+1} replaces the outgoing vector k_{i+a}, and the same strategy is used for replacing value vectors. For (R2), the K/Q of global tokens attend to all the K/Q of input sequences. Since SW attention works on all Q/K/V elements, global attention needs no additional data access. Hence, both attention computations can be performed simultaneously.

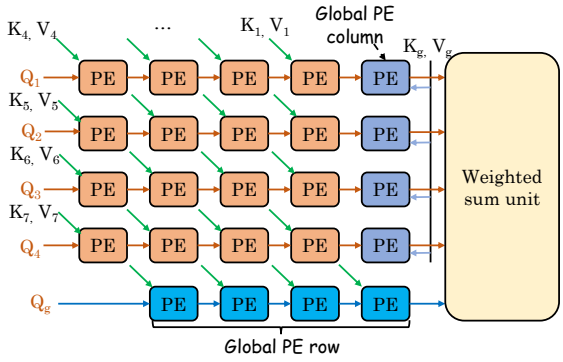


Fig. 38. Accelerator design of Shen et al. [245]

They further propose data division and reorganization schemes to convert hybrid sparse attention into a pattern suitable for their accelerator. They divide matrices into tiles and compute attention in multiple passes. Specifically, they divide the sequence and the window based on the PE array dimensions. Window division splits an attention computation into multiple parts that are weighted and finally accumulated. This is done by the weighted sum unit. The dilated window attention cannot be directly executed on their dataflow. They propose reordering Q matrix such that q_i is grouped with q_{i+d}, q_{i+2d}, etc. This exposes reuse in dilated window attention with a period d, which is similar to sliding window attention. Hence, it can be accelerated on their spatial engine.

Their 2D SA performs computations in five stages: (1) The SA multiplies Q and K using output-stationary dataflow. (2) exponential is computed using the piece-wise linear function that the MAC unit can compute. (3) Exponential values of PEs in the same row are accumulated ($\sum Exp(S_{ij})$ to compute the denominator of softmax. At the last PE of a row, $(\sum Exp(S_{ij})^{-1}$ is computed and broadcast to PEs in a row. This avoids the need for a divider circuit. (4) Every PE computes $S'_{ij} = Exp(S_{ij} \times (\sum Exp(S_{ij})^{-1}$. (5) PE_{ij} now receives value vector v_j. Using weight stationary dataflow,

each PE_{ij} computes $v_j \times S'_{ij}$, adds it to the previous partial sum and passes it to the next PE. The PE row output enters the weighted sum unit and is merged with the previous output. The global PE row computes the attention of queries of global tokens and hence, reuses the K/V vectors from the PE array. Analogously, the global PE column reuses Q vectors. Due to data division, a single row/column suffices to handle global tokens in real-world applications. On ViL and Longformer, their technique achieves higher speedup and energy efficiency compared to CPU and GPU.

ViTCoD [248] technique seeks to mitigate data movement and PE under-utilization issues in accelerators for ViT workloads. In the NLP transformers, the token count depends on the input, but ViTs use a fixed token count (e.g., 196). This avoids the need to predict sparse attention schemes and simplifies the design of hardware accelerators. Further, to keep the accuracy loss small, more than 90% of attention maps in ViTs can be pruned with fixed sparsity layouts for all inputs, whereas in NLP transformers, only 50 to 70% sparsity can be achieved even with dynamic sparsity layouts. However, this high sparsity in ViTs can create multiple challenges. In ViTs, the non-zero elements in sparse attention maps are found in the diagonal lines. This causes a significant data movement and under-utilization of PEs.

They first compute average and normalized attention maps by applying the pretrained models on all training images and then prune based on the amount of the remaining information. To lower the irregularity of resultant sparse maps, they find and cluster the query-key pairs into just two types: sparser or denser. Here, the tokens with more than θ non-zero elements are brought to the front as the denser pattern, and the remaining tokens are kept as the sparser pattern. This is shown in Figure 39. The denser layouts capture the tokens having a substantial correlation with the remaining tokens. The sparser layouts are those except the diagonal lines, where a majority of values are zero. This happens because a high correlation is found in neighboring tokens. After reordering, they finetune the model to restore the accuracy. The above reordering operation enforces fixed sparse attention patterns, which do not change during finetuning. The sparser layouts suffer from high data-movement overhead. By exploiting the redundancy across attention heads, an auto-encoder compresses K and Q to 50% of their original size. Evidently, the pruning+reordering technique reduces computations, and the auto-encoder trades computations for reducing memory accesses.

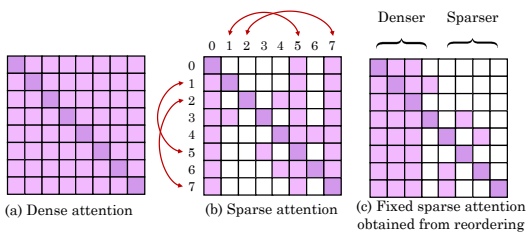


Fig. 39. Reordering of sparse attention weight matrices [248]

Their proposed accelerator has separate engines for processing dense and sparse layouts. The denser engine performs $Q.K^T$ (during sampled GEMM) and $S.V$ (during SpMM). The sparser engine performs the rest of the irregular computations, including activation functions. Since both denser and sparser engines work in parallel, they will likely work on the same Q vectors. Hence, the sparser engine first queries the Q buffer of the denser engine and, only in case of a miss, accesses the off-chip memory. They use K-stationary dataflow, which reads K vectors and sequentially multiplies them with different Q vectors. It outputs attention values in a column-wise manner. This dataflow enables the full reuse of K vectors and requires a small on-chip storage for storing the interim results. This dataflow does not spatially map and multiply all Q/K features but only multiplies paired features depending on the non-zero indexes in S. Under 90%

E. Block-circulant matrix for reducing weight storage

The block-circulant matrix (BCM) reduces the weight storage by replacing the original weight matrix with one or more blocks of circulant matrices, where every row/column is a cyclic reformulation of the other. This allows storing only one first row/column vector, called the index vector, instead of the entire matrix. Further, matrix-vector multiplication can be replaced by an FFT operation, which reduces the complexity from $O(b^2)$ to $O(b \log b)$, where b is the row (or column) size of the circulant matrix.

Ftrans [250] technique proposes an improved BCM approach for compressing transformer models. The previous works on BCM only take the first row/column as the index vector but fail to represent the remaining rows/columns. To better capture the parameter values and maintain accuracy, they take $P_{ij} = (\sum_{j=1}^{b} W_{ij})/b$, where W_{ij} is the circulant matrix. Thus, they also take the representation of the remaining rows/columns. The encoder/decoder layers, which account for two-thirds of total parameters, are stored on-chip, which reduces off-chip communication. The embedding layer has only one-third of the total parameters; hence, they store it off-chip. They develop two PEs for performing matrix-vector multiplication of different matrix sizes and one PE for performing FFT/IFFT. The matrix-vector multiplication PEs can also perform scaling and softmax in an overlapped manner with matrix multiplication. Computation of KV^K etc. takes much larger time than the computation of MHA. To achieve a balanced pipelined execution of different layers, they use a scheduling algorithm to allocate more resources (e.g., the number of PEs) to the slowest layer. Their technique brings up to 16X reduction in model size and also improves performance and energy efficiency compared to CPU.

X. Conclusion and Future Work

Transformers have become one of the most widely used architectures in NLP and computer vision domains due to their capability to capture long-range interdependencies. As the complexity of the transformers continues to grow, there is an increasing need for model compression and hardware optimization methods to accelerate these models. To address the enormous challenges the transformers face, researchers have developed several model enhancement techniques, including

pruning, quantization, neural architecture search, distillation, and lightweight self-attention design. This paper provides taxonomy and a comprehensive overview of the recently proposed inference optimization techniques for transformer-based networks. We discuss the optimization methods, both from transformer architecture and hardware perspectives. We plot the predictive performance against the number of parameters/FLOPs of several optimization techniques to provide meaningful insights. We conclude this paper by emphasizing the future directions in this rapidly evolving field of research. We hope our effort will create further interest in developing novel optimization methods to facilitate efficient inference of large models on a wide range of hardware.

The development of optimization techniques for efficient transformer inference has become an important area of research due to their broad applications across various architectures and domains. The current solutions have effectively developed more robust and efficient algorithms for the family of transformer architectures. However, there is still ample room for improvement in both algorithmic and hardware optimization methods. Several challenges exist, such as scalability, interpretability, and fair comparison of the performance of different optimization techniques. The following list identifies the limitations and gaps in current research to enhance the development of efficient and effective methods:

AutoML for Model Compression: The automatic ML and NAS techniques are becoming mainstream for designing networks for several architectures, datasets and tasks. These methods have great potential to be applied to transformer model compression. This could involve exploring different search approaches, such as automatic identification of sparse weights and mixed precision quantization and obtaining optimal models in terms of accuracy and computational efficiency. While AutoML methods for CNNs has received significant attention, their use for CV transformers has not been studied in as much detail. There is a need to use AutoML for transformer model compression to improve the scalability and efficiency and adapt them to a wider range of application areas.

Hardware-aware NAS for CV transformers: The HW-NAS methods for CNNs has been well-studied on various platforms, while the HW-NAS techniques for transformers are not fully explored, especially for computer vision. Although hardware-agnostic NAS algorithms on transformer architectures are useful for research, hardware-aware methods are crucial for real-time deployment. Thus, better hardware-aware search space and NAS algorithms are needed to improve accuracy and performance on multiple platforms.

Transformer NAS benchmarks: The computation resource demand is a major bottleneck for developing novel transformer NAS algorithms, as it requires training and evaluation of several neural architectures during the search process. Also, result reproducibility is challenging due to differences in search space and experiment setting; thus, comparing different methods becomes unfair. In this context, several NAS benchmarks [259, 260] have been developed, which act as a look-up-table to obtain the validation accuracy without actually training them. HW-NAS benchmarks [261] help non-hardware researchers by providing hardware performance metrics of the neural architectures on the selected platforms. Moving forward, transformer NAS and HW-NAS benchmarks will play a key role in developing better NAS algorithms.

Transformer Inference Benchmarks: Some of the existing benchmark suites do not include transformer networks. There is a need to include SOTA transformer networks in such benchmark suites to accelerate and standardize the research on transformers.

Evaluating existing optimization techniques on emerging topologies: The current optimization techniques are effective on standard transformer models on widely used datasets and applications. Recently, transformer/attention modules have been utilized in emerging neural architecture topologies, such as image segmentation [262], graph neural networks (GNN) [263], graph convolutional network (GCN) [264], 3D convolutional neural networks [265], point clouds [266] and spiking transformer neural network [267]. Therefore, it is crucial to expand the scope of model compression techniques to generalize over other tasks and emerging network topologies.

Sparse transfer learning: Sparse transfer learning refers to leveraging the sparsity pattern learned by a large and pre-trained teacher model to fine-tune a smaller model. In this way, the sparsity map learned by the large model can be transferred to the smaller model without the need for pruning and additional fine-tuning [268]. This process is nascent and requires experimentation and investigation on several transformer architectures and hardware platforms.

Reliability and security: The model compression techniques can weaken fault-resilience and adversarial-robustness of a model by removing parameters that are unimportant for normal execution but are useful against perturbed input data or faulty weights [269, 270]. As transformers get deployed in critical applications, future work must evaluate the adversarial robustness and fault-tolerance of the proposed optimization methods.

ACKNOWLEDGEMENT

The research reported in this paper was funded in part by the Philip and Virginia Sproul Professorship, and the National Science Foundation grants MRI 1726447 and MRI 2018594 at Iowa State University. This research was also supported by the Argonne Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DEAC02-06CH11357. All opinions, findings, and conclusions expressed are those of the authors.

REFERENCES

[1] P. Research, "Artificial intelligence (ai) market size, growth, report 2022-2030," 2022. [Online]. Available: https://www.precedenceresearch.com/artificial-intelligence-market

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[3] J. Schulman, B. Zoph, C. Kim, J. Hilton, J. Menick, J. Weng, J. Uribe, L. Fedus, L. Metz, M. Pokorny *et al.*, "ChatGPT: Optimizing language models for dialogue," https://openai.com/blog/chatgpt, 2022.

[4] G. BARD, 2023. [Online]. Available: https://bard.google.com/

[5] M. Zvyagin, A. Brace, K. Hippe, Y. Deng, B. Zhang, C. O. Bohorquez, A. Clyde, B. Kale, D. Perez-Rivera, H. Ma *et al.*, "GenSLMs: Genome-scale language models reveal SARS-CoV-2 evolutionary dynamics," *bioRxiv*, pp. 2022–10, 2022.

[6] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *International Conference on Learning Representations*, 2020.

[7] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 10 012–10 022.

[8] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European conference on computer vision*. Springer, 2020, pp. 213–229.

[9] P. Li, J. Yang, M. A. Islam, and S. Ren, "Making AI Less" Thirsty": Uncovering and Addressing the Secret Water Footprint of AI Models," *arXiv preprint arXiv:2304.03271*, 2023.

[10] M. Dehghani, J. Djolonga, B. Mustafa, P. Padlewski, J. Heek, J. Gilmer, A. P. Steiner, M. Caron, R. Geirhos, I. Alabdulmohsin *et al.*, "Scaling vision transformers to 22 billion parameters," in *International Conference on Machine Learning*. PMLR, 2023, pp. 7480–7512.

[11] A. Glaese, N. McAleese, M. Trebacz, J. Aslanides, V. Firoiu, T. Ewalds, M. Rauh, L. Weidinger, M. Chadwick, P. Thacker *et al.*, "Improving alignment of dialogue agents via targeted human judgements," *arXiv preprint arXiv:2209.14375*, 2022.

[12] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[13] B. Kim, H. Kim, S.-W. Lee, G. Lee, D. Kwak, J. D. Hyeon, S. Park, S. Kim, S. Kim, D. Seo *et al.*, "What changes can large-scale language models bring? intensive study on hyperclova: Billions-scale korean generative pretrained transformers," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 3405–3424.

[14] R. Taylor, M. Kardas, G. Cucurull, T. Scialom, A. Hartshorn, E. Saravia, A. Poulton, V. Kerkez, and R. Stojnic, "Galactica: A large language model for science," *arXiv preprint arXiv:2211.09085*, 2022.

[15] A. Zeng, X. Liu, Z. Du, Z. Wang, H. Lai, M. Ding, Z. Yang, Y. Xu, W. Zheng, X. Xia *et al.*, "GLM-130B: An open bilingual pre-trained model," *ICLR*, 2023.

[16] R. Thoppilan, D. De Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, Y. Du *et al.*, "Lamda: Language models for dialog applications," *arXiv preprint arXiv:2201.08239*, 2022.

[17] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," *International Conference on Learning Representations*, 2021.

[18] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[19] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders *et al.*, "WebGPT: Browser-assisted question-answering with human feedback," *arXiv preprint arXiv:2112.09332*, 2021.

[20] S. Iyer, X. V. Lin, R. Pasunuru, T. Mihaylov, D. Simig, P. Yu, K. Shuster, T. Wang, Q. Liu, P. S. Koura *et al.*, "OPT-IML: Scaling Language Model Instruction Meta Learning through the Lens of Generalization," *arXiv preprint arXiv:2212.12017*, 2022.

[21] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 730–27 744, 2022.

[22] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "OPT: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[23] K. Shuster, J. Xu, M. Komeili, D. Ju, E. M. Smith, S. Roller, M. Ung, M. Chen, K. Arora, J. Lane *et al.*, "Blenderbot 3: a deployed conversational agent that continually learns to responsibly engage," *arXiv preprint arXiv:2208.03188*, 2022.

[24] N. Muennighoff, T. Wang, L. Sutawika, A. Roberts, S. Biderman, T. L. Scao, M. S. Bari, S. Shen, Z.-X. Yong, H. Schoelkopf *et al.*, "Crosslingual generalization through multitask finetuning," *arXiv preprint arXiv:2211.01786*, 2022.

[25] O. Lieber, O. Sharir, B. Lenz, and Y. Shoham, "Jurassic-1: Technical details and evaluation," *White Paper. AI21 Labs*, vol. 1, 2021.

[26] Z. Zhang, Y. Gu, X. Han, S. Chen, C. Xiao, Z. Sun, Y. Yao, F. Qi, J. Guan, P. Ke *et al.*, "CPM-2: Large-scale cost-effective pre-trained language models," *AI Open*, vol. 2, pp. 216–224, 2021.

[27] S. Wu, X. Zhao, T. Yu, R. Zhang, C. Shen, H. Liu, F. Li, H. Zhu, J. Luo, L. Xu *et al.*, "Yuan 1.0: Large-scale pre-trained language model in zero-shot and few-shot learning," *arXiv preprint arXiv:2110.04725*, 2021.

[28] Y. Sun, S. Wang, S. Feng, S. Ding, C. Pang, J. Shang, J. Liu, X. Chen, Y. Zhao, Y. Lu *et al.*, "Ernie 3.0: Large-scale knowledge enhanced pre-training for language understanding and generation," *arXiv preprint arXiv:2107.02137*, 2021.

[29] J. W. Rae, S. Borgeaud, T. Cai, K. Millican, J. Hoffmann, F. Song, J. Aslanides, S. Henderson, R. Ring, S. Young *et al.*, "Scaling language models: Methods, analysis & insights from training Gopher," *arXiv preprint arXiv:2112.11446*, 2021.

[30] S. Smith, M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhumoye, G. Zerveas, V. Korthikanti *et al.*, "Using DeepSpeed and Megatron to Train Megatron-Turing NLG 530B, A Large-Scale Generative Language Model," *arXiv preprint arXiv:2201.11990*, 2022.

[31] K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl *et al.*, "Large language models encode clinical knowledge," *arXiv preprint arXiv:2212.13138*, 2022.

[32] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "PaLM: Scaling language modeling with Pathways," *arXiv preprint arXiv:2204.02311*, 2022.

[33] A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo *et al.*, "Solving quantitative reasoning problems with language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 3843–3857, 2022.

[34] Y. Tay, J. Wei, H. W. Chung, V. Q. Tran, D. R. So, S. Shakeri, X. Garcia, H. S. Zheng, J. Rao, A. Chowdhery *et al.*, "Transcending scaling laws with 0.1% extra compute," *arXiv preprint arXiv:2210.11399*, 2022.

[35] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma *et al.*, "Scaling instruction-finetuned language models," *arXiv preprint arXiv:2210.11416*, 2022.

[36] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "GShard: Scaling giant models with conditional computation and automatic sharding," *arXiv preprint arXiv:2006.16668*, 2020.

[37] X. Ren, P. Zhou, X. Meng, X. Huang, Y. Wang, W. Wang, P. Li, X. Zhang, A. Podolskiy, G. Arshinov *et al.*, "Pangu-σ: Towards trillion parameter language model with sparse heterogeneous computing," *arXiv preprint arXiv:2303.10845*, 2023.

[38] M. Artetxe, S. Bhosale, N. Goyal, T. Mihaylov, M. Ott, S. Shleifer, X. V. Lin, J. Du, S. Iyer, R. Pasunuru *et al.*, "Efficient large scale language modeling with mixtures of experts," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 11 699–11 732.

[39] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat *et al.*, "GLaM: Efficient scaling of language models with mixture-of-experts," in *International Conference on Machine Learning*. PMLR, 2022, pp. 5547–5569.

[40] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," *Advances in neural information processing systems*, vol. 28, 2015.

[41] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *ICLR*, 2016.

[42] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients," *arXiv preprint arXiv:1606.06160*, 2016.

[43] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2704–2713.

[44] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv preprint arXiv:1806.08342*, 2018.

[45] G. Hinton, O. Vinyals, J. Dean *et al.*, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, vol. 2, no. 7, 2015.

[46] M. Chen, H. Peng, J. Fu, and H. Ling, "AutoFormer: Searching transformers for visual recognition," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 12 270–12 280.

[47] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[48] P. Michel, O. Levy, and G. Neubig, "Are sixteen heads really better than one?" *Advances in neural information processing systems*, vol. 32, 2019.

[49] E. Kurtic, D. Campos, T. Nguyen, E. Frantar, M. Kurtz, B. Fineran, M. Goin, and D. Alistarh, "The Optimal BERT Surgeon: Scalable and Accurate Second-Order Pruning for Large Language Models," *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2022.

[50] O. Zafrir, G. Boudoukh, P. Izsak, and M. Wasserblat, "Q8BERT: Quantized 8Bit BERT," in *Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*, 2019, pp. 36–39.

[51] S. Mehta and M. Rastegari, "MobileViT: light-weight, general-purpose, and mobile-friendly vision transformer," *International Conference on Learning Representations*, 2021.

[52] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *International Conference on Machine Learning*. PMLR, 2021, pp. 10 347–10 357.

[53] H. Wang, Z. Wu, Z. Liu, H. Cai, L. Zhu, C. Gan, and S. Han, "HAT: Hardware-Aware Transformers for Efficient Natural Language Processing," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 7675–7688.

[54] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "ELSA: Hardware-Software co-design for efficient, lightweight self-attention mechanism in neural networks," in *Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 692–705.

[55] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *arXiv preprint arXiv:2009.06732*, 2020.

[56] Q. Fournier, G. M. Caron, and D. Aloise, "A practical survey on faster and lighter transformers," *ACM Computing Surveys*, vol. 55, no. 14s, pp. 1–40, 2023.

[57] S. Kim, C. Hooper, T. Wattanawong, M. Kang, R. Yan, H. Genc, G. Dinh, Q. Huang, K. Keutzer, M. W. Mahoney *et al.*, "Full stack optimization of transformer inference: a survey," *arXiv preprint arXiv:2302.14017*, 2023.

[58] P. Dufter, M. Schmitt, and H. Schütze, "Position information in transformers: An overview," *Computational Linguistics*, vol. 48, no. 3, pp. 733–763, 2022.

[59] A. F. Agarap, "Deep learning using rectified linear units (ReLU)," *arXiv preprint arXiv:1803.08375*, 2018.

[60] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016.

[61] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[62] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *NAACL-HLT*, 2019.

[63] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, "Improving language understanding by generative pre-training," 2018.

[64] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[65] Z. Chuanyang, Z. Li, K. Zhang, Z. Yang, W. Tan, J. Xiao, Y. Ren, and S. Pu, "SAViT: Structure-Aware Vision Transformer Pruning via Collaborative Optimization," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 9010–9023.

[66] E. Frantar and D. Alistarh, "Massive language models can be accurately pruned in one-shot," *arXiv preprint arXiv:2301.00774*, 2023.

[67] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 2158–2170.

[68] J. Mao, H. Yang, A. Li, H. Li, and Y. Chen, "TPrune: Efficient transformer pruning for mobile devices," *ACM Transactions on Cyber-Physical Systems*, vol. 5, no. 3, pp. 1–22, 2021.

[69] P. Dong, M. Sun, A. Lu, Y. Xie, K. Liu, Z. Kong, X. Meng, Z. Li, X. Lin, Z. Fang *et al.*, "Heatvit: Hardware-efficient adaptive token pruning for vision transformers," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 442–455.

[70] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, "TinyBERT: Distilling BERT for Natural Language Understanding," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 4163–4174.

[71] A. Adhikari, A. Ram, R. Tang, W. L. Hamilton, and J. Lin, "Exploring the limits of simple learners in knowledge distillation for document classification with docbert," in *Proceedings of the 5th Workshop on Representation Learning for NLP*, 2020, pp. 72–77.

[72] C. Liang, H. Jiang, Z. Li, X. Tang, B. Yin, and T. Zhao, "HomoDistil: Homotopic Task-Agnostic Distillation of Pre-trained Transformers," *arXiv preprint arXiv:2302.09632*, 2023.

[73] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," *Advances in Neural Information Processing Systems*, vol. 33, pp. 5776–5788, 2020.

[74] S. Yu, T. Chen, J. Shen, H. Yuan, J. Tan, S. Yang, J. Liu, and Z. Wang, "Unified visual transformer compression," *ICLR*, 2022.

[75] S. Hofstätter, S. Althammer, M. Schröder, M. Sertkan, and A. Hanbury, "Improving efficient neural ranking models with cross-architecture knowledge distillation," *arXiv preprint arXiv:2010.02666*, 2020.

[76] L. Hou, Z. Huang, L. Shang, X. Jiang, X. Chen, and Q. Liu, "DynaBERT: Dynamic BERT with adaptive width and depth," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9782–9793, 2020.

[77] X. Chen, Q. Cao, Y. Zhong, J. Zhang, S. Gao, and D. Tao, "DearKD: data-efficient early knowledge distillation for vision transformers," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 052–12 062.

[78] D. Jia, K. Han, Y. Wang, Y. Tang, J. Guo, C. Zhang, and D. Tao, "Efficient vision transformers via fine-grained manifold distillation," *arXiv preprint arXiv:2107.01378*, 2021.

[79] O. Zafrir, A. Larey, G. Boudoukh, H. Shen, and M. Wasserblat, "Prune once for all: Sparse pre-trained language models," *arXiv preprint arXiv:2111.05754*, 2021.

[80] M. Ji, B. Heo, and S. Park, "Show, attend and distill: Knowledge distillation via attention-based feature matching," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 9, 2021, pp. 7945–7952.

[81] V. Sanh, T. Wolf, and A. Rush, "Movement pruning: Adaptive sparsity by fine-tuning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 20 378–20 389, 2020.

[82] H. Chen, Y. Wang, C. Xu, C. Xu, and D. Tao, "Learning student networks via feature embedding," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 25–35, 2020.

[83] A. Nagarajan, S. Sen, J. R. Stevens, and A. Raghunathan, "AxFormer: Accuracy-driven Approximation of Transformers for Faster, Smaller and more Accurate NLP Models," in *International Joint Conference on Neural Networks (IJCNN)*, 2022, pp. 1–8.

[84] R. Cheong and R. Daniel, "transformers. zip: Compressing transformers with pruning and quantization," *Technical report, tech. rep., Stanford University, Stanford, California*, 2019.

[85] M. A. Gordon, K. Duh, and N. Andrews, "Compressing BERT: Studying the effects of weight pruning on transfer learning," *5th Workshop on Representation Learning for NLP*, 2020.

[86] A. Mishra, J. A. Latorre, J. Pool, D. Stosic, D. Stosic, G. Venkatesh, C. Yu, and P. Micikevicius, "Accelerating sparse deep neural networks," *arXiv preprint arXiv:2104.08378*, 2021.

[87] C. Fang, A. Zhou, and Z. Wang, "An algorithm–hardware co-optimized framework for accelerating N:M sparse transformers," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 11, pp. 1573–1586, 2022.

[88] C. Holmes, M. Zhang, Y. He, and B. Wu, "NxMTransformer: Semi-Structured Sparsification for Natural Language Understanding via ADMM," *Advances in Neural Information Processing Systems*, vol. 34, pp. 1818–1830, 2021.

[89] C. Fang, S. Guo, W. Wu, J. Lin, Z. Wang, M. K. Hsu, and L. Liu, "An efficient hardware accelerator for sparse transformer neural networks," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 2670–2674.

[90] M. Xia, Z. Zhong, and D. Chen, "Structured pruning learns compact and accurate models," in *60th Annual Meeting of the Association for Computational Linguistics*, vol. 1, 2022, pp. 1513–1528.

[91] M. Zhu, Y. Tang, and K. Han, "Vision transformer pruning," *arXiv preprint arXiv:2104.08500*, 2021.

[92] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, "Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[93] A. Fan, E. Grave, and A. Joulin, "Reducing transformer depth on demand with structured dropout," *International Conference on Learning*

Representations, 2019.
[94] F. Lagunas, E. Charlaix, V. Sanh, and A. M. Rush, "Block pruning for faster transformers," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 10 619–10 629.
[95] M. Zhu and S. Gupta, "To prune, or not to prune: exploring the efficacy of pruning for model compression," *arXiv preprint arXiv:1710.01878*, 2017.
[96] D. Campos, A. Marques, T. Nguyen, M. Kurtz, and C. Zhai, "Sparse* BERT: Sparse Models are Robust," *arXiv preprint arXiv:2205.12452*, 2022.
[97] Q. Zhang, S. Zuo, C. Liang, A. Bukharin, P. He, W. Chen, and T. Zhao, "Platon: Pruning large transformer models with upper confidence bound of weight importance," in *International Conference on Machine Learning*. PMLR, 2022, pp. 26 809–26 823.
[98] T. Chen, Y. Cheng, Z. Gan, L. Yuan, L. Zhang, and Z. Wang, "Chasing sparsity in vision transformers: An end-to-end exploration," *Advances in Neural Information Processing Systems*, vol. 34, pp. 19 974–19 988, 2021.
[99] F. Yu, K. Huang, M. Wang, Y. Cheng, W. Chu, and L. Cui, "Width & depth pruning for vision transformers," in *AAAI Conference on Artificial Intelligence (AAAI)*, vol. 2022, 2022.
[100] H. Yu and J. Wu, "A unified pruning framework for vision transformers," *Science China Information Sciences*, vol. 66, no. 7, pp. 1–2, 2023.
[101] S. Huang, S. Chen, H. Peng, D. Manu, Z. Kong, G. Yuan, L. Yang, S. Wang, H. Liu, and C. Ding, "HMC-Tran: A Tensor-core Inspired Hierarchical Model Compression for Transformer-based DNNs on GPU," in *Great Lakes Symposium on VLSI*, 2021, pp. 169–174.
[102] P. Ganesh, Y. Chen, X. Lou, M. A. Khan, Y. Yang, H. Sajjad, P. Nakov, D. Chen, and M. Winslett, "Compressing large-scale transformer-based models: A case study on BERT," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 1061–1080, 2021.
[103] S. Kim, S. Shen, D. Thorsley, A. Gholami, W. Kwon, J. Hassoun, and K. Keutzer, "Learned token pruning for transformers," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 784–794.
[104] Y. Tang, K. Han, Y. Wang, C. Xu, J. Guo, C. Xu, and D. Tao, "Patch slimming for efficient vision transformers," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 165–12 174.
[105] K. Luo, H. Li, X. Zhou, and B. Huang, "An attention-based token pruning method for vision transformers," in *International Joint Conference on Rough Sets*. Springer, 2022, pp. 274–288.
[106] T. Yang, H. Ma, X. Li, F. Liu, Y. Zhao, Z. He, and L. Jiang, "DTATrans: Leveraging Dynamic Token-based Quantization with Accuracy Compensation Mechanism for Efficient Transformer Architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.
[107] Z. Song, B. Fu, F. Wu, Z. Jiang, L. Jiang, N. Jing, and X. Liang, "DRQ: Dynamic region-based quantization for deep neural network acceleration," in *International Symposium on Computer Architecture (ISCA)*, 2020, pp. 1010–1021.
[108] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient sparse attention architecture with cascade token and head pruning," in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 97–110.
[109] B. Pan, R. Panda, Y. Jiang, Z. Wang, R. Feris, and A. Oliva, "IA-RED2: Interpretability-Aware Redundancy Reduction for Vision Transformers," *Advances in Neural Information Processing Systems*, vol. 34, pp. 24 898–24 911, 2021.
[110] X. Liu, T. Wu, and G. Guo, "Adaptive Sparse ViT: Towards Learnable Adaptive Token Pruning by Fully Exploiting Self-Attention," *arXiv preprint arXiv:2209.13802*, 2022.
[111] A. Chavan, Z. Shen, Z. Liu, Z. Liu, K.-T. Cheng, and E. P. Xing, "Vision transformer slimming: Multi-dimension searching in continuous optimization space," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 4931–4941.
[112] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, and C.-J. Hsieh, "DynamicViT: Efficient vision transformers with dynamic token sparsification," *Advances in neural information processing systems*, vol. 34, pp. 13 937–13 949, 2021.
[113] Y. Xu, Z. Zhang, M. Zhang, K. Sheng, K. Li, W. Dong, L. Zhang, C. Xu, and X. Sun, "Evo-ViT: Slow-Fast Token Evolution for Dynamic Vision Transformer," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 3, 2022, pp. 2964–2972.
[114] W. Kwon, S. Kim, M. W. Mahoney, J. Hassoun, K. Keutzer, and A. Gholami, "A fast post-training pruning framework for transformers," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 101–24 116, 2022.
[115] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé *et al.*, "BLOOM: A 176B-Parameter Open-Access Multilingual Language Model," *arXiv preprint arXiv:2211.05100*, 2022.
[116] Z. Liu, F. Li, G. Li, and J. Cheng, "EBERT: Efficient BERT Inference with Dynamic Structured Pruning," in *Findings of the Association for Computational Linguistics: ACL-IJCNLP*, 2021, pp. 4814–4823.
[117] H. Fan, T. Chau, S. I. Venieris, R. Lee, A. Kouris, W. Luk, N. D. Lane, and M. S. Abdelfattah, "Adaptable Butterfly Accelerator for Attention-based NNs via Hardware and Algorithm Co-design," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 599–615.
[118] X. Zhang, Y. Wu, P. Zhou, X. Tang, and J. Hu, "Algorithm-hardware Co-design of Attention Mechanism on FPGA Devices," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 20, no. 5, pp. 1–24, 2021.
[119] S. Lu, M. Wang, S. Liang, J. Lin, and Z. Wang, "Hardware accelerator for multi-head attention and position-wise feed-forward in the transformer," in *2020 IEEE 33rd International System-on-Chip Conference (SOCC)*, 2020, pp. 84–89.
[120] H. Li, J. Choi, and J. Ahn, "A Slice and Dice Approach to Accelerate Compound Sparse Attention on GPU," in *IEEE International Symposium on Workload Characterization (IISWC)*, 2022.
[121] P. Qi, Y. Song, H. Peng, S. Huang, Q. Zhuge, and E. H.-M. Sha, "Accommodating transformer onto FPGA: Coupling the balanced model compression and FPGA-implementation optimization," in *Great Lakes Symposium on VLSI*, 2021, pp. 163–168.
[122] B. Li, Z. Kong, T. Zhang, J. Li, Z. Li, H. Liu, and C. Ding, "Efficient transformer-based large scale language representations using hardware-friendly block structured pruning," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 3187–3199.
[123] S. Narang, E. Undersander, and G. Diamos, "Block-sparse recurrent neural networks," *arXiv preprint arXiv:1711.02782*, 2017.
[124] H. Peng, S. Huang, T. Geng, A. Li, W. Jiang, H. Liu, S. Wang, and C. Ding, "Accelerating transformer-based deep learning models on fpgas using column balanced block pruning," in *International Symposium on Quality Electronic Design (ISQED)*, 2021, pp. 142–148.
[125] Z. Yuan, C. Xue, Y. Chen, Q. Wu, and G. Sun, "PTQ4ViT: Post-training quantization for vision transformers with twin uniform quantization," in *European Conference on Computer Vision*. Springer, 2022, pp. 191–207.
[126] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "GOBO: Quantizing attention-based NLP models for low latency and energy efficient inference," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 811–824.
[127] N. Wang, C.-C. Liu, S. Venkataramani, S. Sen, C.-Y. Chen, K. El Maghraoui, V. Srinivasan, and L. Chang, "Deep compression of pre-trained transformer models," in *Advances in Neural Information Processing Systems*, 2022.
[128] Z. Liu, Y. Wang, K. Han, W. Zhang, S. Ma, and W. Gao, "Post-training quantization for vision transformer," *Advances in Neural Information Processing Systems*, vol. 34, pp. 28 092–28 103, 2021.
[129] Z. Li, M. Chen, J. Xiao, and Q. Gu, "PSAQ-ViT V2: Towards Accurate and General Data-Free Quantization for Vision Transformers," *arXiv preprint arXiv:2209.05687*, 2022.
[130] N. Frumkin, D. Gope, and D. Marculescu, "CPT-V: A Contrastive Approach to Post-Training Quantization of Vision Transformers," *arXiv preprint arXiv:2211.09643*, 2022.
[131] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," *arXiv preprint arXiv:2208.07339*, 2022.
[132] A. H. Zadeh, M. Mahmoud, A. Abdelhadi, and A. Moshovos, "Mokey: enabling narrow fixed-point inference for out-of-the-box floating-point transformer models," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 888–901.
[133] B. Li, S. Lu, K. Xie, and Z. Wang, "Accelerating NLP Tasks on FPGA with Compressed BERT and a Hardware-Oriented Early Exit Method," in *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2022, pp. 410–413.
[134] A. Rock, A. Untether, O. Khalil, O. Shai, and P. Grouchy, "INT8 Transformers for Inference Acceleration," *36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.
[135] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "HAQ: Hardware-aware automated quantization with mixed precision," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp.

8612–8620.
[136] M. Nagel, M. v. Baalen, T. Blankevoort, and M. Welling, "Data-free quantization through weight equalization and bias correction," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1325–1334.
[137] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.
[138] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.
[139] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan, "PACT: Parameterized clipping activation for quantized neural networks," *arXiv preprint arXiv:1805.06085*, 2018.
[140] S. Jung, C. Son, S. Lee, J. Son, J.-J. Han, Y. Kwak, S. J. Hwang, and C. Choi, "Learning to quantize deep networks by optimizing quantization intervals with task loss," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4350–4359.
[141] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," *International Conference on Learning Representations*, 2019.
[142] Z. Liu, G. Li, and J. Cheng, "Hardware acceleration of fully quantized bert for efficient natural language processing," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 513–516.
[143] Z. Li, L. Ma, M. Chen, J. Xiao, and Q. Gu, "Patch similarity aware data-free quantization for vision transformers," in *European Conference on Computer Vision*. Springer, 2022, pp. 154–170.
[144] J. Choi, S. Venkataramani, V. V. Srinivasan, K. Gopalakrishnan, Z. Wang, and P. Chuang, "Accurate and efficient 2-bit quantized neural networks," *Proceedings of Machine Learning and Systems*, vol. 1, pp. 348–359, 2019.
[145] T. J. Ham, S. J. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J.-H. Park, S. Lee, K. Park, J. W. Lee *et al.*, "A^ 3: Accelerating attention mechanisms in neural networks with approximation," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 328–341.
[146] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "Zeroquant: Efficient and affordable post-training quantization for large-scale transformers," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 168–27 183, 2022.
[147] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "Q-BERT: Hessian based ultra low precision quantization of BERT," in *AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, 2020, pp. 8815–8821.
[148] Y. Bondarenko, M. Nagel, and T. Blankevoort, "Understanding and overcoming the challenges of efficient transformer quantization," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 7947–7969.
[149] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "HAWQ: Hessian aware quantization of neural networks with mixed-precision," in *IEEE/CVF International Conference on Computer Vision*, 2019, pp. 293–302.
[150] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-BERT: Integer-only BERT quantization," in *International conference on machine learning*. PMLR, 2021, pp. 5506–5518.
[151] Z. Li and Q. Gu, "I-vit: integer-only quantization for efficient vision transformer inference," *arXiv preprint arXiv:2207.01405*, 2022.
[152] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. Mahoney *et al.*, "HAWQ-v3: Dyadic neural network quantization," in *International Conference on Machine Learning*. PMLR, 2021, pp. 11 875–11 886.
[153] Y. Lin, T. Zhang, P. Sun, Z. Li, and S. Zhou, "FQ-ViT: Post-Training Quantization for Fully Quantized Vision Transformer," in *International Joint Conference on Artificial Intelligence, IJCAI-22*, 2022, pp. 1173–1179.
[154] A. Marchisio, D. Dura, M. Capra, M. Martina, G. Masera, and M. Shafique, "SwiftTron: An Efficient Hardware Accelerator for Quantized Transformers," *arXiv preprint arXiv:2304.03986*, 2023.
[155] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
[156] Z. Liu, B. Wu, W. Luo, X. Yang, W. Liu, and K.-T. Cheng, "Bi-Real Net: Enhancing the Performance of 1-bit CNNs With Improved Representational Capability and Advanced Training Algorithm," in *European conference on computer vision (ECCV)*, 2018, pp. 722–737.
[157] H. Qin, R. Gong, X. Liu, M. Shen, Z. Wei, F. Yu, and J. Song, "Forward and backward information retention for accurate binary neural networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2250–2259.
[158] H. Qin, Y. Ding, M. Zhang, Q. Yan, A. Liu, Q. Dang, Z. Liu, and X. Liu, "BiBERT: Accurate fully binarized BERT," *ICLR*, 2022.
[159] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," *arXiv preprint arXiv:1804.07461*, 2018.
[160] Z. Liu, B. Oguz, A. Pappu, L. Xiao, S. Yih, M. Li, R. Krishnamoorthi, and Y. Mehdad, "Bit: Robustly binarized multi-distilled transformer," *Advances in neural information processing systems*, vol. 35, pp. 14 303–14 316, 2022.
[161] X. Wu, Z. Yao, M. Zhang, C. Li, and Y. He, "Xtc: Extreme compression for pre-trained transformers made simple and efficient," *Advances in Neural Information Processing Systems*, vol. 35, pp. 3217–3231, 2022.
[162] H. Bai, W. Zhang, L. Hou, L. Shang, J. Jin, X. Jiang, Q. Liu, M. Lyu, and I. King, "BinaryBERT: Pushing the Limit of BERT Quantization," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, vol. 1, 2021, pp. 4334–4348.
[163] J. Tian, C. Fang, H. Wang, and Z. Wang, "BEBERT: Efficient and robust binary ensemble BERT," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
[164] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen *et al.*, "A study of bfloat16 for deep learning training," *arXiv preprint arXiv:1905.12322*, 2019.
[165] B. Darvish Rouhani, D. Lo, R. Zhao, M. Liu, J. Fowers, K. Ovtcharov, A. Vinogradsky, S. Massengill, L. Yang, R. Bittner *et al.*, "Pushing the limits of narrow precision inferencing at cloud scale with microsoft floating point," *Advances in neural information processing systems*, vol. 33, pp. 10 271–10 281, 2020.
[166] J. Choquette, W. Gandhi, O. Giroux, N. Stam, and R. Krashinsky, "Nvidia a100 tensor core gpu: Performance and innovation," *IEEE Micro*, vol. 41, no. 2, pp. 29–35, 2021.
[167] M. van Baalen, A. Kuzmin, S. S. Nair, Y. Ren, E. Mahurin, C. Patel, S. Subramanian, S. Lee, M. Nagel, J. Soriaga *et al.*, "Fp8 versus int8 for efficient deep learning inference," *arXiv preprint arXiv:2303.17951*, 2023.
[168] A. Kuzmin, M. Van Baalen, Y. Ren, M. Nagel, J. Peters, and T. Blankevoort, "Fp8 quantization: The power of the exponent," *arXiv preprint arXiv:2208.09225*, 2022.
[169] N. Kitaev, Ł. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," *International Conference on Learning Representations*, 2020.
[170] Z. Qin, W. Sun, H. Deng, D. Li, Y. Wei, B. Lv, J. Yan, L. Kong, and Y. Zhong, "CosFormer: Rethinking softmax in attention," *ICLR*, 2022.
[171] M. Maaz, A. Shaker, H. Cholakkal, S. Khan, S. W. Zamir, R. M. Anwer, and F. Shahbaz Khan, "Edgenext: efficiently amalgamated cnn-transformer architecture for mobile vision applications," in *European Conference on Computer Vision*. Springer, 2022, pp. 3–20.
[172] S. Mehta and M. Rastegari, "Separable self-attention for mobile vision transformers," *Transactions on Machine Learning Research*, 2022.
[173] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser *et al.*, "Rethinking attention with Performers," *International Conference on Learning Representations*, 2020.
[174] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," *arXiv preprint arXiv:2006.04768*, 2020.
[175] Z. Lu, C. Ding, F. Juefei-Xu, V. N. Boddeti, S. Wang, and Y. Yang, "TFormer: A Transmission-Friendly ViT Model for IoT Devices," *IEEE Transactions on Parallel and Distributed Systems*, 2022.
[176] A. Srinivas, T.-Y. Lin, N. Parmar, J. Shlens, P. Abbeel, and A. Vaswani, "Bottleneck transformers for visual recognition," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 16 519–16 529.
[177] C. Yang, Y. Wang, J. Zhang, H. Zhang, Z. Wei, Z. Lin, and A. Yuille, "Lite vision transformer with enhanced self-attention," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 998–12 008.
[178] Y. Chen, X. Dai, D. Chen, M. Liu, X. Dong, L. Yuan, and Z. Liu, "Mobile-Former: Bridging mobilenet and transformer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recog-*

nition, 2022, pp. 5270–5279.
[179] Y. Li, G. Yuan, Y. Wen, J. Hu, G. Evangelidis, S. Tulyakov, Y. Wang, and J. Ren, "EfficientFormer: Vision transformers at mobilenet speed," *Advances in Neural Information Processing Systems*, vol. 35, pp. 12 934–12 949, 2022.
[180] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.
[181] Y. Chen, X. Dai, M. Liu, D. Chen, L. Yuan, and Z. Liu, "Dynamic ReLU," in *European Conference on Computer Vision*. Springer, 2020, pp. 351–367.
[182] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao, "Pyramid vision transformer: A versatile backbone for dense prediction without convolutions," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 568–578.
[183] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
[184] L. Yuan, Y. Chen, T. Wang, W. Yu, Y. Shi, Z.-H. Jiang, F. E. Tay, J. Feng, and S. Yan, "Tokens-to-token ViT: Training vision transformers from scratch on imagenet," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 558–567.
[185] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
[186] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan *et al.*, "Searching for mobilenetv3," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 1314–1324.
[187] K. T. Chitty-Venkata and A. K. Somani, "Neural architecture search survey: A hardware perspective," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–36, 2022.
[188] K. T. Chitty-Venkata, M. Emani, V. Vishwanath, and A. K. Somani, "Neural architecture search for transformers: A survey," *IEEE Access*, vol. 10, pp. 108 374–108 412, 2022.
[189] J. Liu, X. Huang, G. Song, H. Li, and Y. Liu, "UniNet: Unified Architecture Search with Convolution, Transformer, and MLP," in *European Conference on Computer Vision*. Springer, 2022, pp. 33–49.
[190] X. Su, S. You, J. Xie, M. Zheng, F. Wang, C. Qian, C. Zhang, X. Wang, and C. Xu, "ViTAS: Vision Transformer Architecture Search," in *European Conference on Computer Vision*. Springer, 2022, pp. 139–157.
[191] C. Gong, D. Wang, M. Li, X. Chen, Z. Yan, Y. Tian, V. Chandra *et al.*, "NASViT: Neural Architecture Search for Efficient Vision Transformers with Gradient Conflict aware Supernet Training," in *International Conference on Learning Representations*, 2021.
[192] H. Zhang, K. Hao, W. Pedrycz, L. Gao, X. Tang, and B. Wei, "Vision transformer with convolutions architecture search," *arXiv preprint arXiv:2203.10435*, 2022.
[193] Y. Zhao, L. Dong, Y. Shen, Z. Zhang, F. Wei, and W. Chen, "Memory-efficient differentiable transformer architecture search," *arXiv preprint arXiv:2105.14669*, 2021.
[194] X. Shi, P. Zhou, W. Chen, and L. Xie, "Efficient gradient-based neural architecture search for end-to-end ASR," in *Companion Publication of the 2021 International Conference on Multimodal Interaction*, 2021, pp. 91–96.
[195] H. You, B. Li, S. Huihong, Y. Fu, and Y. Lin, "ShiftAddNAS: Hardware-inspired search for more accurate and efficient neural networks," in *International Conference on Machine Learning*. PMLR, 2022, pp. 25 566–25 580.
[196] C. Zhao, T. Hua, Y. Shen, Q. Lou, and H. Jin, "Automatic Mixed-Precision Quantization Search of BERT," *IJCAI*, 2021.
[197] S. Latifi, S. Muralidharan, and M. Garland, "Efficient sparsely activated transformers," *arXiv preprint arXiv:2208.14580*, 2022.
[198] D. So, Q. Le, and C. Liang, "The evolved transformer," in *International Conference on Machine Learning*. PMLR, 2019, pp. 5877–5886.
[199] D. So, W. Mańke, H. Liu, Z. Dai, N. Shazeer, and Q. V. Le, "Searching for efficient transformers for language modeling," *Advances in Neural Information Processing Systems*, vol. 34, pp. 6010–6022, 2021.
[200] H. Benmeziane, H. Ouarnoughi, K. E. Maghraoui, and S. Niar, "Real-time style transfer with efficient vision transformers," in *Proceedings of the 5th International Workshop on Edge Systems, Analytics and Networking*, 2022, pp. 31–36.
[201] Y.-L. Liao, S. Karaman, and V. Sze, "Searching for efficient multi-stage vision transformers," *arXiv preprint arXiv:2109.00642*, 2021.
[202] M. Chen, K. Wu, B. Ni, H. Peng, B. Liu, J. Fu, H. Chao, and H. Ling, "Searching the search space of vision transformer," *Advances in Neural Information Processing Systems*, vol. 34, 2021.
[203] B. Chen, P. Li, C. Li, B. Li, L. Bai, C. Lin, M. Sun, J. Yan, and W. Ouyang, "GLiT: Neural architecture search for global and local image transformer," in *IEEE/CVF International Conference on Computer Vision*, 2021, pp. 12–21.
[204] L. Yang, Y. Hu, S. Lu, Z. Sun, J. Mei, Y. Han, and X. Li, "Searching for BurgerFormer with Micro-Meso-Macro Space Design," in *International Conference on Machine Learning*. PMLR, 2022, pp. 25 055–25 069.
[205] Y. Yin, C. Chen, L. Shang, X. Jiang, X. Chen, and Q. Liu, "AutoTinyBERT: Automatic Hyper-parameter Optimization for Efficient Pre-trained Language Models," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021, pp. 5146–5157.
[206] R. Wang, Q. Bai, J. Ao, L. Zhou, Z. Xiong, Z. Wei, Y. Zhang, T. Ko, and H. Li, "LightHuBERT: Lightweight and Configurable Speech Representation Learning with Once-for-All Hidden-Unit BERT," *arXiv preprint arXiv:2203.15610*, 2022.
[207] X. Zhang, Z. Zhou, D. Chen, and Y. E. Wang, "AutoDistill: an End-to-End Framework to Explore and Distill Hardware-Efficient Language Models," *arXiv preprint arXiv:2201.08539*, 2022.
[208] T. Kim, H. Myeong, and S.-Y. Yun, "Revisiting architecture-aware knowledge distillation: Smaller models and faster search," *arXiv preprint arXiv:2206.13130*, 2022.
[209] R. Luo, X. Tan, R. Wang, T. Qin, J. Li, S. Zhao, E. Chen, and T.-Y. Liu, "LightSpeech: Lightweight and fast text to speech with neural architecture search," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 5699–5703.
[210] Q. Zhou, K. Sheng, X. Zheng, K. Li, X. Sun, Y. Tian, J. Chen, and R. Ji, "Training-free transformer architecture search," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10 894–10 903.
[211] M. Javaheripi, S. Shah, S. Mukherjee, T. L. Religa, C. C. Mendes, G. H. de Rosa, S. Bubeck, F. Koushanfar, and D. Dey, "LiteTransformerSearch: Training-free On-device Search for Efficient Autoregressive Language Models," *AutoML Workshop*, 2022.
[212] B. Ni, G. Meng, S. Xiang, and C. Pan, "NASformer: Neural Architecture Search for Vision Transformer," in *Asian Conference on Pattern Recognition*. Springer, 2022, pp. 47–61.
[213] W. Chen, W. Huang, X. Du, X. Song, Z. Wang, and D. Zhou, "Auto-scaling vision transformers without training," *ICLR*, 2022.
[214] W. Zhu, X. Wang, Y. Ni, and G. Xie, "Autotrans: Automating transformer design via reinforced architecture search," in *CCF International Conference on Natural Language Processing and Chinese Computing*. Springer, 2021, pp. 169–182.
[215] D. Chen, Y. Li, M. Qiu, Z. Wang, B. Li, B. Ding, H. Deng, J. Huang, W. Lin, and J. Zhou, "AdaBERT: Task-adaptive BERT compression with differentiable neural architecture search," *IJCAI*, 2020.
[216] J. Xu, X. Tan, R. Luo, K. Song, J. Li, T. Qin, and T.-Y. Liu, "NAS-BERT: task-agnostic and adaptive-size BERT compression with neural architecture search," in *27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 1933–1943.
[217] J. Gao, H. Xu, H. Shi, X. Ren, L. Philip, X. Liang, X. Jiang, and Z. Li, "AutoBERT-zero: Evolving BERT backbone from scratch," in *AAAI Conference on Artificial Intelligence*, vol. 36, no. 10, 2022, pp. 10 663–10 671.
[218] Y. Wang, Y. Yang, Y. Chen, J. Bai, C. Zhang, G. Su, X. Kou, Y. Tong, M. Yang, and L. Zhou, "TextNAS: A neural architecture search space tailored for text representation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, 2020, pp. 9242–9249.
[219] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," *International Conference on Learning Representations*, 2016.
[220] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," *International Conference on Learning Representations*, 2018.
[221] Y. Chen, G. Meng, Q. Zhang, S. Xiang, C. Huang, L. Mu, and X. Wang, "RENAS: Reinforced evolutionary neural architecture search," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4787–4796.
[222] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," *International Conference on Learning Representations*, 2019.
[223] L. Li and A. Talwalkar, "Random search and reproducibility for neural architecture search," in *Uncertainty in artificial intelligence*. PMLR,

2020, pp. 367–377.
[224] J. Mellor, J. Turner, A. Storkey, and E. J. Crowley, "Neural architecture search without training," in *International Conference on Machine Learning*. PMLR, 2021, pp. 7588–7598.
[225] C. White, W. Neiswanger, and Y. Savani, "BANANAS: Bayesian optimization with neural architectures for neural architecture search," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, 2021, pp. 10 293–10 301.
[226] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit *et al.*, "MLP-Mixer: An all-MLP architecture for vision," *Advances in Neural Information Processing Systems*, vol. 34, pp. 24 261–24 272, 2021.
[227] W. Niu, Z. Kong, G. Yuan, W. Jiang, J. Guan, C. Ding, P. Zhao, S. Liu, B. Ren, and Y. Wang, "Real-time execution of large-scale language models on mobile," *arXiv preprint arXiv:2009.06823*, 2020.
[228] A. N. Gomez, M. Ren, R. Urtasun, and R. B. Grosse, "The reversible residual network: Backpropagation without storing activations," *Advances in neural information processing systems*, vol. 30, 2017.
[229] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, and K. Keutzer, "FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable Neural Architecture Search," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 10 734–10 742.
[230] S. Zhang, X. Zheng, C. Yang, Y. Li, Y. Wang, F. Chao, M. Wang, S. Li, J. Yang, and R. Ji, "You only compress once: Towards effective and elastic bert compression via exploit-explore stochastic nature gradient," *arXiv preprint arXiv:2106.02435*, 2021.
[231] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3451–3460, 2021.
[232] M. S. Abdelfattah, A. Mehrotra, Ł. Dudziak, and N. D. Lane, "Zero-cost proxies for lightweight NAS," *ICLR*, 2021.
[233] S. Huang, N. Liu, Y. Liang, H. Peng, H. Li, D. Xu, M. Xie, and C. Ding, "An Automatic and Efficient BERT Pruning for Edge AI Systems," in *2022 23rd International Symposium on Quality Electronic Design (ISQED)*. IEEE, 2022, pp. 1–6.
[234] Z. Liu, D. Li, K. Lu, Z. Qin, W. Sun, J. Xu, and Y. Zhong, "Neural architecture search on efficient transformers and beyond," *arXiv preprint arXiv:2207.13955*, 2022.
[235] H. Chen, Y. Wang, C. Xu, B. Shi, C. Xu, Q. Tian, and C. Xu, "AdderNet: Do we really need multiplications in deep learning?" in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 1468–1477.
[236] H. You, X. Chen, Y. Zhang, C. Li, S. Li, Z. Liu, Z. Wang, and Y. Lin, "ShiftAddNet: A hardware-inspired deep network," *Advances in Neural Information Processing Systems*, vol. 33, pp. 2771–2783, 2020.
[237] K. Sreedhar, J. Clemons, R. Venkatesan, S. W. Keckler, and M. Horowitz, "Enabling and accelerating dynamic vision transformer inference for real-time applications," *arXiv preprint arXiv:2212.02687*, 2022.
[238] Y. Li, Y. Hu, F. Wu, and K. Li, "DiVIT: Algorithm and architecture co-design of differential attention in vision transformer," *Journal of Systems Architecture*, vol. 128, p. 102520, 2022.
[239] Y. Wang, Y. Qin, D. Deng, J. Wei, Y. Zhou, Y. Fan, T. Chen, H. Sun, L. Liu, S. Wei *et al.*, "A 28nm 27.5 TOPS/W approximate-computing-based transformer processor with asymptotic sparsity speculating and out-of-order computing," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 1–3.
[240] D. Ma, X. Qin, and X. Jiao, "AxBy-ViT: Reconfigurable Approximate Computation Bypass for Vision Transformers," in *International Symposium on Quality Electronic Design (ISQED)*, 2022, pp. 1–5.
[241] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, "DOTA: detect and omit weak attentions for scalable transformer acceleration," in *ASPLOS*, 2022, pp. 14–26.
[242] Q. Chen, C. Sun, Z. Lu, and C. Gao, "Enabling energy-efficient inference for self-attention mechanisms in neural networks," in *IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2022, pp. 25–28.
[243] H. Peng, S. Huang, S. Chen, B. Li, T. Geng, A. Li, W. Jiang, W. Wen, J. Bi, H. Liu *et al.*, "A length adaptive algorithm-hardware co-design of transformer on FPGA through sparse attention and dynamic pipelining," in *Design Automation Conference*, 2022, pp. 1135–1140.
[244] W. Ye, X. Zhou, J. T. Zhou, C. Chen, and K. Li, "Accelerating attention mechanism on FPGAs based on efficient reconfigurable systolic array," *ACM Transactions on Embedded Computing Systems (TECS)*, 2022.
[245] G. Shen, J. Zhao, Q. Chen, J. Leng, C. Li, and M. Guo, "SALO: an efficient spatial accelerator enabling hybrid sparse attention mechanisms for long sequences," in *Design Automation Conference*, 2022, pp. 571–576.
[246] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 977–991.
[247] Z. Zhao, R. Cao, K.-F. Un, W.-H. Yu, P.-I. Mak, and R. P. Martins, "An FPGA-Based Transformer Accelerator Using Output Block Stationary Dataflow for Object Recognition Applications," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2022.
[248] H. You, Z. Sun, H. Shi, Z. Yu, Y. Zhao, Y. Zhang, C. Li, B. Li, and Y. Lin, "ViTCoD: Vision Transformer Acceleration via Dedicated Algorithm and Accelerator Co-Design," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 273–286.
[249] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, "DFX: A Low-latency Multi-FPGA Appliance for Accelerating Transformer-based Text Generation," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 616–630.
[250] B. Li, S. Pandey, H. Fang, Y. Lyv, J. Li, J. Chen, M. Xie, L. Wan, H. Liu, and C. Ding, "FTRANS: energy-efficient acceleration of transformers using FPGA," in *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*, 2020, pp. 175–180.
[251] R. Rizk, D. Rizk, F. Rizk, A. Kumar, and M. Bayoumi, "A Resource-Saving Energy-Efficient Reconfigurable Hardware Accelerator for BERT-based Deep Neural Network Language Models using FFT Multiplication," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 1675–1679.
[252] X. Yang and T. Su, "EFA-Trans: An Efficient and Flexible Acceleration Architecture for Transformers," *Electronics*, vol. 11, no. 21, p. 3550, 2022.
[253] H.-Y. Chang, S. H. Mozafari, C. Chen, J. J. Clark, B. H. Meyer, and W. J. Gross, "PipeBERT: High-throughput BERT Inference for ARM Big. LITTLE Multi-core Processors," *Journal of Signal Processing Systems*, pp. 1–18, 2022.
[254] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," *Advances in Neural Information Processing Systems*, vol. 34, pp. 12 077–12 090, 2021.
[255] E. Xie, W. Wang, W. Wang, P. Sun, H. Xu, D. Liang, and P. Luo, "Segmenting transparent object in the wild with transformer," *International Joint Conference on Artificial Intelligence*, 2021.
[256] S. Zheng, J. Lu, H. Zhao, X. Zhu, Z. Luo, Y. Wang, Y. Fu, J. Feng, T. Xiang, P. H. Torr *et al.*, "Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers," in *IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 6881–6890.
[257] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable DETR: Deformable transformers for end-to-end object detection," *International Conference on Learning Representations*, 2020.
[258] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-pragmatic deep neural network computing," in *IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 382–394.
[259] K. T. Chitty-Venkata, M. Emani, V. Vishwanath, and A. K. Somani, "Neural architecture search benchmarks: Insights and survey," *IEEE Access*, 2023.
[260] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter, "NAS-bench-101: Towards reproducible neural architecture search," in *International Conference on Machine Learning*. PMLR, 2019, pp. 7105–7114.
[261] C. Li, Z. Yu, Y. Fu, Y. Zhang, Y. Zhao, H. You, Q. Yu, Y. Wang, and Y. Lin, "HW-NAS-Bench: Hardware-aware neural architecture search benchmark," *ICLR*, 2021.
[262] H. Cao, Y. Wang, J. Chen, D. Jiang, X. Zhang, Q. Tian, and M. Wang, "Swin-Unet: Unet-like Pure Transformer for Medical Image Segmentation," in *European conference on computer vision*. Springer, 2022, pp. 205–218.
[263] V. P. Dwivedi and X. Bresson, "A generalization of transformer networks to graphs," *arXiv preprint arXiv:2012.09699*, 2020.
[264] X. Dong, C. Long, W. Xu, and C. Xiao, "Dual graph convolutional networks with transformer and curriculum learning for image captioning," in *Proceedings of the 29th ACM International Conference on Multimedia*, 2021, pp. 2615–2624.
[265] C. Cao, Y. Zhang, Y. Wu, H. Lu, and J. Cheng, "Egocentric gesture

recognition using recurrent 3d convolutional neural networks with spatiotemporal transformer modules," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 3763–3771.
[266] M.-H. Guo *et al.*, "PCT: Point cloud transformer," *Computational Visual Media*, vol. 7, no. 2, pp. 187–199, 2021.
[267] E. Mueller, V. Studenyak, D. Auge, and A. Knoll, "Spiking transformer networks: A rate coded approach for processing sequential data," in *2021 7th International Conference on Systems and Informatics (ICSAI)*. IEEE, 2021, pp. 1–5.
[268] E. Iofinova, A. Peste, M. Kurtz, and D. Alistarh, "How well do sparse ImageNet models transfer?" in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 266–12 276.
[269] S. Mittal, "A Survey on Modeling and Improving Reliability of DNN Algorithms and Accelerators," *Journal of Systems Architecture*, vol. 104, p. 101689, 2020.
[270] S. Mittal, H. Gupta, and S. Srivastava, "A Survey on Hardware Security of DNN Models and Accelerators," *Journal of Systems Architecture*, 2021.



Krishna Teja Chitty-Venkata is a PhD student at Iowa State University, Ames, Iowa, USA. He received his Bachelor's of Engineering degree in Electronics and Communication from University College of Engineering, Osmania University, Hyderabad, India in 2017. His research interests include designing network design algorithms and compression methods for Neural Network processing, such as Pruning, Quantization and Neural Architecture Search, for General and Special purpose hardware platforms. He interned at Argonne National Laboratory, Intel Corporation and AMD where he worked on various problems on efficient Deep Learning.



Sparsh Mittal received the B.Tech. degree from IIT, Roorkee, India, and the Ph.D. degree from Iowa State University (ISU), USA. He worked as a Postdoctoral Research Associate with the Oak Ridge National Laboratory (ORNL), USA. He was as an Assistant Professor at IIT Hyderabad. He is currently working as an Assistant Professor with the Department of Electronics and Communication Engineering, IIT Roorkee, and as a Joint Faculty with the Mehta Family School of Data Science and Artificial Intelligence. He was the graduating topper of his batch in B.Tech. degree and has received fellowship from ISU and performance award from ORNL. He has published more than 110 research papers at top venues. His research has been covered by insideHPC, HPCwire, Phys.org, and scientific computing. He has received research funding from SERB, Intel, and SRC (USA).



Murali Emani is an Assistant Computer Scientist in the Data Science group with the Argonne Leadership Computing Facility (ALCF) at Argonne National Laboratory. Prior to this, he was a Postdoctoral Research Staff Member at Lawrence Livermore National Laboratory. He obtained his Ph.D. from the School of Informatics, University of Edinburgh, UK in 2015. His research interests are in scalable machine learning, emerging HPC and AI architectures, AI for Science. He serves as a co-chair for MLPerf HPC group at MLCommons to benchmark large scale ML on HPC systems. He also co-leads the AI Testbed at ALCF to evaluate novel AI accelerators for scientific machine learning applications. Murali has organized workshops and participated in turorials that include benchmarking deep learning workloads on emerging hardware, MLPerf-Bench at MLSys'20, MLSys'21, ISPASS'20, ISPASS'21, ASPLOS'21. He has also co-chaired the MLPerf birds-of-a-feather sessions at SC'19, SC'20 and SC'21.



Venkatram Vishwanath is a computer scientist at Argonne National Laboratory. He is the Data Science Team Lead at the Argonne leadership computing facility (ALCF). His current focus is on algorithms, system software, and workflows to facilitate data-centric applications on supercomputing systems. His interests include scientific applications, supercomputing architectures, parallel algorithms and runtimes, scalable analytics and collaborative workspaces. He has received best papers awards at venues including HPDC and LDAV, and a Gordon Bell winner. Vishwanath received his Ph.D. in computer science from the University of Illinois at Chicago in 2009.



Arun K. Somani received M.S. and Ph.D. degrees in Electrical Engineering from McGill University, Montreal, in 1983 and 1985, respectively. He served as a Scientific Officer with the Government of India, New Delhi, and a Faculty Member with the University of Washington, Seattle. He is currently an Anson Marston Distinguished Professor of Electrical and Computer Engineering with Iowa State University. His research interests are in the areas of computer system design, architecture, fault tolerant computing, computer interconnection networks, optical networking, reconfigurable and parallel computer systems. He served as IEEE Distinguished Visitor, IEEE Distinguished Tutorial Speaker, and an IEEE Communication Society Distinguished Visitor. He delivered several keynote speeches, and distinguished and invited talks all over the world. He is a Life Fellow of IEEE, Distinguished Engineer of ACM, Eminent Engineer of Tau Beta Pi, and a fellow of AAAS.

Source File (Latex or word file)



Click here to access/download

Source File (Latex or word file)

UploadFolder.zip

Declaration of Conflict of Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.